



Rapport de Projet de Fin d'Etudes

LICENCE EN TECHNOLOGIES DE L'INFORMATIQUE

PARCOURS : SYSTEMES EMBARQUES ET MOBILES (SEM)

Système de Gestion et de Supervision de Parking « MacroPark »

Entreprise : Scheidt & Bachmann Maghreb

Réalisé par :

- Manai Yassine
- Zayati Sofiene

Encadré par :

Encadrant entreprise : Mr. Bradai Zied

Encadrant ISET : Mr. Melki Sami

Code PFE : SEM-24-04

Année universitaire : 2023/2024

Remerciements

Avant d'entamer l'exposition de notre travail, nous sommes très heureux de réserver cette page en signe de gratitude et de profonde reconnaissance à tous ceux qui ont contribué directement ou indirectement à la réussite de notre projet de fin d'études. Nous souhaitons exprimer notre profonde gratitude au corps professoral de l'Institut Supérieur des Études Technologiques de Rades (**ISET Rades**) pour les bases techniques qui nous ont été inculquées au cours de notre formation et qui nous ont permis d'avoir une approche analytique beaucoup plus raffinée lors de notre travail d'étude.

Nous tenons tout particulièrement à remercier notre encadrant pédagogique, **Monsieur Melki Sami**, pour son accueil chaleureux à chaque fois que nous avons sollicité son aide, ainsi que pour ses multiples encouragements tout au long de notre projet. Nous aimerions également lui dire à quel point nous avons apprécié sa grande disponibilité et son respect sans faille des délais serrés de relecture des documents que nous lui avons adressés.

Nous exprimons nos sincères remerciements à toute l'équipe du service des systèmes d'information à **Scheidt & Bachmann Maghreb** et plus particulièrement, nous tenons à remercier notre encadrant professionnel, **Monsieur Bradai Zied**, pour la confiance qu'il nous a accordée en acceptant d'encadrer ce travail, pour ses multiples conseils et pour le temps qu'il a bien voulu consacrer à nous aider par son expérience et ses compétences. Nous avons été extrêmement reconnaissants de ses qualités d'écoute et de compréhension tout au long de ce travail.

À tous les membres du jury, nous espérons que notre travail sera à la hauteur de vos attentes. Nous vous remercions pour l'honneur que vous nous avez fait en assistant à notre soutenance et en acceptant de le juger. À toutes celles et ceux qui de près ou de loin ont contribué à la tenue et à la réussite de ce travail, veuillez recevoir ici l'expression de notre reconnaissance.

Dédicace

Nous dédions ce rapport de Projet de Fin d'Études à toutes les personnes qui ont contribué à notre parcours académique et professionnel.

À nos parents, pour leur amour inconditionnel, leur soutien sans faille et leurs encouragements constants. Votre confiance en nous a été notre plus grande source de motivation.

À nos professeurs et encadrants, pour leur patience, leurs conseils avisés et leur expertise. Vous nous avez aidés à acquérir les compétences nécessaires et à surmonter les défis rencontrés tout au long de ce projet.

Enfin, nous tenons à remercier toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce projet. Votre soutien et votre bienveillance ont été essentiels pour mener à bien ce travail.

TABLE DES MATIERES

INTRODUCTION GENERALE.....	1
CHAPITRE 1	3
ÉTAT DE L'ART	3
1. Introduction.....	4
2. Présentation de l'organisme d'accueil.....	4
2.1.Introduction	4
2.2.Solutions	4
2.3.Valeurs.....	4
2.4.Objectifs	5
2.5 Organigramme de la société	6
2.6 Historique de Scheidt & Bachmann GmbH.....	7
2.7 Cadre du projet	7
3. Etude de l'existant	8
3.1. Problématique.....	8
3.2. Solution	8
4. Méthodologie de travail.....	9
4.1 Comparaison entre les différentes Framework agiles	9
4.2 Description SCRUM	10
4.3 Structure de l'équipe	10
4.4 Planification des rôles	11
4.5 Planification des Sprints.....	12
5. Unified Modeling Language (UML)	13
6. Conclusion.....	13
CHAPITRE 2 :Etude Conceptuelle	14
1. Introduction.....	15
2. Analyse des besoins.....	15
2.1. Les besoins fonctionnels	15
2.2. Les besoins non fonctionnels	16
3. Product Backlog	17
4. Environnement de travail	23
4.1. Environnement Matériel	23
4.2. Les technologies du travail.....	23
5 Architecture	23
5.1 Front-End.....	23
5.2 Back-End	23

5.3 Intégration Embarqué.....	24
6 Conclusion.....	24
CHAPITRE 3	25
SPRINT 1 : MacroPark Admin Plateform	25
1 Introduction.....	26
2. Backlog sprint 1.....	26
3. Modélisation UML.....	27
3.1 Diagramme de cas d'utilisation du sprint 1	27
3.2 Diagramme de Class du sprint 1	28
3.3 Diagramme de Séquence du sprint 1	28
4. Implémentation et technologie utilisé	29
4.1 Frontend	29
4.2 Backend	29
4.3 Liaison entre Frontend et Backend.....	30
4.4 Base de Données	30
5. Description détaillée du sprint 1	30
5.1 Description 'Authentification'	30
5.2 Description 'Consulter et gérer la liste des utilisateurs'.....	31
5.3 Description 'Consulter et gérer la liste des invités'.....	32
5.4 Description 'Consulter l'historique des entrées au parking'.....	32
5.5 Description ' Consulter et gérer la liste des barrières '	33
5.6 Description ' Tester les barrières'	34
5.7 Description ' Consulter l'historique des barrières '	34
6. Réalisation.....	35
6.1 Interfaces graphiques.....	35
6.2 Logique et Codes.....	50
7. Sprint Review	54
8. Perspective.....	55
9. Conclusion :	55
CHAPITRE 4	56
SPRINT 2 : MacroPark Mobile Application.....	56
1. Introduction.....	57
2. Backlog sprint 2.....	57
3 Modélisation UML.....	58
3.1 Diagramme de cas d'utilisation du sprint 2	58
3.2 Diagramme de Class du sprint 2	58
3.3 Diagramme de Séquence du sprint 2	59

4	Implémentation et technologie utilisé	59
4.1	Frontend	59
4.2	Backend	59
4.3	Liaison entre Frontend et Backend.....	60
4.4	Liaison entre Application mobile et dispositif IOT (contrôleur de la barrière).....	60
4.5	Base de Données.....	60
5	Description détaillée du sprint 2	61
5.1	Description 'Se connecter'	61
5.2	Description 'S'inscrire'	62
5.3	Description 'Page d'accueil'	63
5.4	Description 'Consulter Profile et Mise à jour des données'	63
5.5	Description ' Consulter Historique'	64
5.6	Description ' Se déconnecter'	65
6.	Réalisation.....	61
6.1	Interfaces graphiques.....	66
6.2	Logique et Codes.....	71
7.	Sprint Review	74
8.	Perspective Sprint 4.....	74
9.	Conclusion.....	74
CHAPITRE 5		75
SPRINT 3 : MacroPark Barrier Controller		75
1.	Introduction.....	76
2.	Backlog Sprint 3	76
3.	Modélisation UML	77
4.	Description détaillée du sprint 3.....	78
4.1	Description 'Identification du matérielle '	78
4.1.1	Qu'est-ce que le WT32 ?	78
4.1.2	Caractéristiques Clés du WT32	78
4.2	Pourquoi Utiliser le WT32 pour le Système de Contrôle de Barrière ?	83
4.2.1	Connectivité Complète	83
4.2.2	Puissance de Traitement.....	84
4.2.3	Rentabilité et Intégration Simplifiée	84
4.2.4	Efficacité Énergétique.....	84
4.2.5	Flexibilité et Scalabilité	85
4.2.6	Développement et Maintenance	85
4.2.7	Utilisation de l'Adaptateur CP2102	85
5.	Barrière.....	88

5.1 Description du ‘ Découvrir le fonctionnement du système’	89
5.2 Description du ‘Gestion des configurations’	90
5.3 Description du ‘ Implémentation et réalisation ’	91
5.3.1 Logiciel Utilisé	91
5.3.2 Interface Graphique.....	93
5.4 Description du ‘Schéma et Housing’	95
5.4.1 Housing	95
5.4.2 Impression 3d.....	98
5.4.3 Câblage	100
5.5 Explication du code	101
6. Sprint Review	117
7. Perspective.....	117
8. Conclusion.....	118
CHAPITRE 6	119
SPRINT 4 : MacroPark Licence Plate Recognizer	119
1. Introduction.....	120
2. Backlog sprint 4.....	120
3. Implémentation et technologie utilisé	121
4. Description détaillée du sprint 4	121
4.1. Description ‘Identification du matérielle’	121
4.1.1.Caractéristique	122
4.1.2.Installation	122
4.1.3.Description ‘Configuration du Caméra ‘	124
4.1.4.Description ‘Configuration du contrôleur de la caméra (BRIDGE) :.....	126
4.1.5.Description ‘Configuration du serveur principale (CORE SERVER) :.....	128
4.2. Description ‘Communication’	131
4.2.1.Communication entre la caméra et Bridge.....	131
4.2.2 Communication entre le CoreServer et le Bridge.....	131
4.2.3 Communication entre le CoreServer et le Contrôleur (Backend)	132
4.2.4 communication entre le CoreServer et le contrôleur de la barrière	132
5 Sprint Review	133
6 Perspectives	133
7 Conclusion.....	133
CHAPITRE 7	134
SPRINT 5: MacroPark Test & Validation	134
1. Introduction:.....	135
2. Backlog sprint 5.....	135

3. Description des outils utilisés	136
4. Description détaillée du sprint 5	139
4.1 Répartition du notre APIs	139
4.2. Test de l'application mobile	146
4.3. Exécution du projet	147
5. Test Hardware	151
5.1. Configuration WiFi Initiale	151
5.2. Fonctionnement Normal après Configuration	153
5.3. Réaction aux Messages MQTT après Approbation de la Plaque d'Immatriculation	153
5.4. Réaction à l'Ouverture avec BLE et Envoi de Message MQTT	154
5.5. Réception de l'ID d'Entreprise via MQTT et Modification du Filtre de Scan BLE.	155
6. Sprint Review	157
7. Perspective	157
8. Conclusion	157
CONCLUSION GENERALE	158
BIBLIOGRAPHIE et NETOGRAPHIE	159

LISTE DE FIGURES

Figure 1. Logo Scheidt & Bachmann.....	4
Figure 2. Organigramme de la société	6
Figure 3. Historique de Scheidt & Bachmann	7
Figure 4. Méthode SCRUM.....	10
Figure 5. Structure d'équipe SCRUM	11
Figure 6. UML	13
Figure 7 : Ecosystème du MacroPark	19
Figure 8 : Cas 1	20
Figure 9. Cas 2.....	22
Figure 10 . Diagramme de cas d'utilisation du sprint 1	27
Figure 11 . Diagramme de Class du sprint 1	28
Figure 12 . Diagramme de Séquence du sprint 1	28
Figure 13 . Diagramme de cas d'utilisation 'Authentification' / Sprint 1	30
Figure 14. Diagramme de cas d'utilisation 'Utilisateurs ' / Sprint 1	31
Figure 15 . Diagramme de cas d'utilisation 'invité' / Sprint 1	32
Figure 16 . Diagramme de cas d'utilisation 'Consulter historique' / Sprint 1	32
Figure 17 . Diagramme de cas d'utilisation 'barrière' / Sprint 1.....	33
Figure 18 . Diagramme de cas d'utilisation 'Tester barrière' / Sprint 1	34
Figure 19 . Diagramme de cas d'utilisation 'Historique des barrières' / Sprint 1	34
Figure 20. Page de connexion / Sprint 1	35
Figure 21. Erreur de connexion / Sprint 1	36
Figure 22. Connexion réussite / Sprint 1	36
Figure 23 : Page d'accueil / Sprint 1.....	36
Figure25. Liste utilisateurs vide (Page Users) / Sprint 1	37
Figure 24 . Lister utilisateurs (Page Users) / Sprint 1	37
Figure 26. Ajouter utilisateur (Page Users) / Sprint 1	38
Figure27. Modifier Utilisateur (Page Users) / Sprint 1	38
Figure 28. Modification réussite (Page Users) / Sprint 1	39
Figure 28 . Modification réussite (Page Users) / Sprint 1	39
Figure 29. Supprimer utilisateurs (Page Users) / Sprint 1	39
Figure 31. Liste vide (Page Guests) / Sprint 1	40
Figure 30 . Liste d'invité (Page invité) / Sprint 1.....	40
Figure32. Accepter invité (Page Guests) / Sprint 1.....	41
Figure33.Refuser invité (Page Guests) / Sprint 1	42

Figure34. Lister Historique(Page Users Logs) / Sprint 1	43
Figure35. Recherche par date (Page Users Logs) / Sprint 1	43
Figure 36. Recherche non trouvée (Page Users Logs) / Sprint 1	44
Figure 37. Fenêtre d'immatriculation TN (Page Users Logs) / Sprint 1	44
Figure 38. Fenêtre d'immatriculation RS (Page Users Logs) / Sprint 1	44
Figure39. Lister barrières (Barrier Lister) / Sprint 1	45
Figure40. Ajouter barrière (Barrier List) / Sprint 1	45
Figure 41. Modifier barrière (Barrier List) / Sprint 1	46
Figure 42 . Supprimer barrière (Barrier List) / Sprint 1	46
Figure 43 .Page Test barrier / Sprint 1	47
Figure44. Page Test Barrier / Sprint 1	48
Figure45. Message d'action (Test Barriers) / Sprint 1	48
Figure 46 : Historique des barrières (Test Barrier) / Sprint 1	49
Figure47: Recherche par identifiant de la barrière / Sprint 1	49
Figure 48. Recherche non trouvée / Sprint 1	50
Figure49. A propos de nous / Sprint 1	50
Figure50. Diagramme de cas d'utilisation du sprint 2.....	58
Figure 51. Diagramme de Class du sprint 2	58
Figure 52. Diagramme de Séquence du sprint 2	59
Figure53. Diagramme de cas d'utilisation de « se connecter ».....	61
Figure54. Diagramme de cas d'utilisation de « s'inscrire ».....	62
Figure55. Diagramme de cas d'utilisation de « Page d'accueil».....	63
Figure56. Diagramme de cas d'utilisation de « Consulter Profile et Mise à jour des données».....	63
Figure57. Diagramme de cas d'utilisation de « Consulter Historique».....	64
Figure58. Diagramme de cas d'utilisation de « Se déconnecter»	65
Figures 59. Page de connexion	66
Figures 60. Page d'inscription.....	67
Figure 61. UUID	68
Figures62. Page d'accueil.....	69
Figures 63. Page de Profile	70
Figures 64. Page historique	71
Figure 65. Diagramme de classe.....	77
Figure 66. Diagramme de séquence.....	77
Figure 67. Environnement de Développement Intégré (IDE).....	80
Figure68. WT32.....	83
Figure 69. L'Adaptateur CP2102.....	85

Figures 70. Systèmes de barrières automatiques	88
Figure 71 : Système d'I/O de la barrière.....	88
Figure 72. MQTT Explorer.....	91
Figure73. Beacon Simulator	92
Figure 74. Interface Graphique.....	94
Figure74. Interface Graphique	94
Figure 75. Type de vue : en haut complète.....	96
Figure 76. Type de vue : en haut sans cache	96
Figure 77. Type de vue : en droite	97
Figure 78. Type de vue : Cache	97
Figure 77. Type de vue : en droite	97
Figure 78. L'imprimante3D	98
Figure 79. Modèle numérique	99
Figure 80. Câblage.....	100
Figure 81 . Les Bibliothèques.....	101
Figure 82. Variables globales et constantes	101
Figure 83. Contrôle du timing et de l'état des barrières	102
Figure 84. Configuration BLE et réseau	102
Figure 85. Classes de rappel et fonctions.....	103
Figure 86. Fonctions supplémentaires	104
Figure87. Fonction de rappel MQTT	105
Figure 88. Fonction de reconnexion	105
Figure89. Fonction setup	106
Figure 90. Initialisation des périphériques BLE	107
Figure 91. Config Réseau	108
Figure92. Ajouter des paramètres au WebServer du WifiManager	108
Figure93. Config ETH	108
Figure 94. connexion.....	109
Figure 95. ArduinoOTA	109
Figure96. affichages données	110
Figure97. MQTT server	110
Figure 98. Gestion des barrières	111
Figure99. Gestion de l'OTA (Over-The-Air) et vérification du bouton	112
Figure 100. Gestion du portail web et du client MQTT	113
Figure 101. Gestion des messages d'accès	113
Figure 102. Scan BLE et traitement des appareils trouvés	116

Figure103. Sauvegarde de la configuration (callback)	116
Figure104. Système de contrôle des barrières de parking	118
Figures 105. Modèle Caméra	122
Figure106. câble du réseau	122
Figure107. Connexion câble d'alimentation vers une source 220 V	123
Figure 108. Pins d'entrées & sorties	123
Figure 109. Système d'entrée / Sortie.....	123
Figure 110. Configuration camera.....	124
Figure 111. Accès au page de la configuration.....	124
Figure 112. Configurer le central host	124
Figure 113. Configurer l'interface de configuration	125
Figure 114. Reconnaissance des plaques d'immatriculation tunisiennes	125
Figure 115. Configurer l'adresse IP et le port de l'API REST	125
Figure 116. Description 'Configuration du contrôleur de la caméra (BRIDGE)	126
Figure 117. Résultat du bridge	127
Figure 118. Test par MQTT Explorer	127
Figure119. CoreServer	128
Figure 120. Entre la caméra et Bridge	131
Figure 121. Entre CoreServer et le Bridge	131
Figure 122. Entre CoreServer et le contrôleur	132
Figure 123. Entre CoreServer et le contrôleur de la bannière.....	132
Figure 124: Swagger UI.....	136
Figure125. Détail de Swagger ui.....	137
Figure 126. Exécution des APIs	137
Figure 127. Modèle response de swagger.....	138
Figure 128. Répartition du notre APIs : Users.....	139
Figure 129. Répartition du notre APIs :Guests	139
Figure 130. Répartition du notre APIs : Barriers Data.....	139
Figure 131. Répartition du notre APIs : Barriers.....	140
Figure 132. Répartition du notre APIs : Events	140
Figure133. Tester les différents API : Ajouter un utilisateur	140
Figure134. Tester les différents API : Se connecter à l'application mobile	141
Figure135. Tester les différents API : Ajouter un invité	141
Figure136. Tester les différents API : Lister invité(s).....	142
Figure137. Tester les différents API : Accepter invité	142
Figure138. Tester les différents API : Ajouter une barrière	143

Figure139. Tester les différents API : Lister toutes les barrières	143
Figure140. Tester les différents API : Lister l'historique des actions pour les barrières.....	144
Figure141. Tester les différents API : Lister l'historique des entrées au parking	144
Figure142. Tester les différents API : Lister l'historique par date (filtrer)	145
Figure143. Capture d'écran du code androidManifest.xml	146
Figure 144. Dialogue de permission (Application mobile)	146
Figure 145. Docker Hub.....	147
Figure 146. Les versions d'un conteneur	148
Figure 147. code Docker-compose.yaml.....	149
Figure 148. Exécution du Docker compose file	146
Figure149. Création des Conteneurs dans Docker	150
Figure150. Arrêt du Docker.....	150
Figure151. Access Point Serial Monitor	151
Figure 152. Interface Graphique - Accueil	152
Figure 153. Interface Graphique - Config.....	152
Figure 154. Interface Graphique - Submit.....	152
Figure 155. Relay opened with MQTT	154
Figure 156. Relay opened with BLE	154
Figure 157. Mqtt Explorer	156
Figure 158. Company ID Scan	156

LISTE DES TABLEAUX

Tableau 1: Comparaison entre les différentes Framework agiles	9
Tableau 2: Répartition des rôles	11
Tableau 3 : Planification des sprints.....	12
Tableau 4 : Product Backlog	18
Tableau 5 : Environnement Matériel.....	23
Tableau 6 : Les technologies du travail.....	23
Tableau 7 : Backlog sprint 1	26
Tableau 8 : Description textuelle 'Authentification' / Sprint 1	31
Tableau 9 : Description textuelle 'Utilisateurs' / Sprint 1.....	31
Tableau 10 : Description textuelle 'invités' / Sprint 1	32
Tableau 12 : Description textuelle 'Consulter barrières' / Sprint 1	33
Tableau 13 : Description textuelle 'Tester les barrières' / Sprint 1.....	34
Tableau 14 : Description textuelle 'Historique barrières' / Sprint 1	35
Tableau 15 : Tableau des actions (TCP) / Sprint 1	47
Tableau 16. Réparation du Sprint2.....	57
Tableau 17. Description textuelle / Se Connecter / sprint 2	61
Tableau 18. Description textuelle / S'inscrire / Sprint 2	62
Tableau 19. Description textuelle / Page d'accueil / sprint 2.....	63
Tableau 20. Description textuelle / Profile / Sprint 2.....	64
Tableau 21. Description textuelle / Historique / Sprint 2	64
Tableau 22. Description textuelle / Se déconnecter / Sprint 2	65
Tableau 23. Backlog Sprint 3	76
Tableau 24. Caractéristique WT32	81
Tableau 25. Les composants de WT32.....	86
Tableau 26. Les caractéristiques	88
Tableau 26. Planification de sprint 4	120
Tableau 27. Les caractéristiques	122
Tableau 28. Planification de sprint5	135

LISTE DES ABREVIATIONS

BLE : Bluetooth Low Energy
UML : Unified Modeling Language
WSL : Windows Subsystem for Linux
TCP : Transmission Control Protocol
IDE : Integrated Development Environment
AP : Access Point (Point d'accès)
HTML : HyperText Markup Language
CSS : Cascading Style Sheets
MQTT : Message Queuing Telemetry Transport
API : Application Programming Interface
REST : Representational State Transfer
RS : Regime suspensif (Suspensive Regime)
CPU : Central Processing Unit
OTA : Over-The-Air (mise à jour logicielle sans fil)
SPIFFS : SPI Flash File System
ASGI : Asynchronous Server Gateway Interface
IOT : Internet of Things (Internet des objets)
UUID : Universally Unique Identifier
WT : Web Template
SPI : Serial Peripheral Interface
I2C : Inter-Integrated Circuit
GPIO : General-Purpose Input/Output
UART : Universal Asynchronous Receiver-Transmitter
RAM : Random Access Memory

INTRODUCTION GENERALE

Le concept de parking intelligent est une composante clé de la mobilité intelligente et des villes intelligentes. Il repose sur l'utilisation de l'Internet des Objets (IoT) et des technologies de caméras de surveillance pour optimiser la gestion des places de stationnement. Dans un contexte où les villes deviennent de plus en plus densément peuplées, la gestion efficace des ressources urbaines, y compris le stationnement, est essentielle pour améliorer la qualité de vie et réduire l'empreinte environnementale.

Le parking intelligent est de plus en plus privilégié pour renforcer la sécurité et la traçabilité pour diverses raisons. Notamment elles permettent une surveillance accrue des véhicules et des personnes, réduisant ainsi les risques de vol et d'incidents de sécurité. En plus, grâce à des technologies comme la reconnaissance de plaques d'immatriculation et les capteurs de mouvement, il est possible de suivre précisément le flux des véhicules et de détecter toute activité suspecte. En fin les capteurs et la connectivité permettent une utilisation optimale de l'espace de stationnement, ce qui est crucial dans les zones urbaines densément peuplées.

Il s'agit d'un système avancé qui utilise des capteurs, des caméras et des logiciels pour surveiller, analyser et gérer les espaces de stationnement en temps réel. L'objectif principal est d'optimiser l'utilisation des places de stationnement disponibles et de faciliter l'accès des conducteurs à ces places. Les systèmes de parking intelligent peuvent inclure des capteurs intégrés dans le sol, des caméras de surveillance, des applications mobiles et des panneaux d'affichage pour informer les conducteurs des places libres.

Ce rapport est organisé en sept chapitres comme suite :

Le premier chapitre **État de l'art**.

Le deuxième chapitre **Etude Conceptuelle**.

Le troisième chapitre **MacroPark Admin Plateforme** (Sprint1)

Le quatrième chapitre **MacroPark Mobile Application** (Sprint 2)

Le cinquième chapitre **MacroPark Barrier Controller** (Sprint 3)

Le sixième chapitre **MacroPark Licence Plate Recognizer** (Sprint 4)

Le septième chapitre **MacroPark Test & Validation** (Sprint 5)

CHAPITRE 1

ÉTAT DE L'ART

1. Introduction

Ce premier chapitre est consacré à la présentation du cadre général du projet. Tout d'abord nous décrivons l'entreprise dans laquelle nous avons réalisé notre projet de fin d'études, ensuite, nous exposons le contexte général, analyse de l'existant et la solution envisagée pour la réalisation de ce projet, enfin, nous terminons par choisir le Framework de développement agile SCRUM.

2. Présentation de l'organisme d'accueil

2.1. Introduction

Scheidt & Bachmann GmbH est une entreprise familiale de cinquième génération. Aujourd'hui, ils emploient environ 3 000 employés au siège et dans 25 filiales dans le monde entier.

De plus, de nombreux partenaires dans plus de 50 pays assurent qu'ils sont toujours proches de leurs clients.



Figure 1. Logo Scheidt & Bachmann

2.2. Solutions

Ils développent des solutions de pointe dans quatre domaines d'activité différents :

- Systèmes de stationnement et d'accès
- Systèmes de signalisation
- Systèmes de perception des tarifs
- Solutions de vente au détail de carburant

2.3. Valeurs

- Respect
- Confiance et responsabilité personnelle
- Esprit d'équipe et passion

- Fiabilité
- Responsabilité sociale

2.4. Objectifs

- La satisfaction des employés
- Orientation client
- Innovation
- Internationalité
- Indépendance économique

2.5 Organigramme de la société

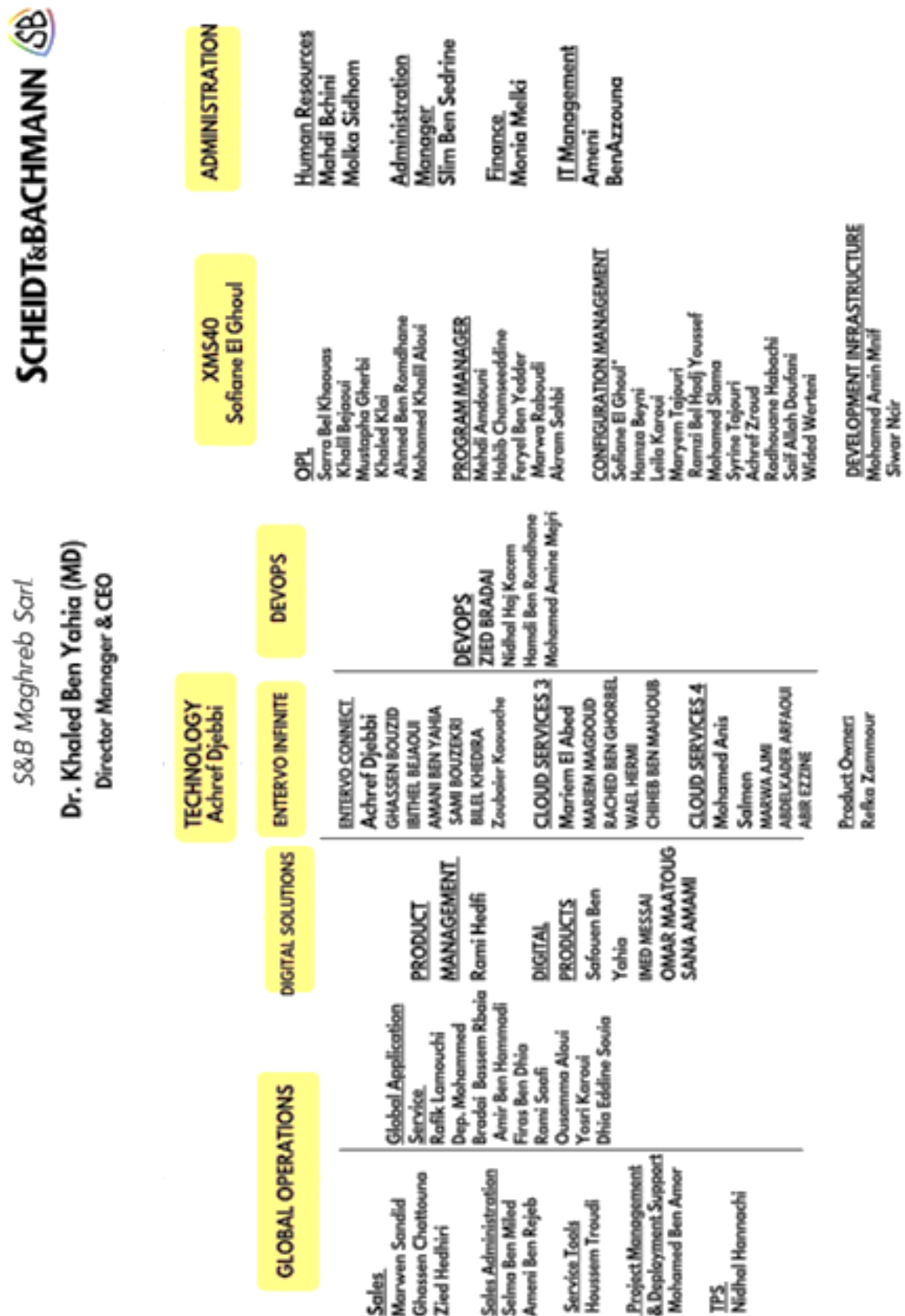


Figure 2. Organigramme de la société

2.6 Historique de Scheidt & Bachmann GmbH

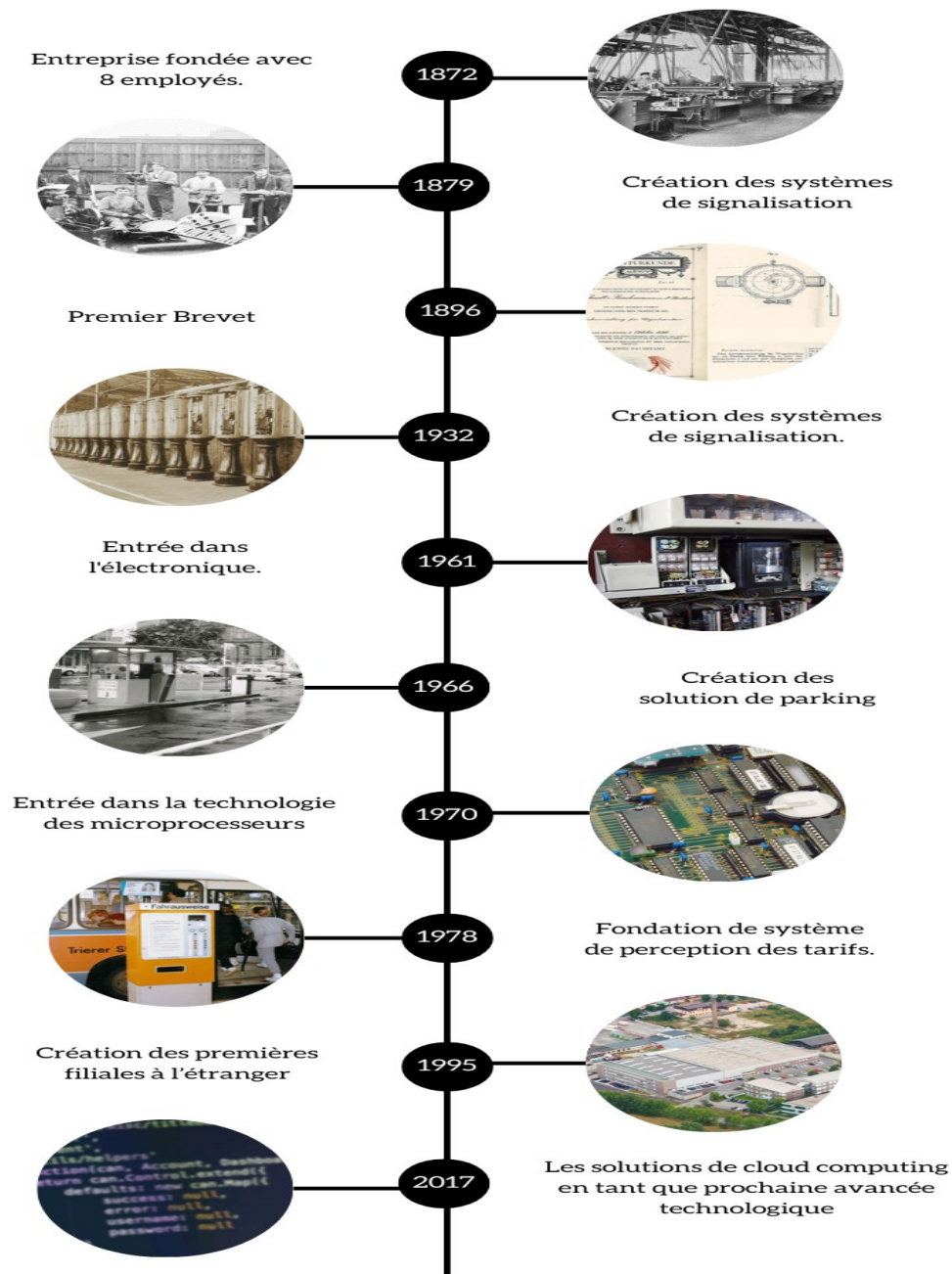


Figure 3. Historique de Scheidt & Bachmann

2.7 Cadre du projet

Ce travail s'inscrit dans le cadre d'un sujet de fin d'études en vue de l'obtention du diplôme de licence système embarqué et mobiles au sein de l'Institut supérieur des études technologiques de Rades (ISET Rades). Durant un stage de quatre mois effectués à Scheidt & Bachmann Maghreb, Notre mission est de mettre en place d'un système contrôle et de la gestion d'un parking privé.

3. Etude de l'existant

3.1. Problématique

Actuellement, le parking de la société manque d'un système efficace de contrôle d'accès et de gestion des véhicules.

3.2. Solution

✓ **Développer une application mobile pour les utilisateurs :**

- Gestion du profile.
- Accès au parking plus sécurisée.
- Gestion des historiques.

✓ **Développer un système de contrôle de parking privé :**

- Basé sur des barrières contrôlées par un microcontrôleur WT32.
- Utiliser la technologie Bluetooth Low Energy (BLE) pour les signaux de contrôle.
- Intégrer la reconnaissance de plaques d'immatriculation via une caméra pour une gestion plus précise.

✓ **Développer un système de gestion du parking pour l'administrateur :**

- Offrant des fonctionnalités avancées pour un contrôle efficace.
- Permettant la surveillance des entrées au parking.
- Incluant des outils de gestion des utilisateurs et des invitées.

3.3. Cahier des charges

- ✓ Le système de contrôle de parking doit être robuste et fiable, assurant un fonctionnement continu et sécurisé.
- ✓ Les barrières doivent réagir de manière rapide et précise aux signaux BLE et à la reconnaissance de plaques d'immatriculation.
- ✓ Le système de gestion du parking doit être convivial, offrant une interface intuitive pour l'administrateur.
- ✓ Il doit permettre la configuration et la personnalisation des paramètres selon les besoins spécifiques du parking.
- ✓ La gestion des données doit être sécurisée, avec des protocoles de communication et de stockage appropriés.
- ✓ Le système doit être évolutif, permettant l'ajout de fonctionnalités supplémentaires à l'avenir.

Ce cahier des charges définit les exigences et les spécifications pour le développement du système de contrôle de parking privé, ainsi que pour le système de gestion associé.

4. Méthodologie de travail

4.1 Comparaison entre les différentes Framework agiles

Tableau 1: Comparaison entre les différentes Framework agiles

	SCRUM	KANBAN	ASD
Cycle de Sprint	2 à 4 semaines	Pas de sprint – livraison continue	4 à 8 semaines
Taille de projet	Tous types et tailles	Tous types et tailles	Equipe de petite taille
Participation des client	Impliqué via le propriétaire du produit	Dépend de la préférence de l'équipe	Grâce à des versions fréquentes
Documentation	Documentation médiocre	Pas de documentation	Documentation de base

Approche	Incréments itératifs	Aucun flux de travail ne prescrit, les équipes peuvent suivre n'importe quelle approche incrémentielle	Incrémental
-----------------	----------------------	--	-------------

4.2 Description SCRUM

La méthode Scrum est une méthode agile de gestion / développement produit ou projet, utilisée dans le domaine du développement informatique (logiciels, services, applications mobiles, sites web, etc.). Dans le cadre de la méthode Scrum, le produit logiciel est développé ou évolue sous formes de sprints qui sont des phases de développement de quelques semaines qui se concentrent généralement sur quelques fonctionnalités.

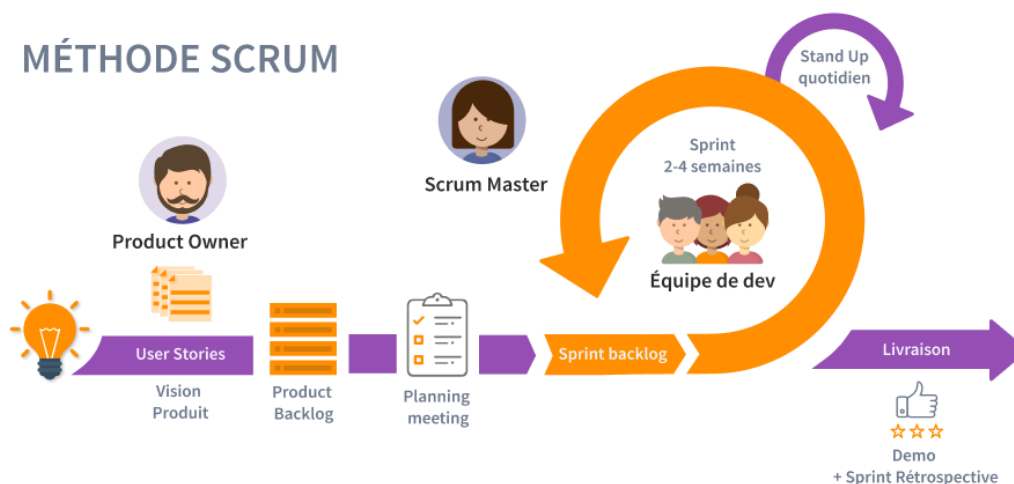


Figure 4. Méthode SCRUM

4.3 Structure de l'équipe

Dans SCRUM il existe trois rôles principaux qui sont le Product Owner, le SCRUM master et l'équipe de développement.

La figure ci-dessous représente l'équipe SCRUM :

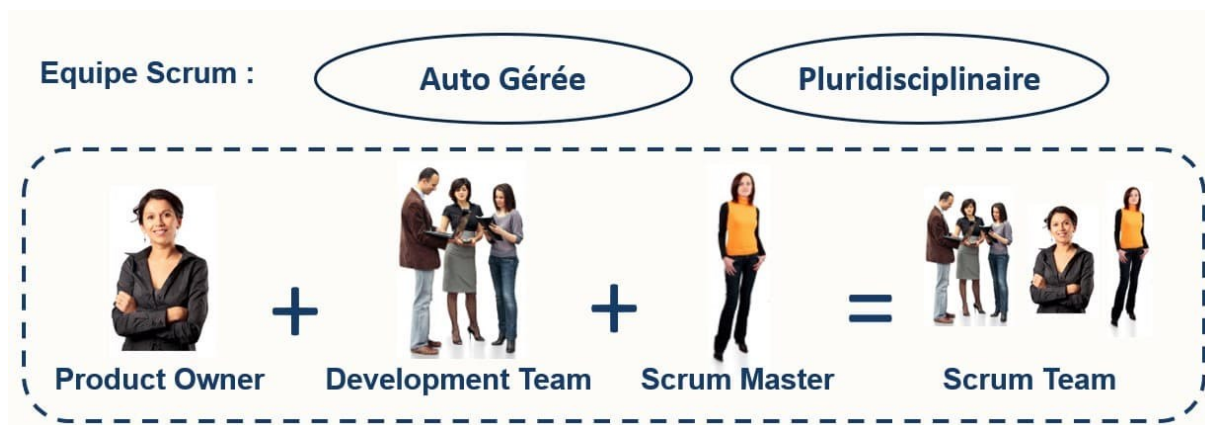


Figure 5. Structure d'équipe SCRUM

4.4 Planification des rôles

✓ **Product Owner :**

Joue le rôle du client éventuel. Son objectif est de valoriser au maximum le produit et le travail de l'équipe de développement.

✓ **SCRUM Master :**

Le SCRUM master aide à s'assurer que tous les membres de l'équipe travaillent ensemble efficacement, en éliminant tous les obstacles sur leur chemin. Ils aident également à éliminer les obstacles qui pourraient ralentir votre projet ou rendre les choses plus difficiles qu'elles ne devraient l'être.

✓ **Equipe de développement :**

L'équipe de développement peut se composer de trois à neuf personnes, il est chargé de fournir une nouvelle version du produit à chaque itération. Cette version doit être améliorée avec de nouvelles fonctions qui respectent les exigences de qualité pour la livraison.

Tableau 2: Répartition des rôles

Rôle	Représentant
Product Owner	Scheidt & Bachmann Maghreb
Scrum Master	Mr. Bradai Zied
Equipe de développement	Yassine Manai – Sofiene Zayati

4.5 Planification des Sprints

C'est l'activité qui consiste à déterminer et à ordonnancer les tâches du projet, à estimer leurs charges et à déterminer les profils nécessaires à leur réalisation. La planification de ce projet consiste principalement en trois étapes menées en parallèle avec notre développement pour s'assurer de bon fonctionnement de ce dernier. Les objectifs du planning sont les suivants :

- ✓ Déterminer si les objectifs sont réalisés ou dépassés
- ✓ Suivre et communiquer l'avancement du projet
- ✓ Affecter les ressources aux tâches

Nous terminons avec le planning de notre projet dans une durée approximative de 4 mois.

Tableau 3 : Planification des sprints

Sprint	Objectifs	Date début	Date fin
Sprint 1	MacroPark Admin Plateform	05 février 2024	20 mars 2024
Sprint 2	MacroPark Mobile Application	25 mars 2024	25 avril 2024
Sprint 3	MacroPark Barrier Controller	05 février 2024	25 avril 2024
Sprint 4	MacroPark Licence Plate Recognizer	29 avril 2024	17 may 2024
Sprint 5	MacroPark Test & Validation	20 may 2024	05 juin 2024

5. Unified Modeling Language (UML)

UML (Unified Modeling Language) est un langage graphique de modélisation des données et des traitements. C'est un moyen d'exprimer des modèles objet en faisant abstraction de leur implémentation.

UML 2.0 comporte treize diagrammes représentant autant de vues distinctes pour représenter des concepts particuliers du système d'information. Dans le cadre de la modélisation de notre système, nous nous contenterons des diagrammes suivants : cas d'utilisation, classes, séquences et composants.



Figure 6. UML

6. Conclusion

Dans ce premier chapitre, nous avons présenté l'organisme d'accueil de notre projet de fin d'études. Nous avons défini le contexte du stage, puis dressé le cadre du projet, en y intégrant les critiques et les solutions envisageables.

Enfin, nous avons expliqué le choix de la méthodologie de travail utilisée, et nous nous dirigerons directement vers la planification.

CHAPITRE 2 :

ETUDE CONCEPTUELLE

1. Introduction

Dans ce chapitre, nous allons commencer par une analyse des besoins fonctionnels et non fonctionnels associés à notre projet. Ensuite, nous déterminerons la méthode à adopter afin de modéliser les besoins auxquels notre solution doit répondre. Enfin, nous clôturerons le chapitre avec la conception des besoins identifiés et une brève conclusion.

2. Analyse des besoins

Le but du projet est de répondre à la demande de l'entreprise.

Cette demande doit être clairement indiquée avant de proposer une solution.

Pour cela, on distingue deux types de besoins :

- ✓ Besoins fonctionnels.
- ✓ Besoins non fonctionnels.

2.1. Les besoins fonctionnels

Les besoins fonctionnels sont les expressions de ce que le produit fourni par le projet devrait faire.

Ces besoins sont spécifiés par les représentants des utilisateurs et des bénéficiaires du produit délivré par le projet.

Dans notre cas le Projet est composé de :

- **Une Interface Web qui doit :**
 - ✓ Gérer la liste des utilisateurs du parking.
 - ✓ Gérer la liste des invités du parking.
 - ✓ Gérer la liste des barrières installées dans le parking.
 - ✓ Gérer les évènements tels que les entrées au parking.
 - ✓ Ouverture les barrières à travers dans les cas d'urgences.

- **Une Application Mobile qui doit :**
 - ✓ Permettre l'utilisateur de gérer son compte
 - ✓ Fournir l'utilisateur un accès au parking.
 - ✓ Permettre l'utilisateur de consulter son historique.
 - ✓ Permettre les invités de l'externe de s'inscrire.
- **Un contrôleur IoT :**
 - ✓ Fournir une communication entre la barrière, l'application mobile et la caméra.
- **Une reconnaissance par caméra :**
 - ✓ Fournir la reconnaissance des immatriculations des véhicules autorisés.

2.2. Les besoins non fonctionnels

Les besoins non fonctionnels représentant les besoins optionnels d'un projet. Ils peuvent être des contraintes (Sécurité, Fiabilité etc. . . .) ou des restrictions qui empêchent le bon fonctionnement du système. Il est important de dégager ces besoins afin de garantir une meilleure qualité du produit.

Dans le cas de notre solution, les besoins non fonctionnels que nous avons distingués sont :

- ✓ **Sécurité** : Notre application mobile / Web doit assurer la sécurité et la confidentialité des données de tous les utilisateurs.
Garantit que seuls les véhicules autorisés accèdent aux points contrôlés, améliorant la sécurité.
- ✓ **Simplicité** : Notre solution doit être non-compiquée afin de simplifier son utilisation par les employées.
- ✓ **Performance** : Le temps de réponse ne doit pas dépasser quelques secondes pour éviter les problèmes d'insatisfaction de l'utilisateur.
- ✓ **Efficacité énergétique** : Utilise BLE pour la communication, ce qui est faible en consommation d'énergie et adapté pour une opération continue.

- ✓ **Fiabilité** : Notre projet doit effectuer son travail d'une manière satisfaisante dans des conditions données et pendant une durée bien déterminée.
- ✓ **Esthétique** : Notre système doit offrir une apparence adéquate, digne d'une certaine norme requise.
- ✓ **Automatisation améliorée** : Réduit le besoin d'intervention manuelle en automatisant le contrôle des barrières.
- ✓ **Précision** : Contrôle précis de chaque barrière en traitant des données spécifiques des signaux BLE.
- ✓ **Flexibilité** : Adaptable à diverses situations avec la capacité de gérer simultanément plusieurs barrières, chacune avec des états et des durées différents.

3. Product Backlog

SCRUM utilise une approche pour récolter les besoins des utilisateurs.

L'objectif est d'établir une liste de fonctionnalités à réaliser qu'on appelle BACKLOG de produit.

Dans cette partie, nous allons détailler les fonctionnalités de l'application que nous avons développée dans un tableau selon les critères suivants :

Tableau 4 : Product Backlog

Fonctionnalité	ID	User Story	Estimation en Jours
MacroPark Admin Plateform	1	En tant qu'admin je veux me connecter	2
	2	En tant que Admin je veux consulter et gérer la liste des utilisateurs	5
	3	En tant que Admin je veux consulter et gérer la liste des invités	5
	4	En tant que Admin je veux consulter et gérer la liste des barrières	6
	5	En tant que Admin je veux consulter l'historique des événements des Barrières	6
	6	En tant que Admin je veux consulter l'historique des utilisateurs	5
	7	En tant que Admin je veux commander manuellement toutes les barrières installées	5
MacroPark Mobile Application	1	En tant qu'utilisateur je veux connecter à l'application avec mon email et mot de passe	5
	2	En tant qu'utilisateur je veux ouvrir la barrière à distance	4
	3	En tant qu'utilisateur je veux consulter et modifier mon profil	6
	4	En tant qu'utilisateur je veux consulter mon historique d'accès au parking.	4
	5	En tant qu'invité je veux inscrire à l'application	6
MacroPark Barrier Controller (1/2)	1	En tant qu'utilisateur, je veux connecter le WT32 à un réseau Wi-Fi, Ethernet	7
	2	En tant qu'utilisateur, je veux pouvoir configurer les paramètres MQTT	5
	3	En tant qu'utilisateur, je veux que le microcontrôleur scanne et identifie les dispositifs BLE autorisés	8
	4	En tant qu'utilisateur, je veux être informé lorsque l'accès est accordé via BLE	5
	5	En tant qu'administrateur, je veux sauvegarder les logs (avec l'identifiant utilisateur) lorsque l'utilisateur autorisé ouvre la barrière avec BLE	3

MacroPark Barrier Controller (2/2)	6	En tant qu'administrateur, je veux pouvoir mettre à jour le firmware du microcontrôleur	8
	7	En tant qu'administrateur, je veux pouvoir sauvegarder et restaurer les configurations	7
	8	En tant qu'utilisateur, je veux une interface utilisateur intuitive pour toutes les configurations et opérations	8
	9	En tant qu'ingénieur, je veux concevoir et imprimer en 3D un boîtier pour le dispositif	10
	10	En tant qu'ingénieur, je veux établir un schéma de câblage clair	4
MacroPark Licence Plate Recognizer	1	En tant que système de caméra, je veux capturer des plaques d'immatriculation et les envoyer vers le Bridge.	5
	2	En tant que Bridge Server, je veux traiter les données et les publier vers le serveur MQTT	2
	3	En tant que serveur central (Core Server), je veux s'abonner vers un serveur MQTT et recevoir les données pour les traiter avec le backend	5
	4	En tant que CoreServer, je veux créer l'historique pour chaque action	3
MacroPark Test & Validation	1	En tant que testeur je veux tester toutes les fonctionnalités du projet et trouver les erreurs afin de les corriger	13

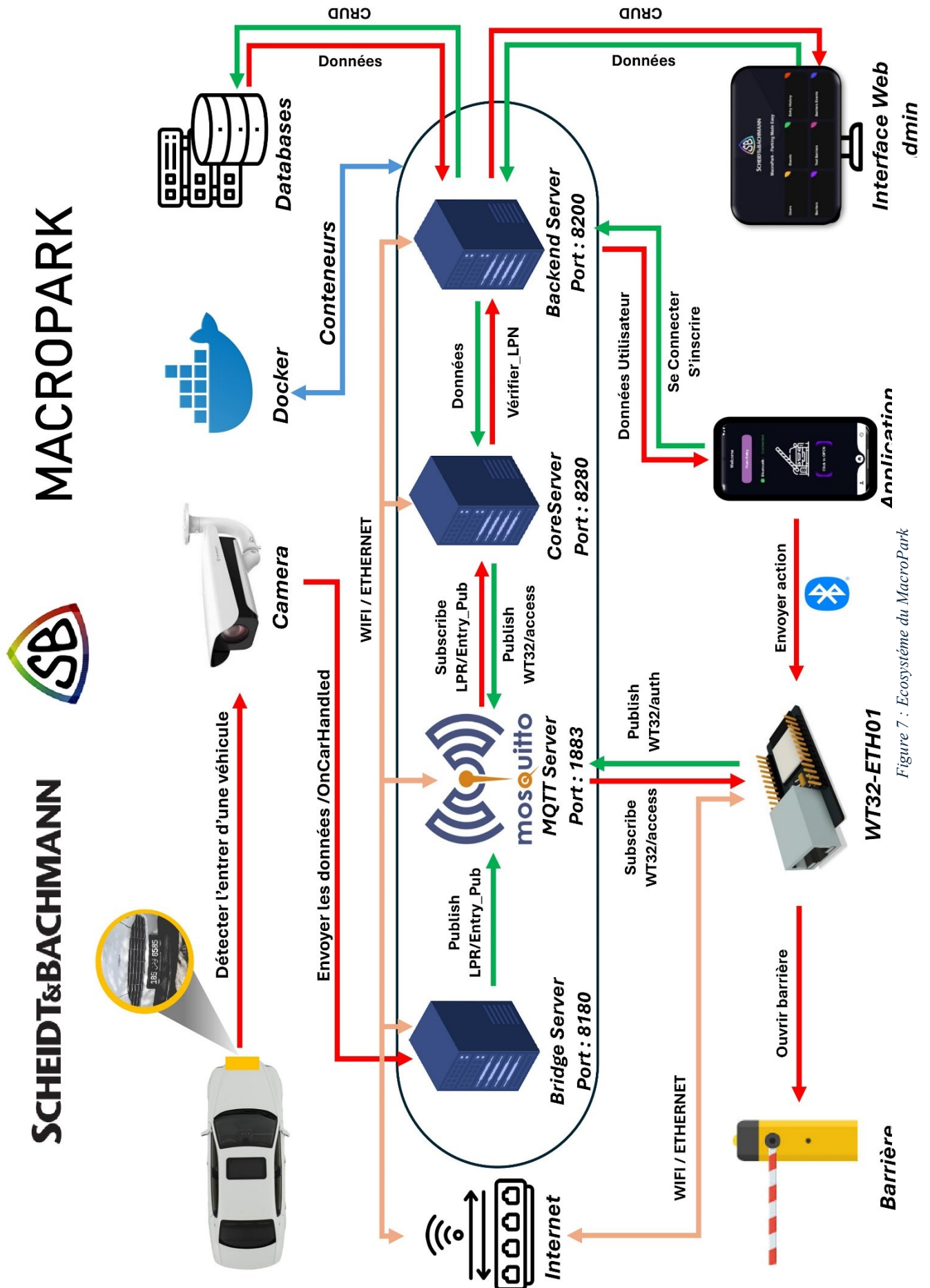


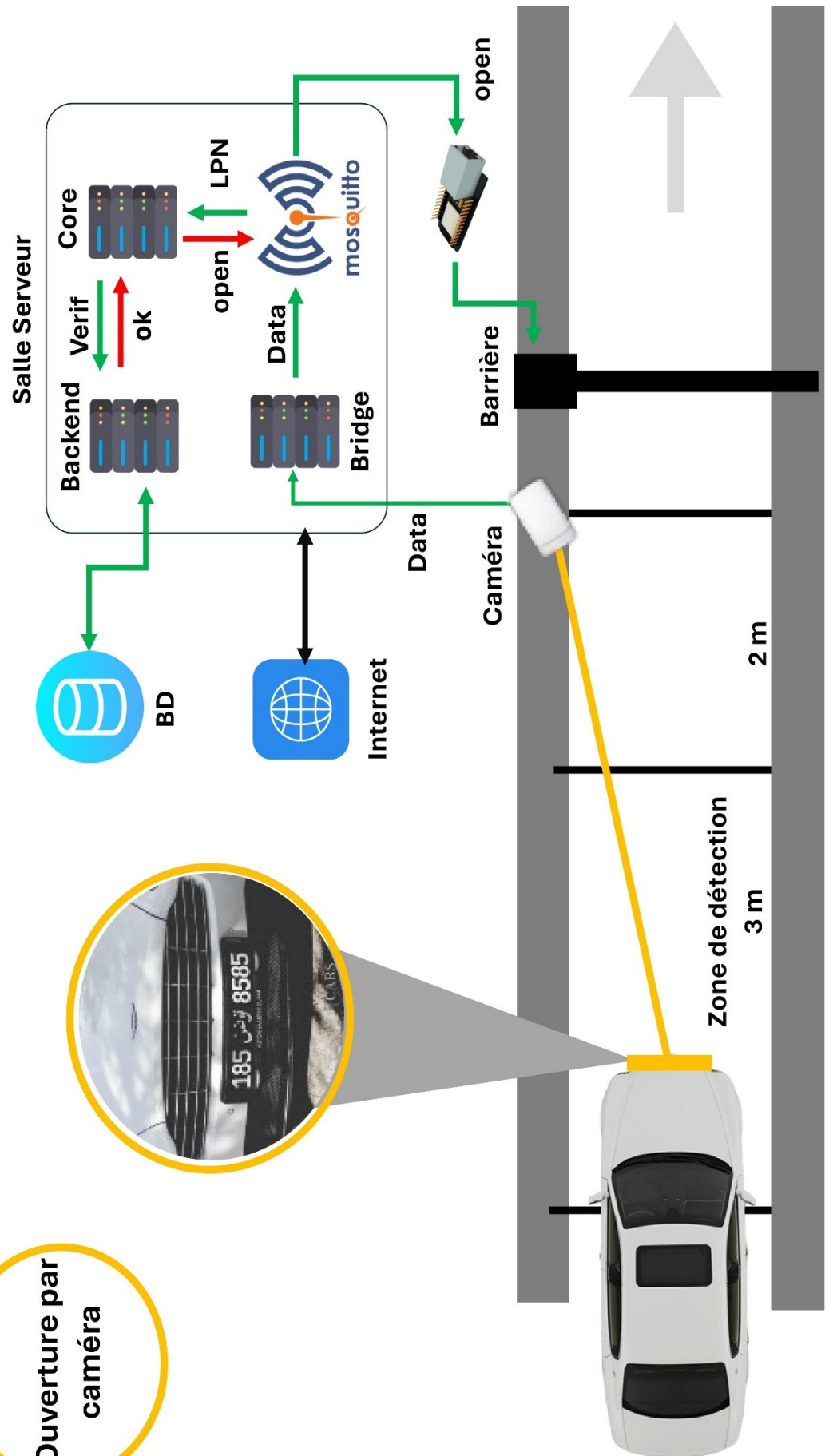
Figure 7 : Ecosystème du MacroPark

MACROPARK



SCHEIDT&BACHMANN

Ouverture par
caméra





SCHEIDT & BACHMANN

**Ouverture par
Application
Mobile**

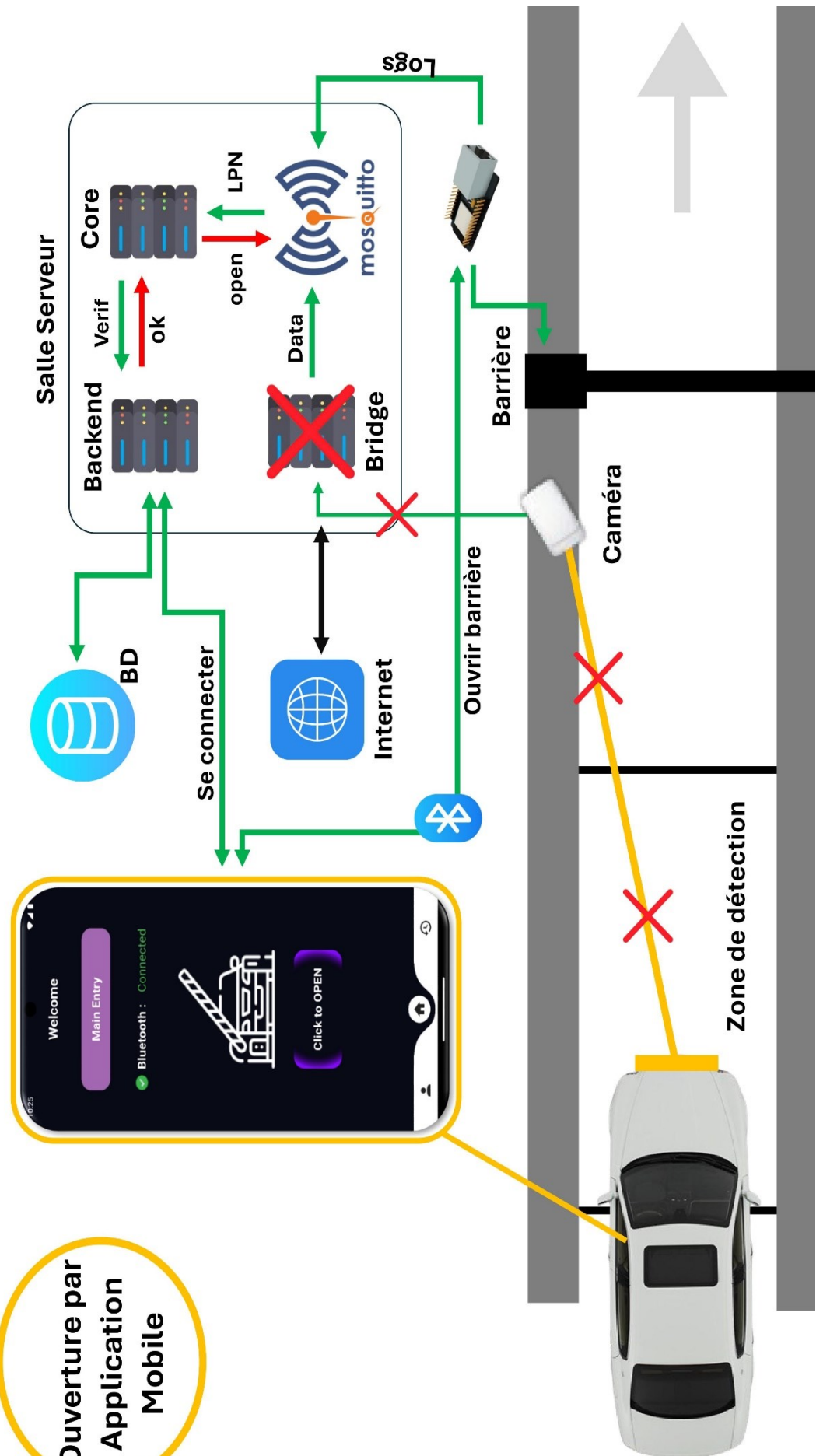


Figure 8. Cas 2

4. Environnement de travail

4.1. Environnement Matériel

Tableau 5 : Environnement Matériel

Caractéristiques		
Modèle	Asus TUF Gaming F15	ASUS X550VX
Processeur	I5 - 10400F	I7-6700HQ
Stockage	SSD de 512 Go	HDD de 1 To
Carte Graphique	NVIDIA GeForce GTX 1650	GTX 950M
Mémoire RAM	16 Go DDR4	16 Go DDR4
Système d'exploitation	Windows 11 pro / 64 bits	Windows 10 pro / 64 bits

4.2. Les technologies du travail

Tableau 6 : Les technologies du travail

Technologie et logiciels				
Logiciels du travail		Language de Programmation		OS
Docker Desktop	VS Code	HTML	Python	Windows
MQTT Explorer	GitHub / Git	CSS	Dart (Flutter)	
Arduino IDE	Postman	JavaScript	C++	Linux
Swagger	SolidWorks	XML	YAML	

5 Architecture

5.1 Front-End

- ✓ Application mobile avec Flutter : Utilisation de Flutter pour créer l'interface utilisateur réactive et attrayante.
- ✓ Interface Web pour l'administrateur en utilisant HTML CSS et JavaScript.

5.2 Back-End

- ✓ Backend avec python et Fast API : Développement d'une API REST avec python pour la communication entre le frontend et le backend.
- ✓ Intégration de la Caméra : Utilisation d'une caméra Quercus Smart LPR [8] pour reconnaître les plaques d'immatriculation et envoie les données via MQTT à un serveur central pour la validation et les traitements des données.

5.3 Intégration Embarqué

- ✓ Utilisation du microcontrôleur WT32-ETH01 : Contrôler les barrières. Ce microcontrôleur reçoit des messages via MQTT depuis core server (Serveur Central) et BLE depuis l'application mobile, assurant une communication fiable et rapide. Configuration simple et rapide pour une gestion efficace des paramètres réseau et des informations du serveur MQTT.

6 Conclusion

Dans ce deuxième chapitre nous avons présenté la capture des besoins par la détermination et l'analyse des besoins fonctionnels et non fonctionnels et l'écosystème, on a aussi détaillé le pilotage de projet en utilisant SCRUM en indiquant l'équipe et rôle, Product backlog, planification des sprints, l'environnement de travail matériel et logiciel et on finalise par l'architecture.

Dans la partie qui suit nous allons entamer notre premier sprint.

CHAPITRE 3

SPRINT 1 : MACROPARK

ADMIN PLATFORM

1 Introduction

Ce chapitre décrit les différentes étapes de la réalisation du premier sprint. Tout d'abord, nous présentons le backlog de ce sprint. Ensuite, nous détaillons l'analyse et la conception du sprint. Enfin, nous présentons les interfaces homme-machine.

2. Backlog sprint 1

Selon la planification de notre méthodologie, le premier sprint est principalement sur le cas d'utilisation « **Développement d'interface Web d'Administrateur** » que nous allons décortiquer par la suite.

Tableau 7 : Backlog sprint 1

Identifiant	Acteur	Tâche	Estimation
Authentification	Administrateur	T1 : Analyse T2 : Conception T3 : Backend T4 : Frontend	T1 : 01 H T2 : 01 H T3 : 02 H T4 : 03 H
Consulter et gérer la liste des utilisateurs			T1 : 02 H T2 : 02 H T3 : 10 H T4 : 09 H
Consulter et gérer la liste des invités			T1 : 03 H T2 : 02 H T3 : 05 H T4 : 07 H
Consulter l'historique des entrées au parking			T1 : 03 H T2 : 02 H T3 : 05 H T4 : 07 H
Consulter et gérer la liste des barrières			T1 : 03 H T2 : 02 H T3 : 05 H T4 : 07 H
Ouvrir les barrières manuellement			T1 : 01 H T2 : 02 H T3 : 07 H T4 : 06 H
Consulter l'historique des événements des barrières			T1 : 01 H T2 : 02 H T3 : 07 H T4 : 06 H

3. Modélisation UML

3.1 Diagramme de cas d'utilisation du sprint 1

Un diagramme de cas d'utilisation illustre comment un système, une partie de celui-ci ou une classe est perçu par un utilisateur externe. Il divise les fonctionnalités en "cas d'utilisation", offrant ainsi une vue compréhensible des besoins des utilisateurs, contrairement à une approche purement technique.

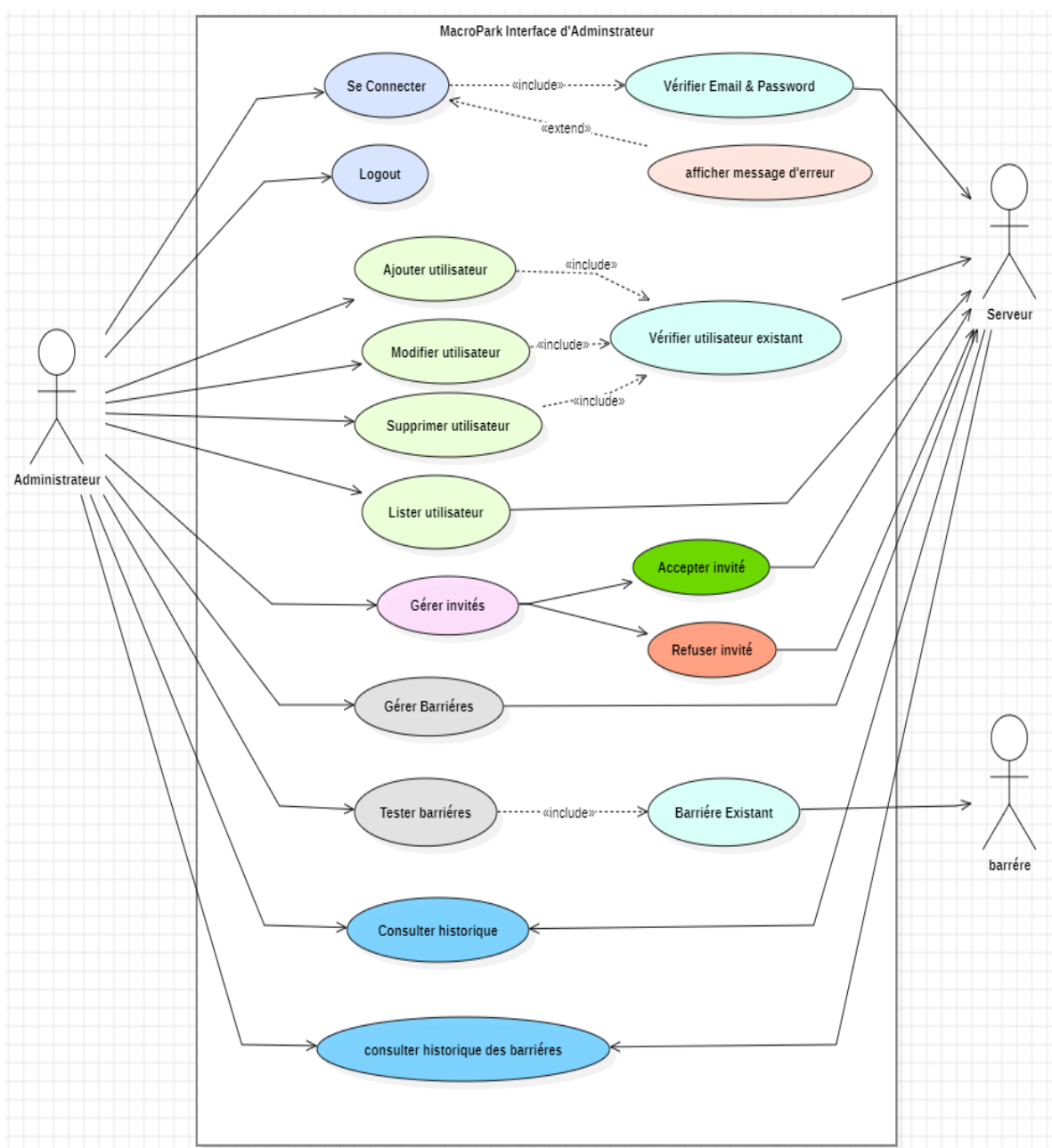


Figure 9 . Diagramme de cas d'utilisation du sprint 1

3.2 Diagramme de Class du sprint 1

Cette phase est consacrée pour la réalisation de notre diagramme de classes qui décrit la structure des données de notre interface web et l'analyse des informations :

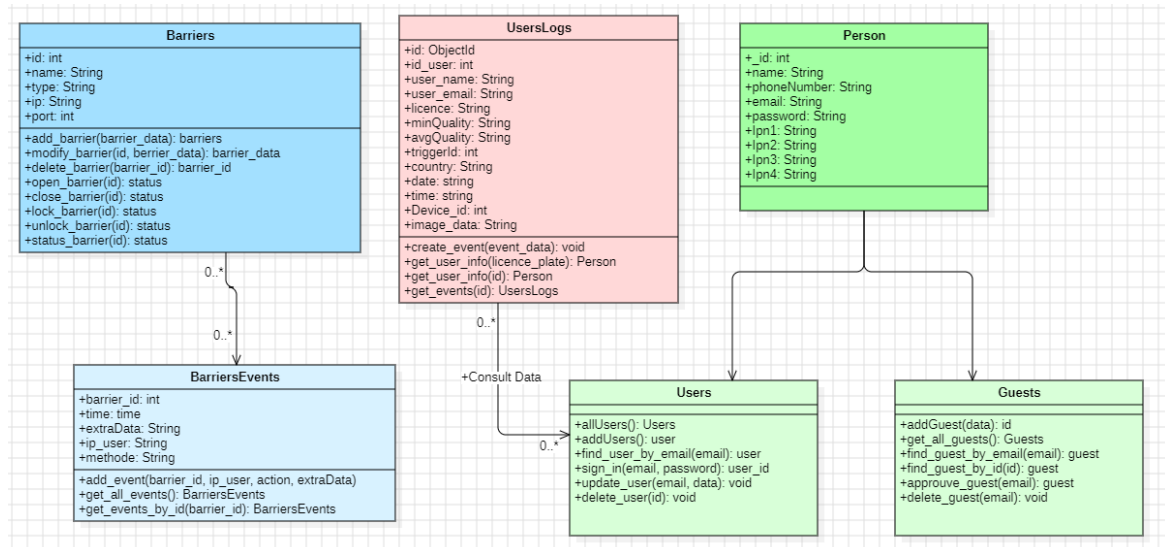


Figure 10 . Diagramme de Class du sprint 1

3.3 Diagramme de Séquence du sprint 1

Le diagramme de séquence qui décrit l'interaction entre notre interface web et l'administrateur est présenté ci-dessous :

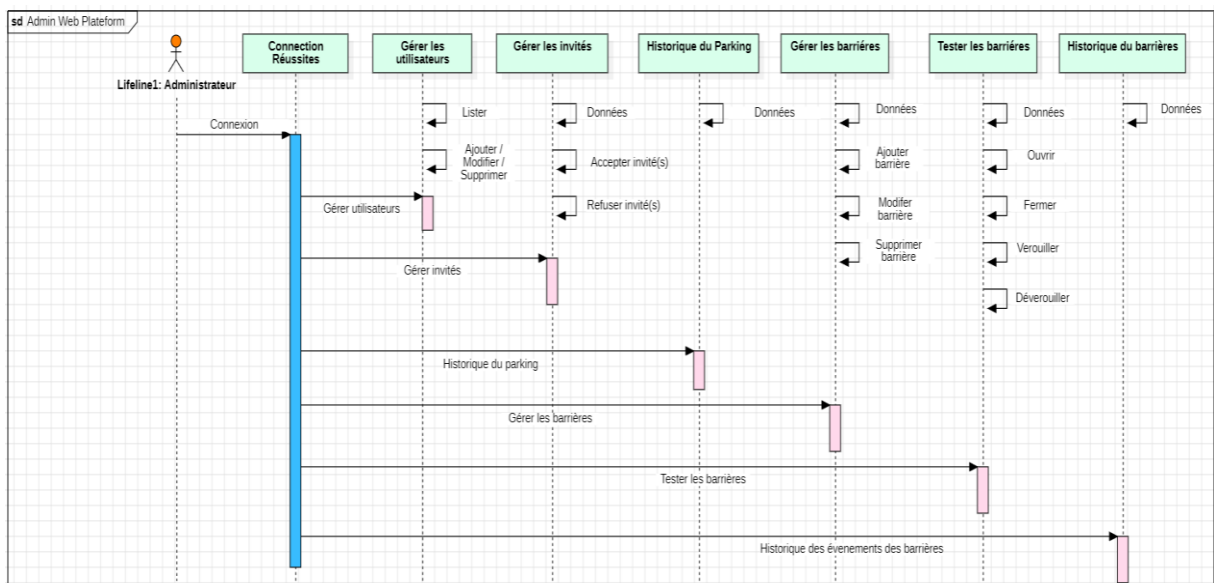


Figure 11 . Diagramme de Séquence du sprint 1

4. Implémentation et technologie utilisé

Nous commençons par configurer notre environnement de développement, nous choisissons donc de travailler avec ces outils afin de développer une interface web bien fonctionnelle et bien structurée.

Voici un aperçu de chaque élément utilisé :

4.1 Frontend

- **HTML** : Utilisé pour structurer le contenu des pages web du notre projet.
- **CSS** : Utilisé pour styliser et disposer les éléments de manière attrayante.
- **JavaScript** : Utilisé pour ajouter des fonctionnalités dynamiques et interactives aux pages web.

4.2 Backend

- **Python** : Langage de programmation utilisé pour écrire la logique côté serveur.
- **Jinja** : Moteur de Template pour Python, permettant de générer du HTML dynamique et exécuter la partie frontend avec la partie backend.
- **Docker** : Outil de virtualisation pour créer des conteneurs légers et portables pour les applications afin de l'exécuter dans différent machines
- **Uvicorn** : Serveur ASGI rapide et léger pour exécuter les applications Python.
- **Environnement Virtuel** : Un espace de développement isolé où nous installons les logiciels et paquets Python nécessaires, isolant ainsi ces paquets du reste de l'environnement global de notre machine.
- **WSL** : Le Sous-système Windows pour Linux (WSL) est une fonctionnalité de Windows qui vous permet d'exécuter un environnement Linux sur votre machine Windows, sans avoir besoin d'une machine virtuelle séparée ou d'un double démarrage.

4.3 Liaison entre Frontend et Backend

- **FastAPI** : Framework web moderne et rapide pour construire des API avec Python.
- **Swagger UI** : Interface utilisateur interactive pour tester et documenter les API créées avec FastAPI.

4.4 Base de Données

- **MongoDB** : MongoDB est une base de données NoSQL orientée document, flexible et évolutive, idéale pour les données non structurées.
- **SQLite** : SQLite est une base de données relationnelle embarquée, légère et sans configuration, idéale pour les applications mobiles et les petits projets.

Cette architecture semble bien équilibrée et utilise des technologies modernes pour développer une interface web fonctionnelle et bien structurée.

5. Description détaillée du sprint 1

Dans ce point, nous avons précisé comment s'il fonctionne notre système avec une description plus détaillée.

5.1 Description 'Authentification'

L'utilisateur (administrateur) peut accéder à l'application à l'aide de son compte.

Pour ce faire, il doit saisir son adresse email et son mot de passe.

- Diagramme de cas d'utilisation :

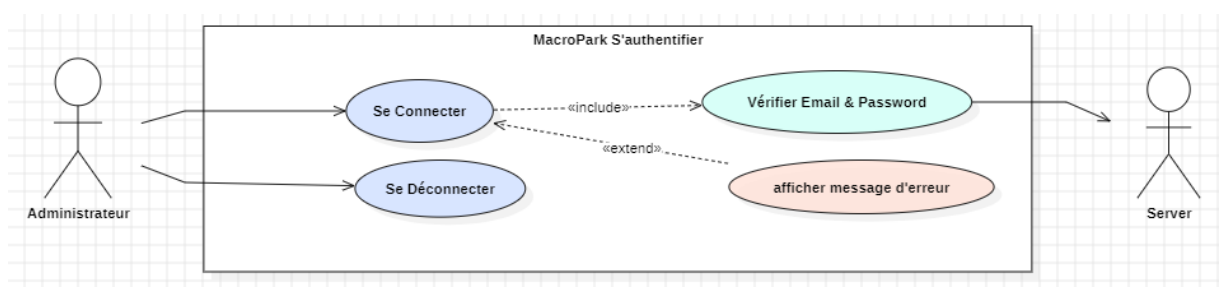


Figure 12 . Diagramme de cas d'utilisation 'Authentification' / Sprint 1

- Description textuelle :

Tableau 8. Description textuelle 'Authentication' / Sprint 1

Cas	S'authentifier
Acteurs	Administrateur
Précondition	Permet l'admin à accéder à l'interface web
Scénario nominal	• L'admin accède à la page d'authentification • Le système fait appel à la page d'authentification • L'admin saisit son login et son mot de passe • Le système vérifie les informations saisies • Le système fait appel à l'interface adéquate
Scénario alternatif	Si les champs sont erronés ou vides : – Le système les indique dans le formulaire Si le login ou le mot de passe est invalide : – Le système affiche un message d'erreur

5.2 Description 'Consulter et gérer la liste des utilisateurs'

- Diagramme de cas d'utilisation :

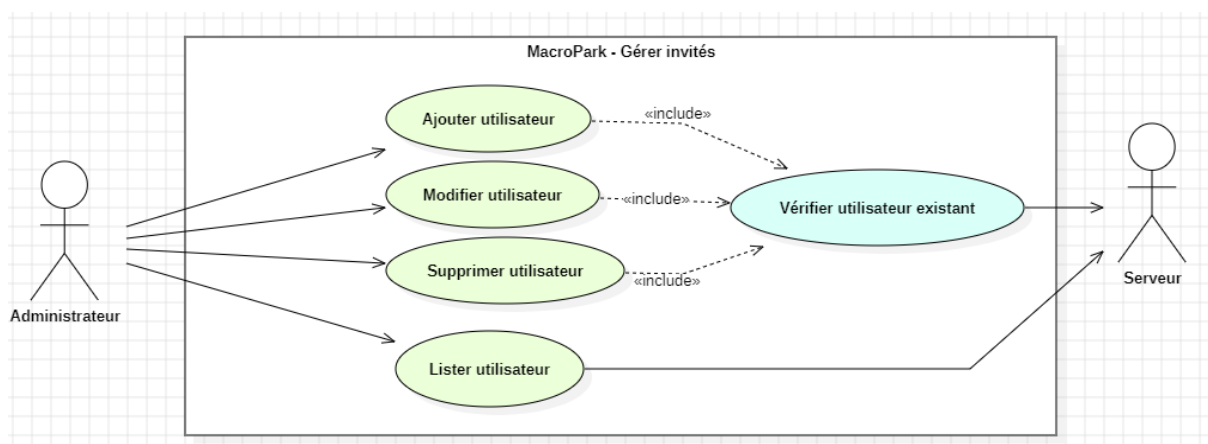


Figure 13. Diagramme de cas d'utilisation 'Utilisateurs' / Sprint 1

- Description textuelle :

Tableau 9. Description textuelle 'Utilisateurs' / Sprint 1

Cas	Consulter et gérer la liste des utilisateurs.
Acteurs	Administrateur
Précondition	Permet à l'admin à gérer et consulter la liste des utilisateurs
Scénario nominal	• Le système fait appel à la liste des utilisateurs. • L'admin consulte la liste des utilisateurs • L'admin ajout un utilisateur • L'admin modifier les informations d'un utilisateur • L'admin supprimer un utilisateur

5.3 Description 'Consulter et gérer la liste des invités'

- Diagramme de cas d'utilisation :

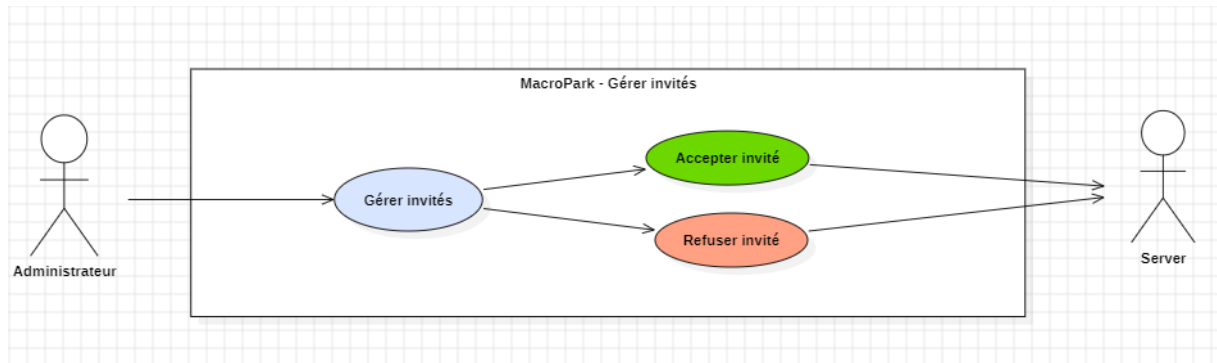


Figure 14 . Diagramme de cas d'utilisation 'invité' / Sprint 1

- Description textuelle :

Tableau 10. Description textuelle 'invités' / Sprint 1

Cas	Consulter et gérer la liste des invités.
Acteurs	Administrateur
Précondition	Permet à l'admin à gérer et consulter la liste des invités.
Scénario nominal	• L'admin accède à la page des invités. • Le système fait appel à l'interface adéquate. • Le système fait appel à la liste des invités. • L'admin consulte la liste des invités • L'admin accepter un / les invité(s) • L'admin refuser un / les invité(s)

5.4 Description 'Consulter l'historique des entrées au parking'

- Diagramme de cas d'utilisation :

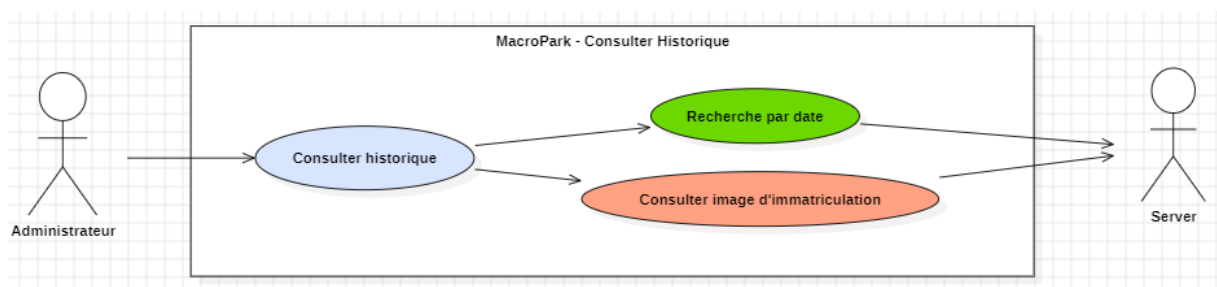


Figure 15 . Diagramme de cas d'utilisation 'Consulter historique' / Sprint 1

- Description textuelle :

Tableau 11. Description textuelle 'invités' / Sprint 1

Cas	Consulter l'historique des entrées au parking
Acteurs	Administrateur
Précondition	Permet l'admin à consulter l'historique des entrées au parking
Scénario nominal	• L'admin accède à la page d'historique. • Le système fait appel à l'interface adéquate. • Le système fait appel à la liste des historiques. • L'admin effectue une recherche par date. • L'admin consulte l'image de chaque historique.

5.5 Description ' Consulter et gérer la liste des barrières '

- Diagramme de cas d'utilisation :

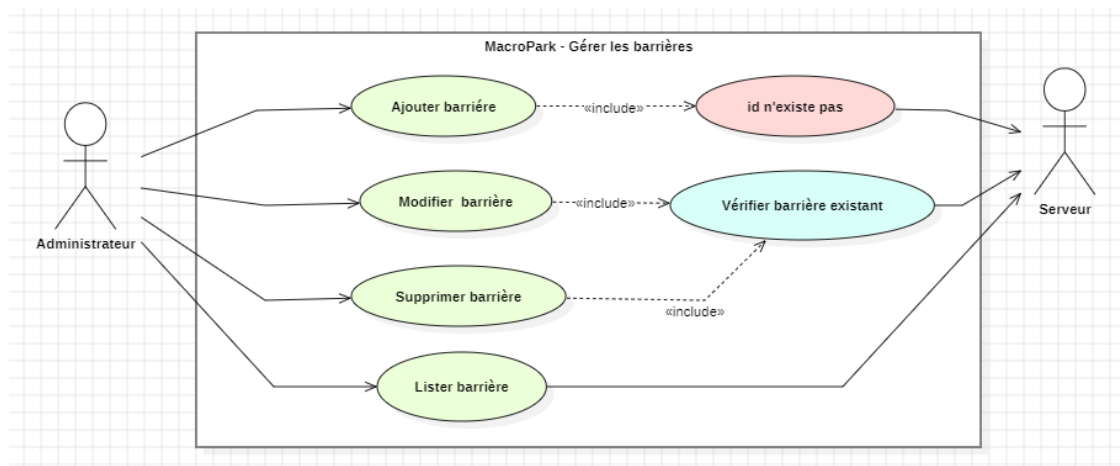


Figure 16 . Diagramme de cas d'utilisation 'barrière' / Sprint 1

- Description textuelle :

Tableau 11 . Description textuelle 'Consulter barrières' / Sprint 1

Cas	Consulter et gérer la liste des barrières
Acteurs	Administrateur
Précondition	Permet l'admin à consulter et gérer les barrières
Scénario nominal	• L'admin accède à la page des barrières • Le système fait appel à la liste des barrières. • L'admin consulte la liste des barrières • L'admin ajout une barrière • L'admin modifier les informations d'une barrière • L'admin supprimer une barrière
Scénario alternatif	Si l'identifiant de la barrière est trouvé un message d'erreur va être affiché lors de l'ajout d'une barrière

5.6 Description ‘ Tester les barrières ’

- Diagramme de cas d'utilisation :

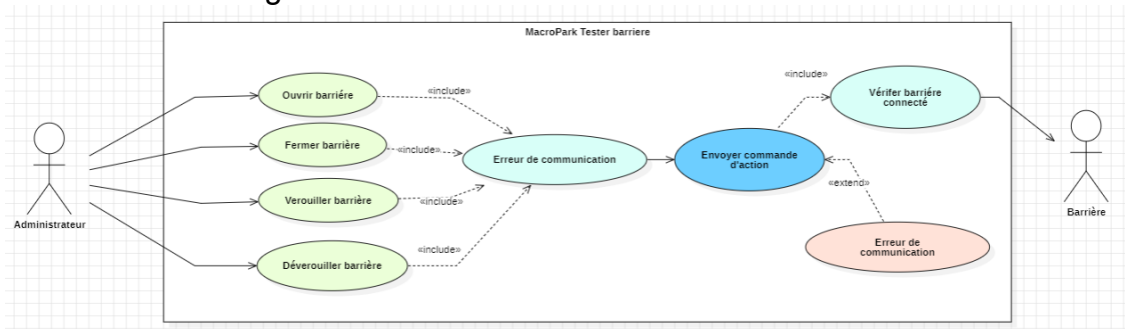


Figure 17 . Diagramme de cas d'utilisation 'Tester barrière' / Sprint 1

- Description textuelle :

Tableau 12 . Description textuelle ‘Tester les barrières’ / Sprint 1

Cas	Tester les barrières
Acteurs	Administrateur
Précondition	Permet à l’admin pour tester les barrières manuellement
Scénario nominal	- L’admin accède à la page de test - Le système fait appel à la liste des barrières - L’admin consulte la liste des barrières - L’admin peut commander manuellement les barrières (Ouvrir, Fermer, Verrouiller et Déverrouiller)
Scénario alternatif	Si la barrière n’a pas connecté un message d’erreur d’affiche

5.7 Description ‘ Consulter l’historique des barrières ’

- Diagramme de cas d'utilisation :

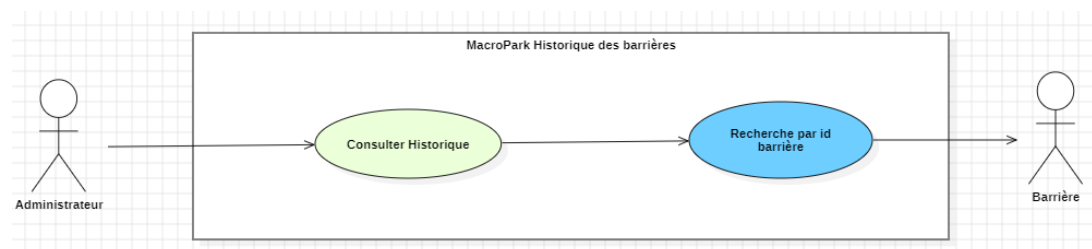


Figure 18 . Diagramme de cas d'utilisation 'Historique des barrières' / Sprint 1

- Description textuelle :

Tableau 13 . Description textuelle 'Historique barrières' / Sprint 1

Cas	Consulter l'historique des barrières
Acteurs	Administrateur
Précondition	Permet l'admin à consulter l'historique des barrières
Scénario nominal	• L'admin accède à la page d'historique • Le système fait appel à la page • L'admin effectue une recherche par identifiant de la barrière • Le système vérifie l'identifiant saisie • Le système fait appel à les données adéquats
Scénario alternatif	Si aucun historique relatif à l'identifiant de la barrière saisie un message ' no event found ' s'affiche.

6. Réalisation

Dans cette partie on a l'opportunité du parler un peu sur la réalisation de ce sprint et les différentes captures d'écran du notre interface ainsi qu'une description courte des parties complexes du code.

6.1 Interfaces graphiques

- Page de connexion

La page de connexion pour l'administrateur offre une interface simple et intuitive pour accéder à l'espace Admin, elle se compose de deux champs essentiels qui permettent au admin de se connecter en toute sécurité.

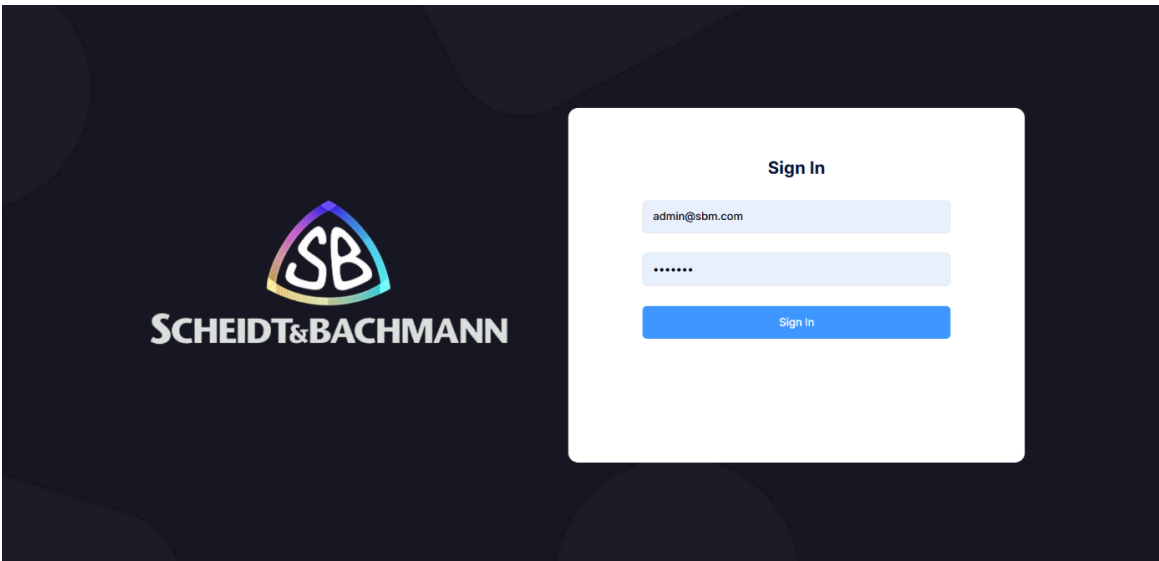


Figure 19. Page de connexion / Sprint 1

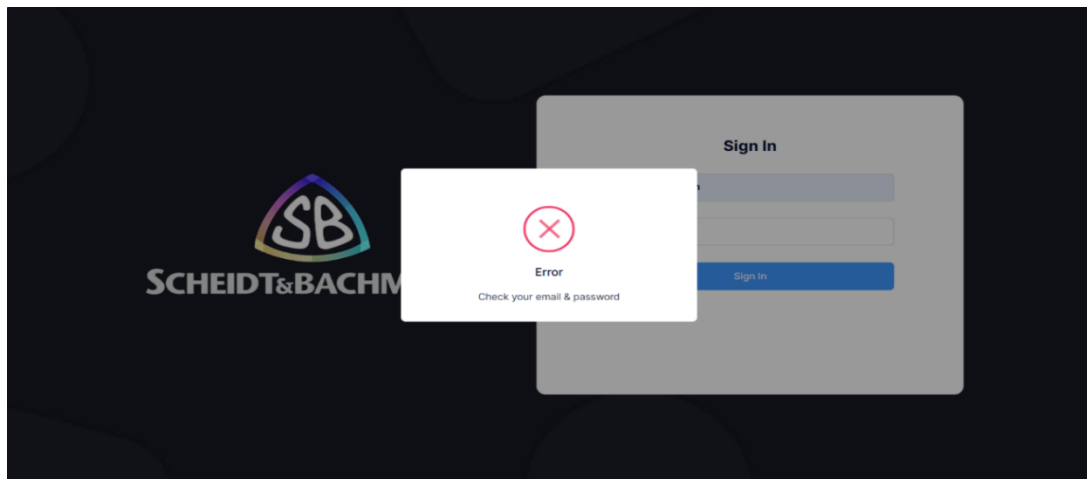


Figure 20. Erreur de connexion / Sprint 1

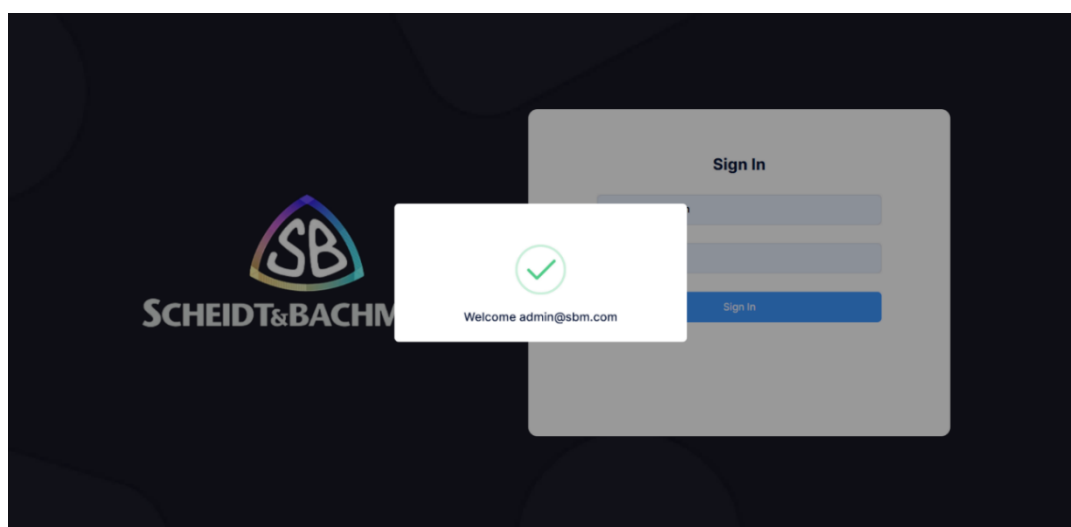


Figure 21. Connexion réussie / Sprint 1

- **Page d'accueil :**

C'est une page définie pour visualiser les différentes opérations de l'interface admin, c'est la page d'entrée à notre tableau de bord d'administrateur du parking pour une gestion facile et simple à utiliser.

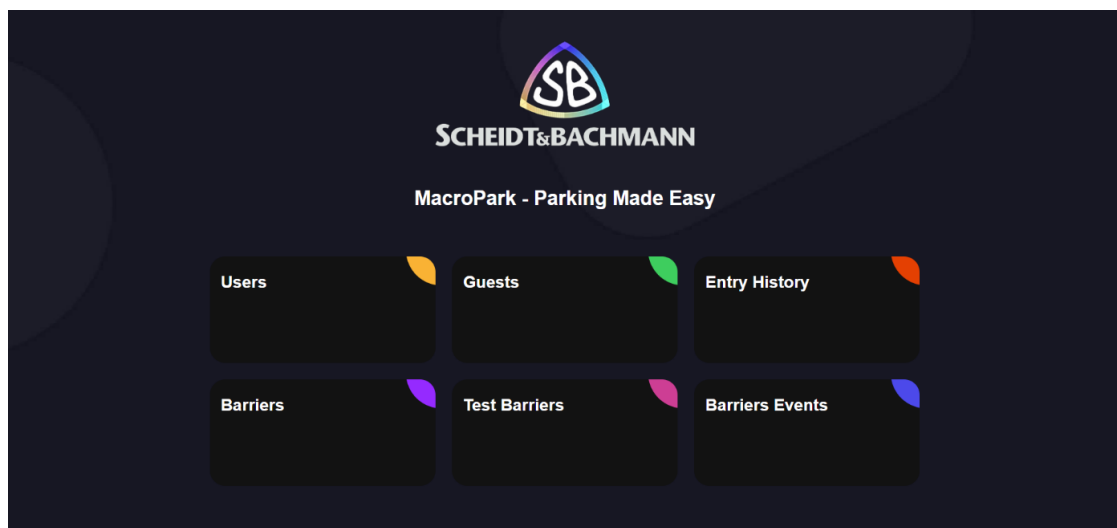


Figure 22 : Page d'accueil / Sprint 1

- **Page Users :**

La page « **Users** » offre une interface simple pour consulter et gérer les utilisateurs, elle se compose d'une liste des utilisateurs tels qu'un bouton '**Add user**' pour ajouter un utilisateur, un bouton '**Modify**' pour modifier les informations de chaque utilisateur et bouton '**Delete**' pour supprimer un utilisateur.

- **Lister Utilisateurs :**

ID	Name	Phone Number	Email	Password	LPN 1	LPN 2	LPN 3	LPN 4
199	Yassine Manai	93014027	yassinemanai955@gmail.com	*****	1254TN44	554TN444	49TN4316	236TN4556
288	Sofiene Zayati	55321315	sofienezayati87@gmail.com	*****	138TN4516	49TN4314	49TN4316	
264	Zied Bradai	29 430 505	zied.bradai@scheidt-bachmann.de	*****	214TN5521	165TN4472		
271	Mohamed Ali Ali	22445566	medali@gmail.com	*****	144TN2525			

Figure 24 . Lister utilisateurs (Page Users) / Sprint 1

ID	Name	Phone Number	Email	Password	LPN 1	LPN 2	LPN 3	LPN 4
No Users found								

Figure25. Liste utilisateurs vide (Page Users) / Sprint 1

- **Ajouter Utilisateur :**

Un bouton **"ADD USER"** est conçu pour l'ajout d'utilisateur. En cliquant sur ce bouton, une fenêtre s'affiche permettant de saisir les informations d'un nouvel utilisateur comprenant : Identifiant, Nom, Numéro de téléphone, Adresse e-mail, Mot de passe (par défaut : 123456), ainsi que 4 immatriculations (dont un est obligatoire pour chaque utilisateur, les trois autres étant optionnelles).

The 'Add user information' modal form includes the following fields:

- User ID *
- Name *
- Phone Number *
- Email *
- Password * (default: 123456)
- Licence Plate Number 1 *
- Licence Plate Number 2
- Licence Plate Number 3
- Licence Plate Number 4

The background 'Users List' table shows:

ID	Name
199	Yassine Manai
288	Sofiene Zayati
264	Zied Bradai
271	Mohamed Ali Ali

Figure 26. Ajouter utilisateur (Page Users) / Sprint 1

- **Modifier utilisateur :**

The 'Modify user information' modal form includes the following fields:

- Name *
- Phone Number *
- Email *
- Password *
- Licence Plate Number 1 *
- Licence Plate Number 2
- Licence Plate Number 3
- Licence Plate Number 4

The background 'Users List' table shows:

ID	Name
212	Yassine Manai
274	Sofiene Zayati
219	Zied Bradai

Figure 27. Modifier Utilisateur (Page Users) / Sprint 1

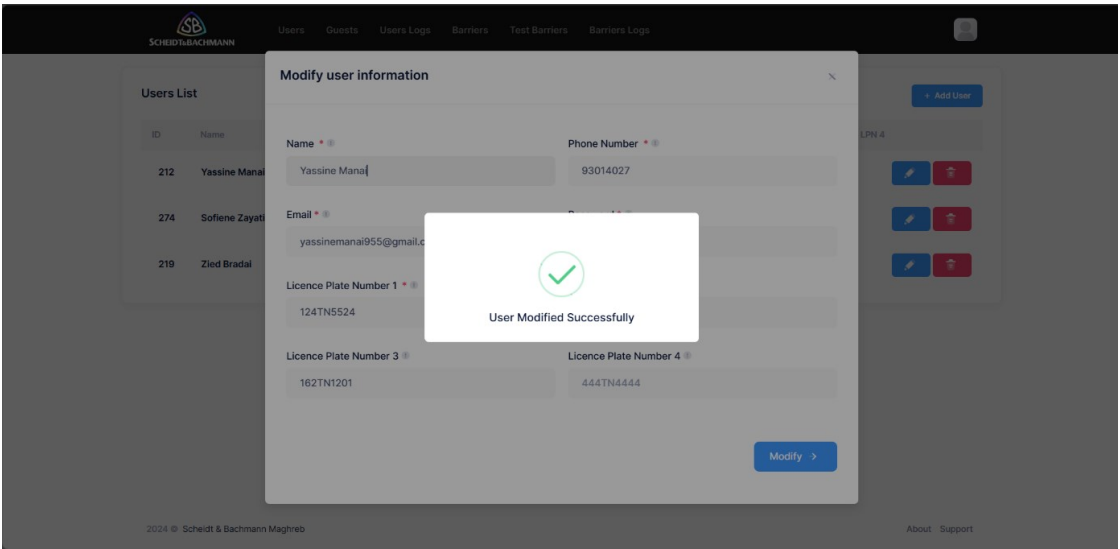


Figure 28. Modification réussite (Page Users) / Sprint 1

- **Supprimer Utilisateur :**

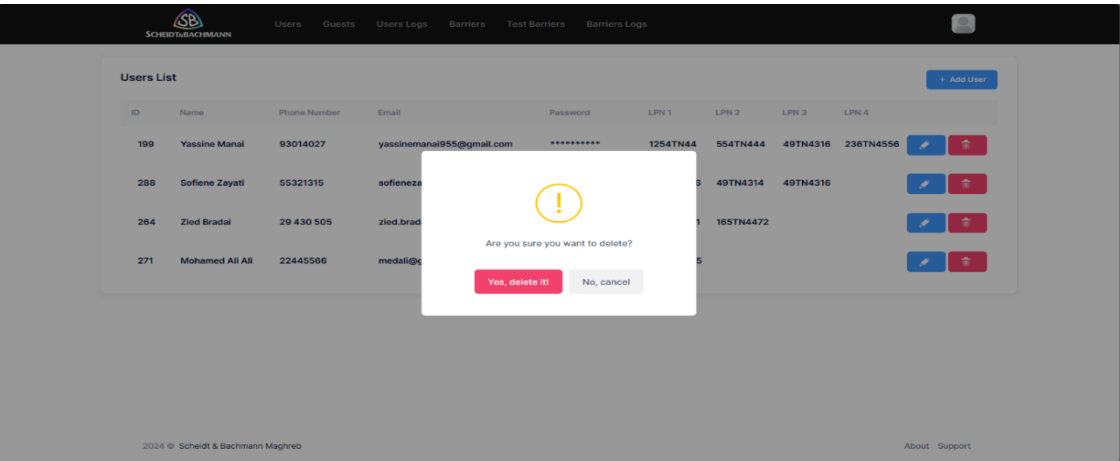
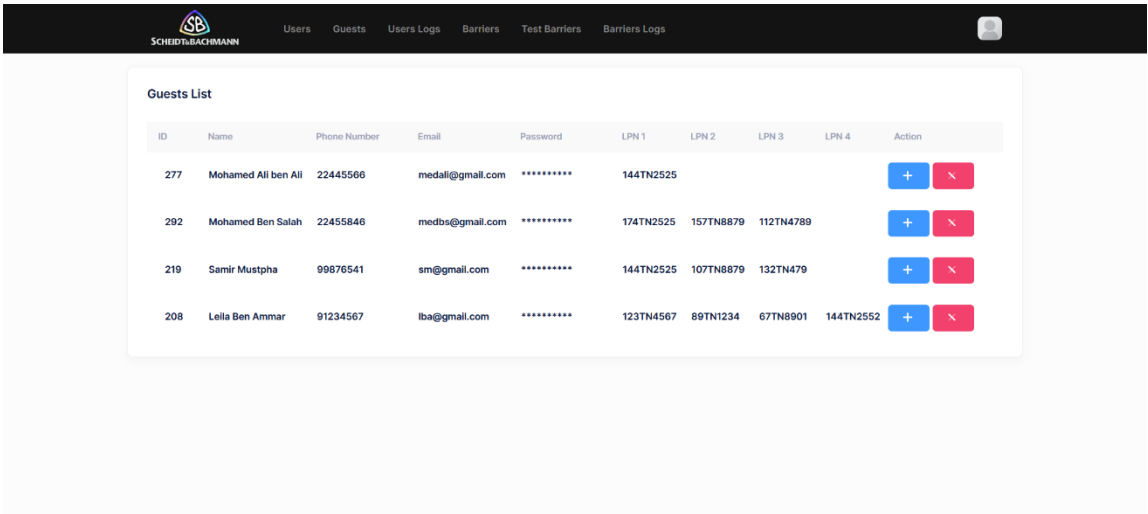


Figure 29. Supprimer utilisateurs (Page Users) / Sprint 1

- **Page Guests :**

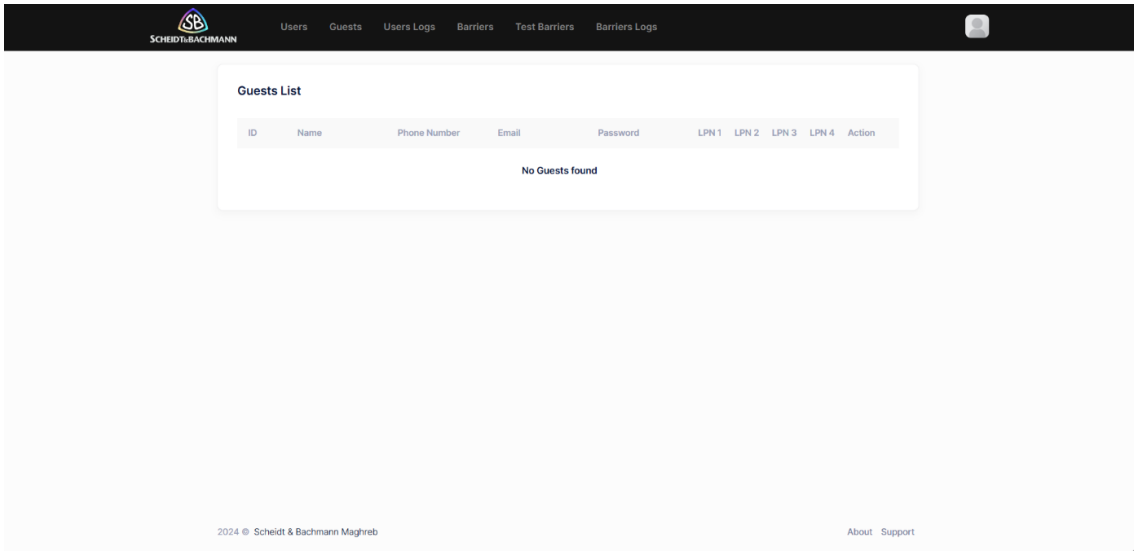
La page « **Guests** » est créé pour consulter et gérer les invités qui sont inscrits par l'application mobile et qui sont dehors de la société, elle se compose d'une liste des invités tels un bouton accepter pour accepter '**Approuve**' un invité et un bouton refuser '**Decline**' pour l'a supprimé, si l'admin l'accepte l'invité a eu un accès d'entrée au parking et d'utiliser l'application mobile sinon il n'a pas l'accès.

- **Lister invités :**



ID	Name	Phone Number	Email	Password	LPN 1	LPN 2	LPN 3	LPN 4	Action
277	Mohamed Ali ben Ali	22445566	medali@gmail.com	*****	144TN2525				+ x i
292	Mohamed Ben Salah	22455846	medbs@gmail.com	*****	174TN2525	157TN8879	112TN4789		+ x i
219	Samir Mustpha	99876541	sm@gmail.com	*****	144TN2525	107TN8879	132TN479		+ x i
208	Lella Ben Ammar	91234567	lba@gmail.com	*****	123TN4567	89TN1234	67TN8901	144TN2552	+ x i

Figure 30 . Liste d'invité (Page invité) / Sprint 1



ID	Name	Phone Number	Email	Password	LPN 1	LPN 2	LPN 3	LPN 4	Action
No Guests found									

Figure 31. Liste vide (Page Guests) / Sprint 1

• **Accepter invité :**

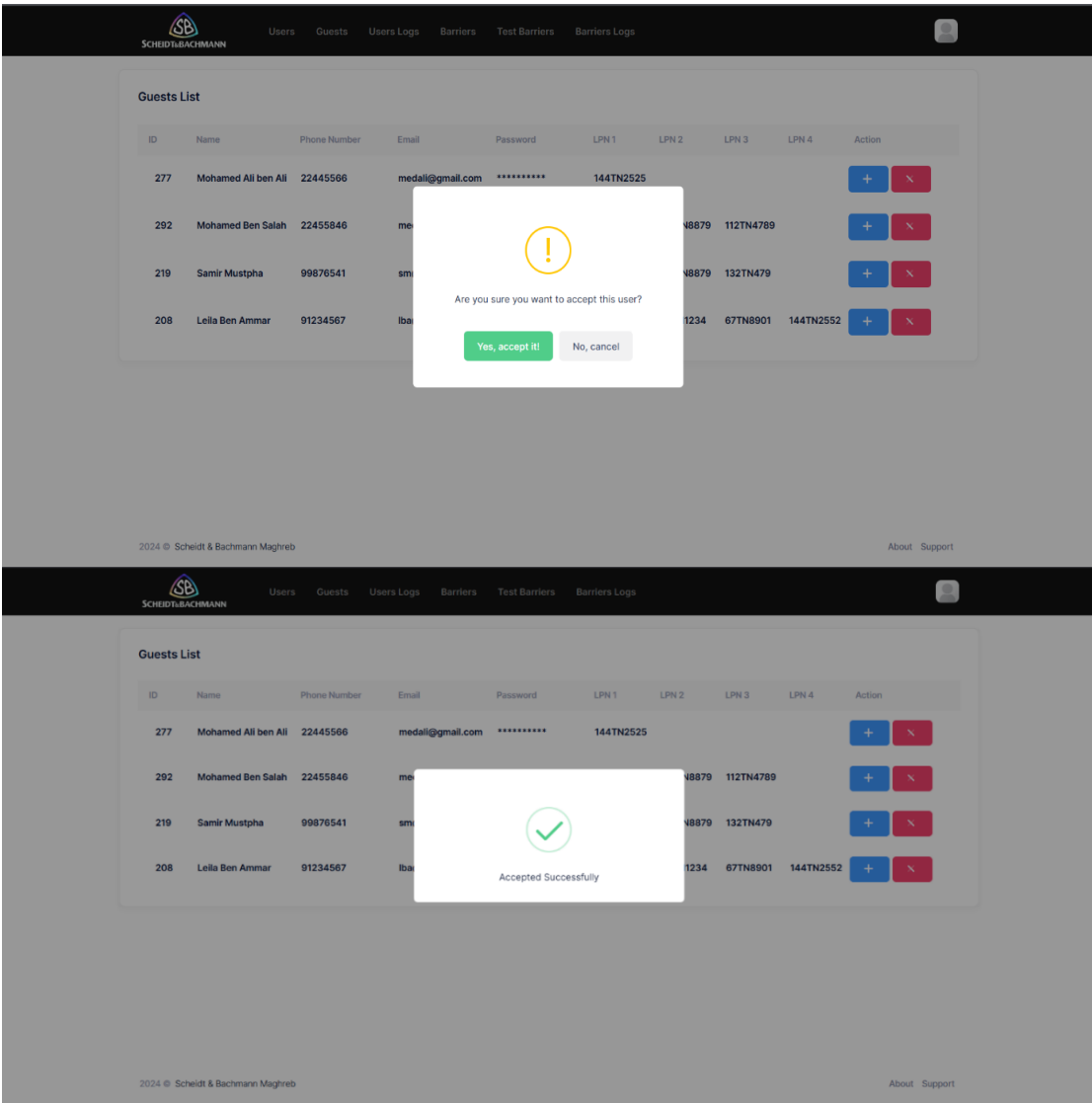


Figure32. Accepter invité (Page Guests) / Sprint 1

- Refuser invité :

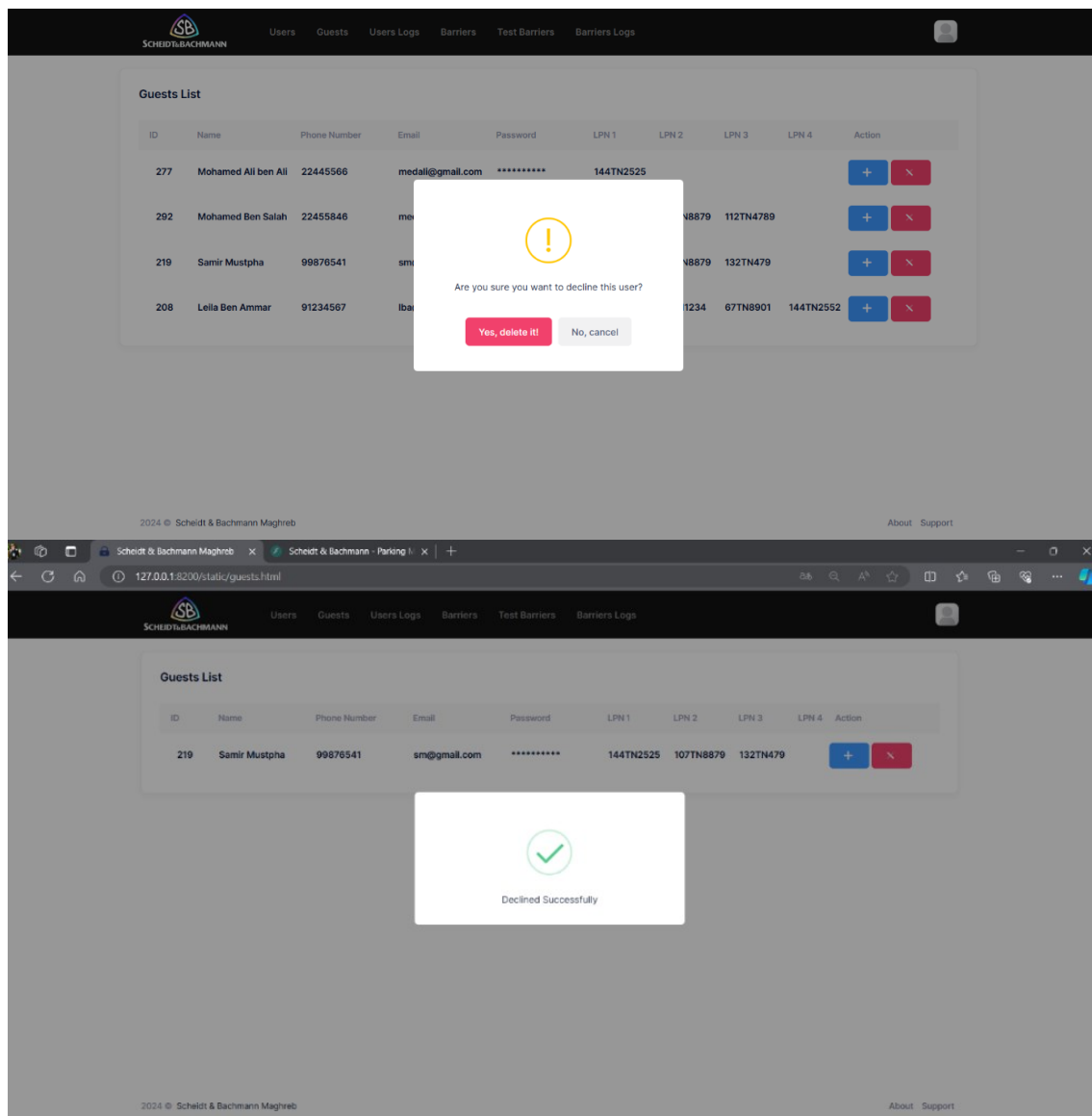


Figure33.Refuser invité (Page Guests) / Sprint 1

- **Page User Logs :**

La page « **Users Logs** » offre une interface facile pour consulter l'historique des entrées au parking, elle se compose d'une liste d'historique avec une fonctionnalité de recherche par date et un bouton '**Show Image**' qui ouvre une fenêtre affiche l'image d'immatriculation capturée par la caméra.

- **Lister historique :**

SCHIEDT & BACHMANN

UsersGuestsUsers LogsBarriersTest BarriersBarriers Logs

24/05/2024

User History List

User ID	User Name	Licence Plate Number	Min Quality	Device ID	Date	Time	Method	Image
288	Sofiene Zayati	138TN4516	0.91	Entry01	2024-05-16	17:01:39	Camera Action	Show Image
288	Sofiene Zayati	138TN4516	0.91	Entry01	2024-05-16	17:02:10	Camera Action	Show Image
288	Sofiene Zayati	No Licence	0	Entry01	2024-05-16	17:18:47	Phone Action	No image found
288	Sofiene Zayati	138TN4516	0.93	Entry01	2024-05-17	12:06:05	Camera Action	Show Image
288	Sofiene Zayati	138TN4516	0.95	Entry01	2024-05-17	12:12:25	Camera Action	Show Image
288	Sofiene Zayati	138TN4516	0.96	Entry01	2024-05-17	12:15:59	Camera Action	Show Image
288	Sofiene Zayati	138TN4516	0.95	Entry01	2024-05-20	13:14:45	Camera Action	Show Image
288	Sofiene Zayati	138TN4516	0.95	Entry01	2024-05-20	13:20:12	Camera Action	Show Image
288	Sofiene Zayati	138TN4516	0.95	Entry01	2024-05-20	15:37:23	Camera Action	Show Image

Figure34. Lister Historique(Page Users Logs) / Sprint 1

- **Recherche par date :**

SCHIEDT & BACHMANN

UsersGuestsUsers LogsBarriersTest BarriersBarriers Logs

16/05/2024

User History List

User ID	User Name	Licence Plate Number	Min Quality	Device ID	Date	Time	Method	Image
288	Sofiene Zayati	No Licence	0	Entry01	2024-05-16	17:18:47	Phone Action	No image found
288	Sofiene Zayati	138TN4516	0.91	Entry01	2024-05-16	17:02:10	Camera Action	Show Image
288	Sofiene Zayati	138TN4516	0.91	Entry01	2024-05-16	17:01:39	Camera Action	Show Image

2024 © Schiedt & Bachmann Maghreb

AboutSupport

Figure35. Recherche par date (Page Users Logs) / Sprint 1

○ Recherche non trouvée :

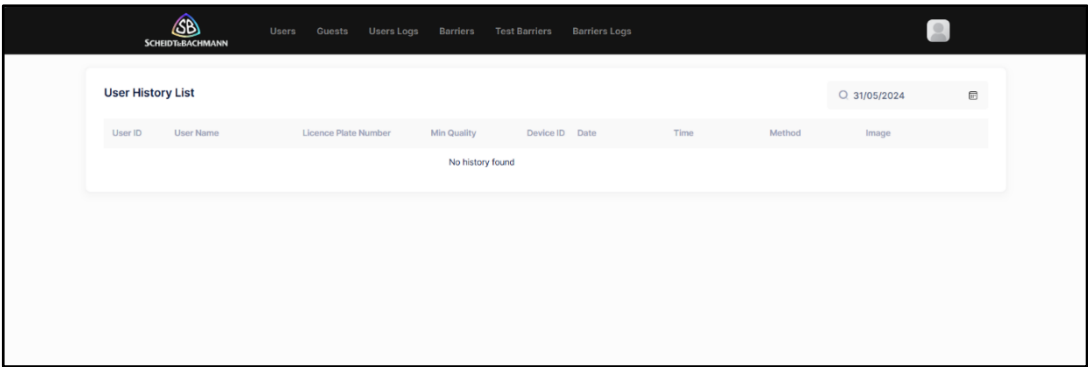


Figure 36. Recherche non trouvée (Page Users Logs) / Sprint 1

○ Fenêtre d’affichage d’image (TN) :

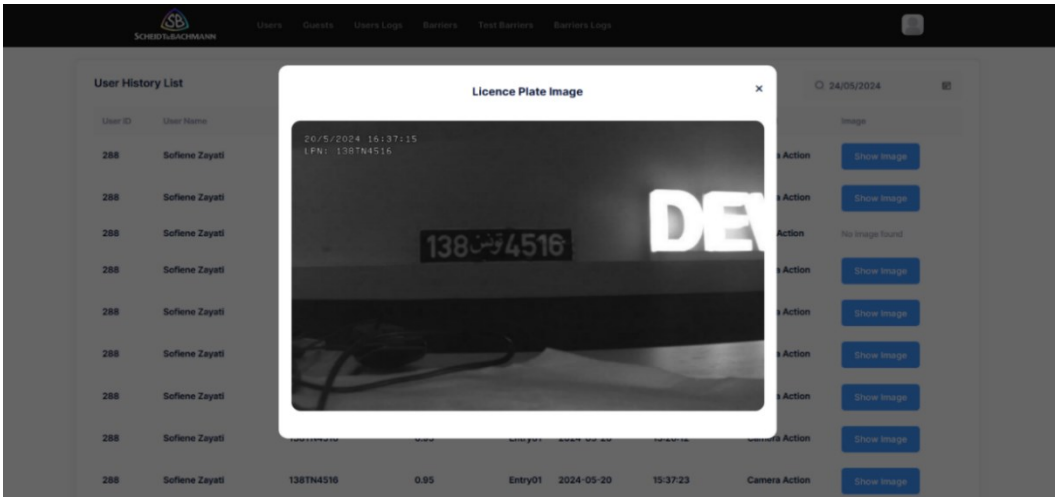


Figure 37. Fenêtre d'immatriculation TN (Page Users Logs) / Sprint 1

○ Fenêtre d’affichage d’image (RS) :

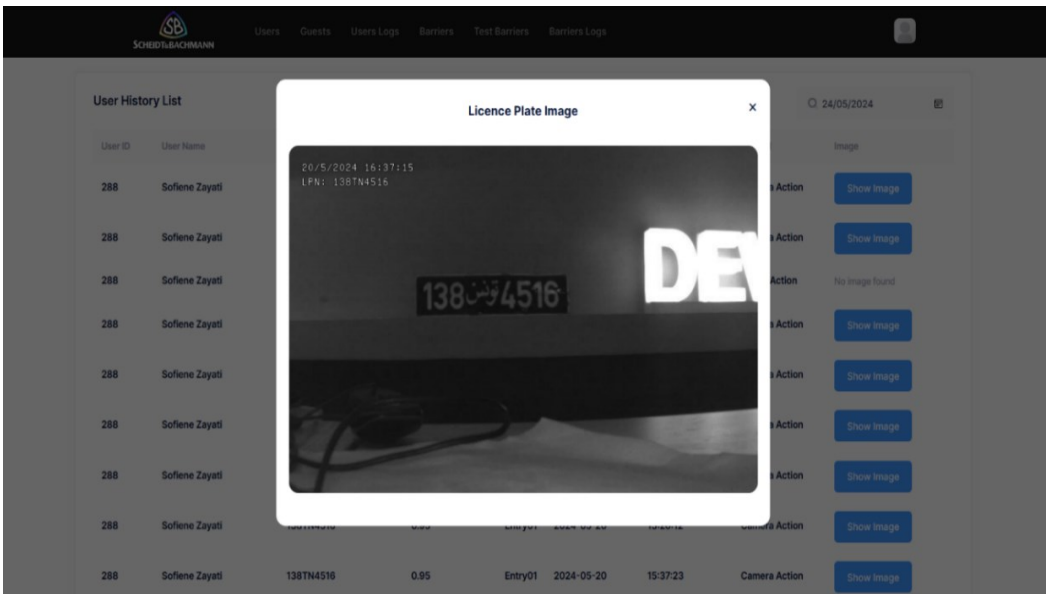


Figure 38. Fenêtre d'immatriculation RS (Page Users Logs) / Sprint 1

- **Page Barriers :**

La page « **Barriers** » offre une vue facile pour consulter et gérer la liste des barrières installés au parking, elle se compose d’une liste des barrières tels qu’un bouton ‘**Add Barrier**’ pour ajouter une barrière, un bouton ‘**Modify**’ pour modifier les informations de chaque barrière et bouton ‘**Delete**’ pour supprimer une barrière.

- Lister barrière :

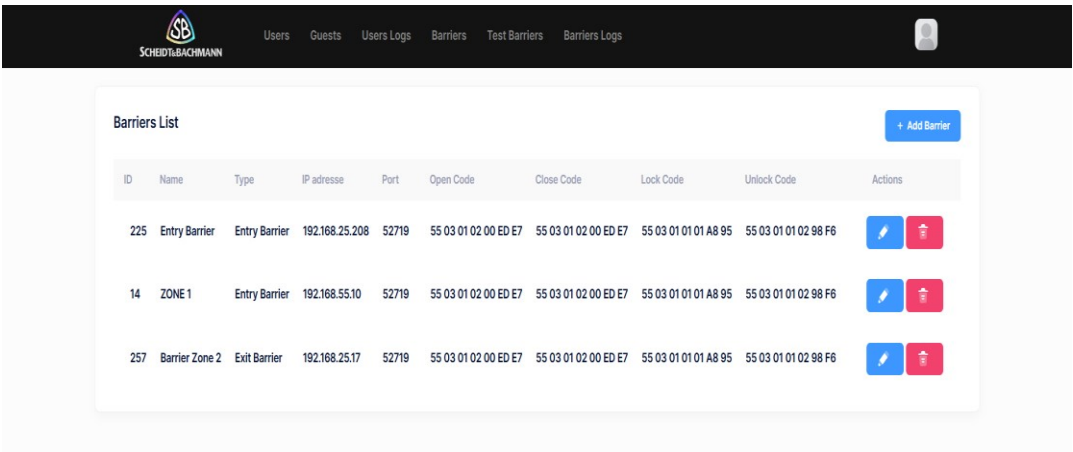


Figure39. Lister barrières (Barrier Lister) / Sprint 1

- Ajouter barrière :

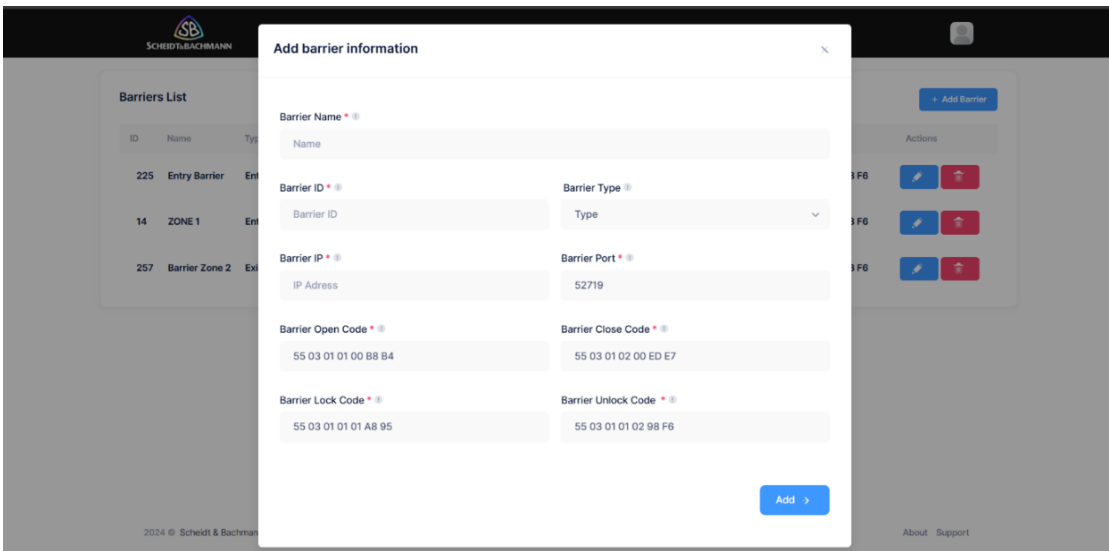


Figure40. Ajouter barrière (Barrier List) / Sprint 1

○ Modifier barrière :

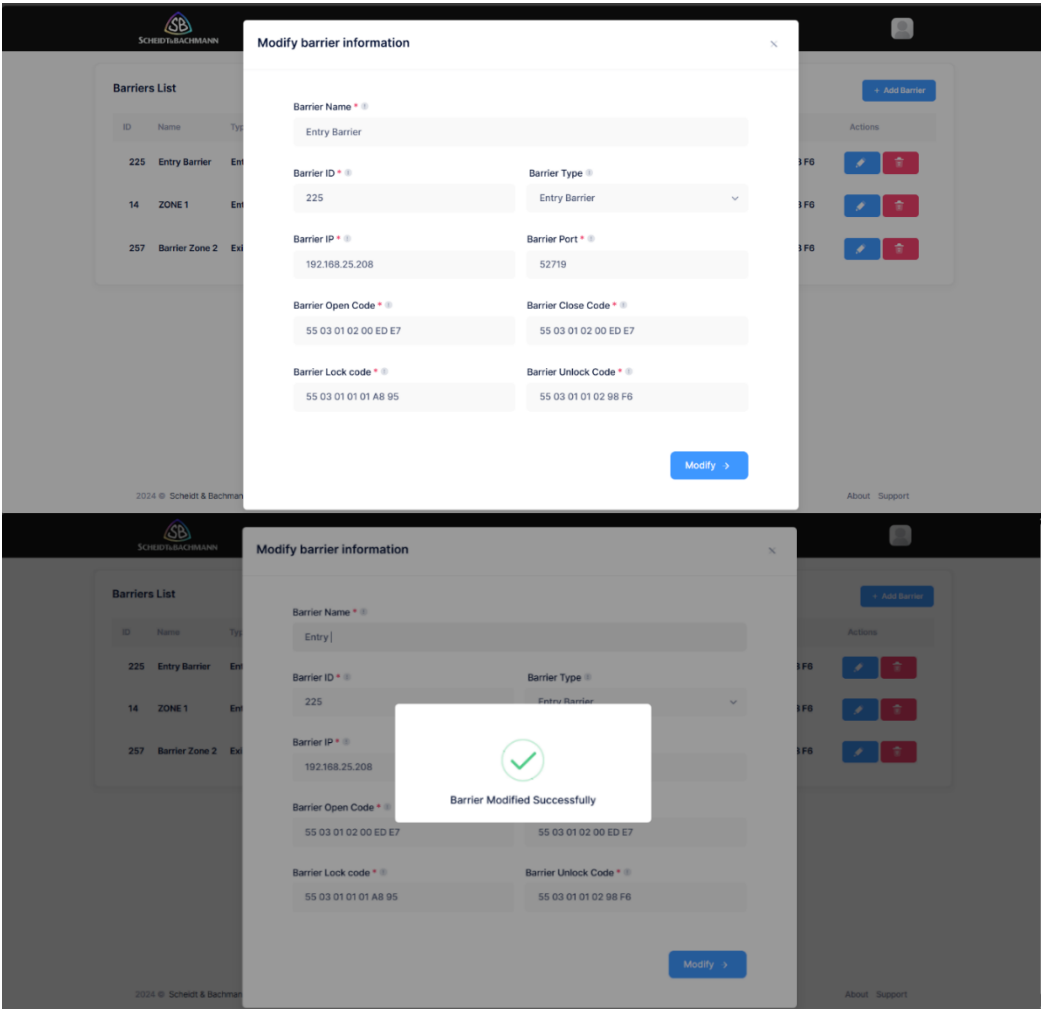


Figure 41. Modifier barrière (Barrier List) / Sprint 1

○ Supprimer barrière :

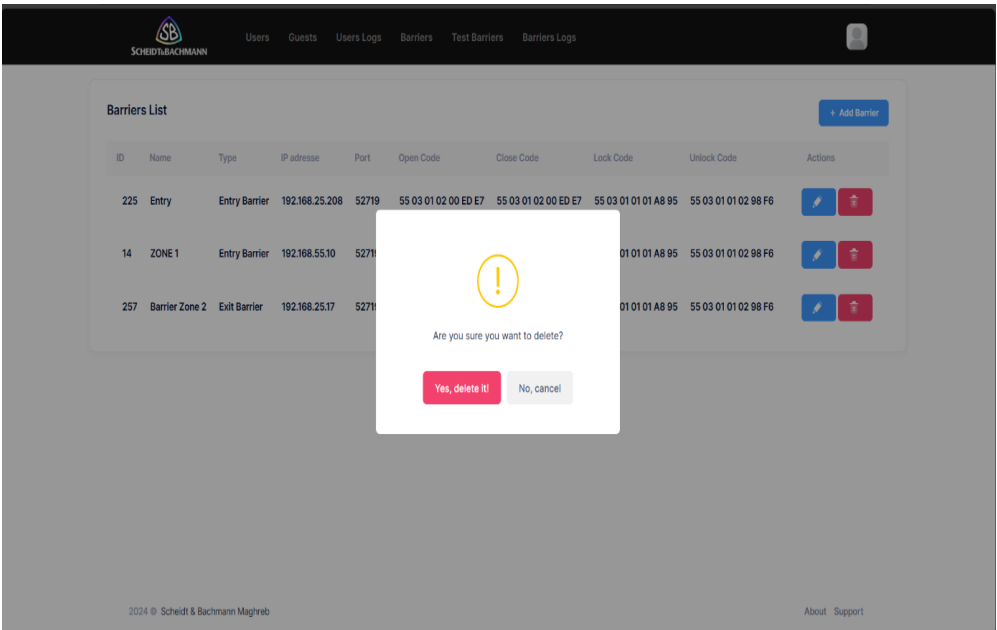


Figure 42 . Supprimer barrière (Barrier List) / Sprint 1

- **Page Entré / Sortie d'urgence :**

Dans notre interface d'administration, nous avons une section dédiée aux situations d'urgence qui permet d'ouvrir, de fermer, de verrouiller et de déverrouiller les barrières. Cette fonctionnalité utilise des messages TCP que la barrière lit et exécute. Lorsque vous cliquez sur une action, un message TCP [5] est envoyé à la barrière, et cette action est enregistrée dans les journaux. Cela permet de conserver un historique des ouvertures et fermetures manuelles, garantissant ainsi une traçabilité et une gestion efficace des urgences.

Tableau 14. Tableau des actions (TCP) [5] / Sprint 1

Action	Message TCP en Hex
Open	55 03 01 02 00 ED E7
Close	55 03 01 02 00 ED E7
Lock	55 03 01 01 01 A8 95
Unlock	55 03 01 01 02 98 F6

- Lister barrières :

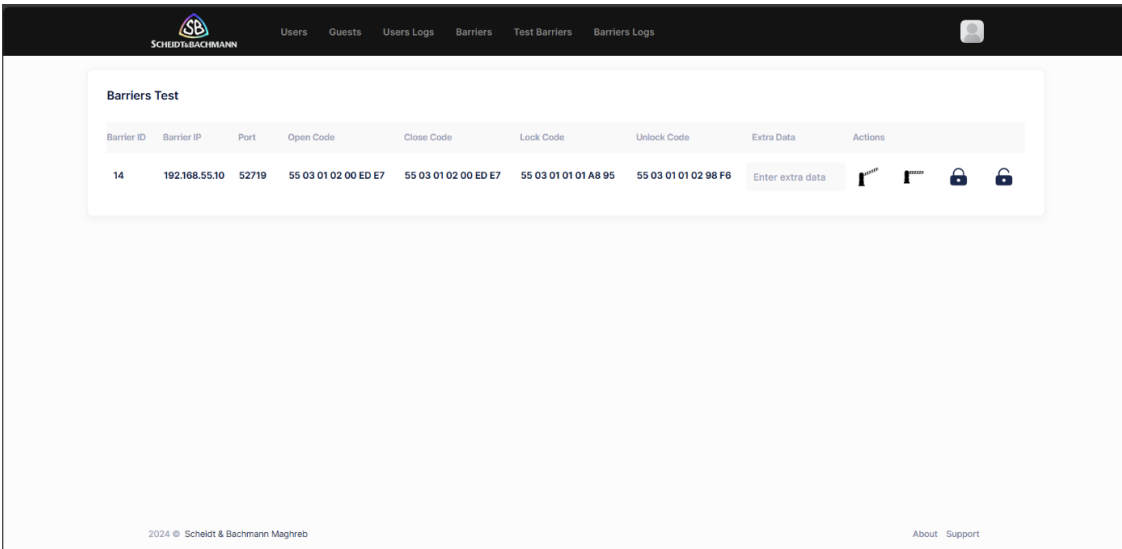


Figure 43 .Page Test barrier / Sprint 1

- Les actions d'urgence pour chaque barrière :

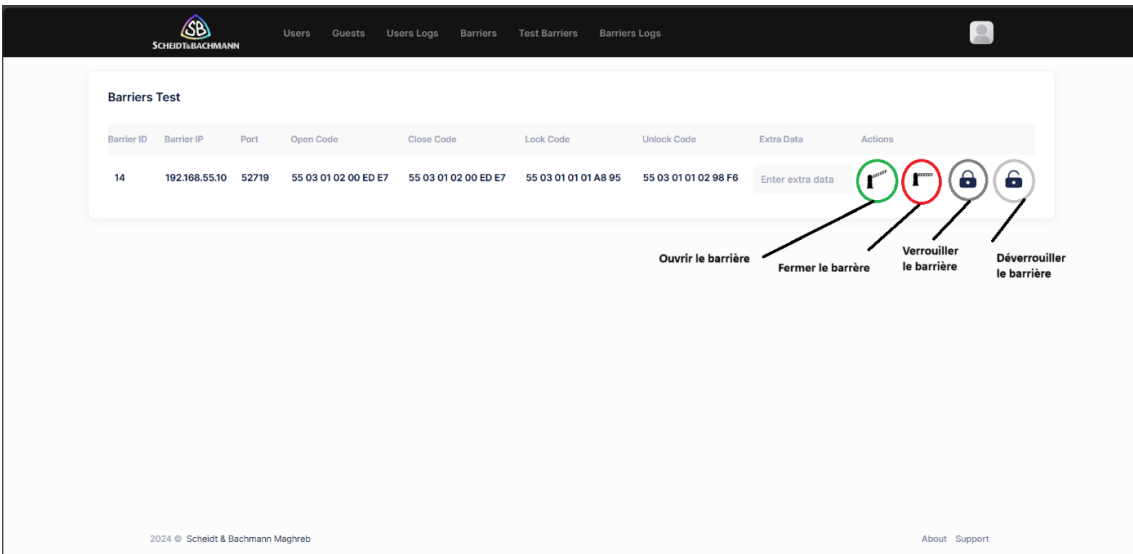


Figure44. Page Test Barrier / Sprint 1

- Les différents messages :

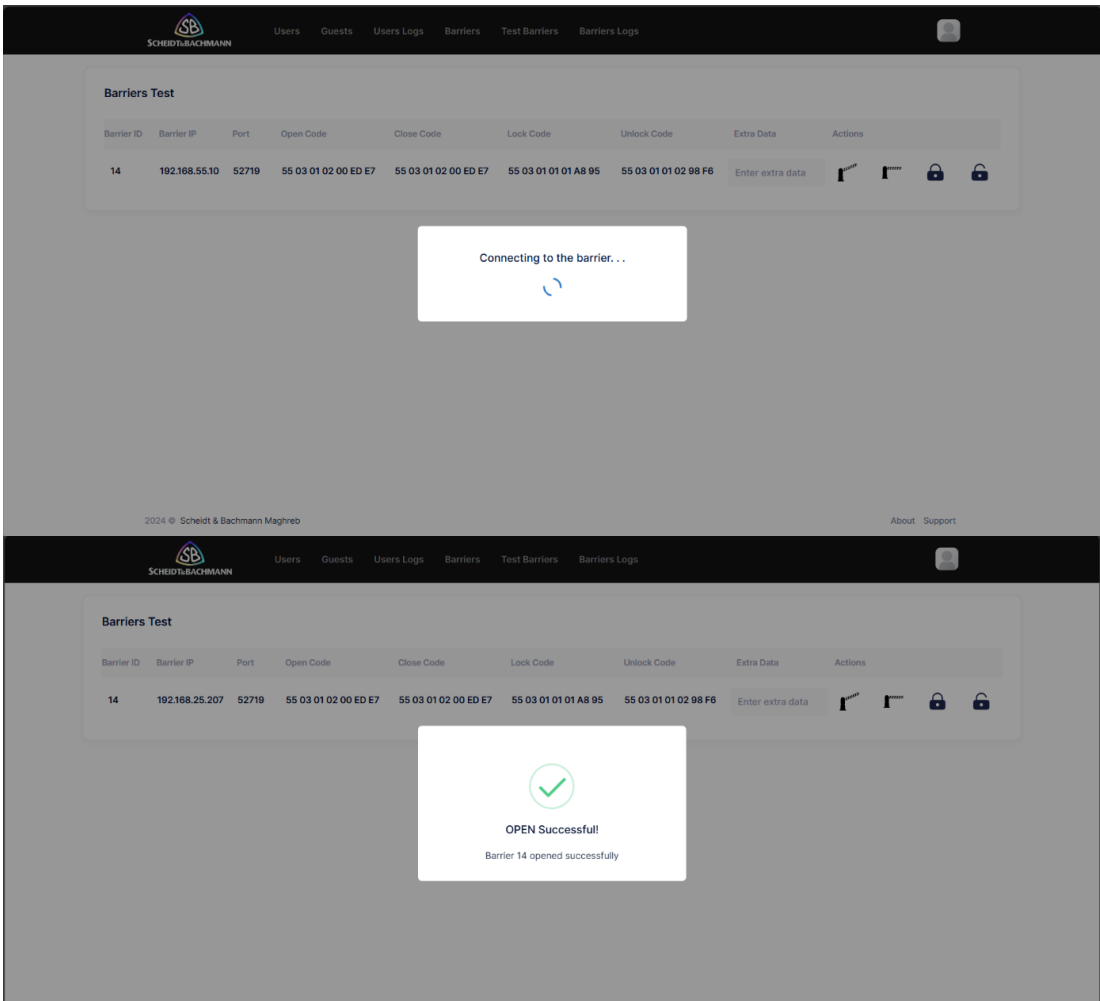
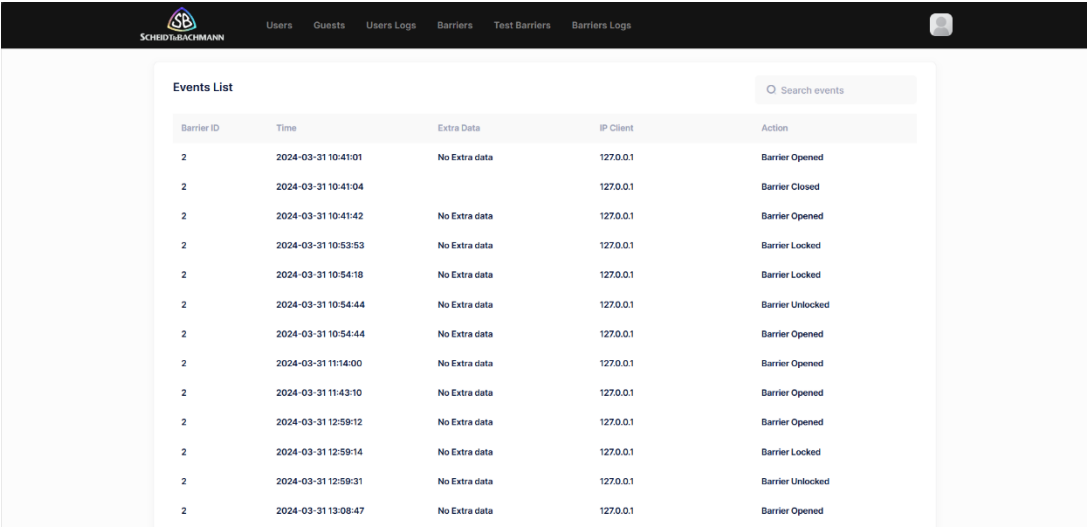


Figure45. Message d'action (Test Barriers) / Sprint 1

- **Page Barriers Logs :**

La page « **Barriers Logs** » offre vue détaillée pour consulter les évènements des barrières installés, elle se compose d'un historique tels qu'une zone de recherche '**Search Event**' pour une recherche plus optimale par l'identifiant de la barrière ainsi d'afficher le temps d'action, l'adresse IP d'hôte et l'action (Opened, Closed, Locked, Unlocked).

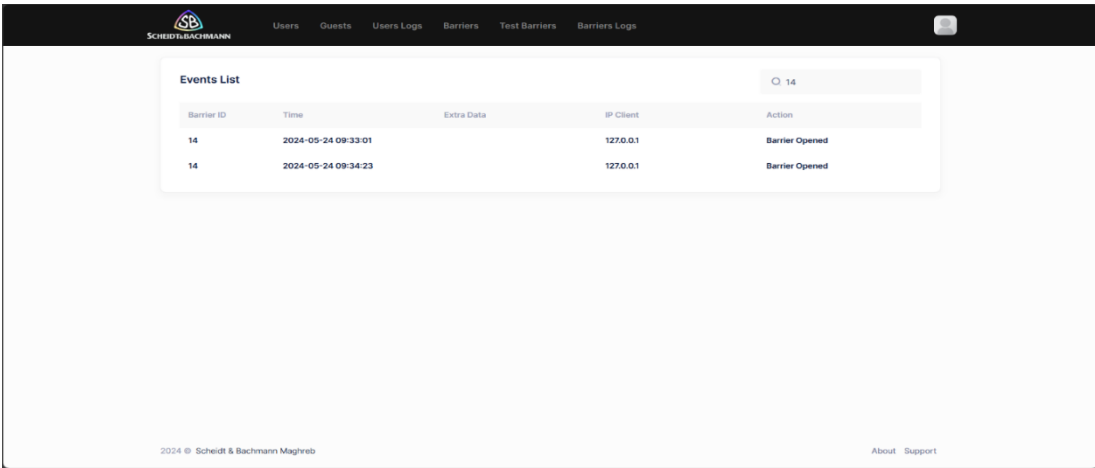
- Lister Barrières Log :



Barrier ID	Time	Extra Data	IP Client	Action
2	2024-03-31 10:41:01	No Extra data	127.0.0.1	Barrier Opened
2	2024-03-31 10:41:04		127.0.0.1	Barrier Closed
2	2024-03-31 10:41:42	No Extra data	127.0.0.1	Barrier Opened
2	2024-03-31 10:53:53	No Extra data	127.0.0.1	Barrier Locked
2	2024-03-31 10:54:18	No Extra data	127.0.0.1	Barrier Locked
2	2024-03-31 10:54:44	No Extra data	127.0.0.1	Barrier Unlocked
2	2024-03-31 10:54:44	No Extra data	127.0.0.1	Barrier Opened
2	2024-03-31 11:14:00	No Extra data	127.0.0.1	Barrier Opened
2	2024-03-31 11:43:10	No Extra data	127.0.0.1	Barrier Opened
2	2024-03-31 12:58:12	No Extra data	127.0.0.1	Barrier Opened
2	2024-03-31 12:58:14	No Extra data	127.0.0.1	Barrier Locked
2	2024-03-31 12:58:31	No Extra data	127.0.0.1	Barrier Unlocked
2	2024-03-31 13:08:47	No Extra data	127.0.0.1	Barrier Opened

Figure 46 : Historique des barrières (Test Barrier) / Sprint 1

- Recherche par identifiant de la barrière :



Barrier ID	Time	Extra Data	IP Client	Action
14	2024-05-24 09:33:01		127.0.0.1	Barrier Opened
14	2024-05-24 09:34:23		127.0.0.1	Barrier Opened

Figure47: Recherche par identifiant de la barrière / Sprint 1

- Recherche par identifiant non trouvée :

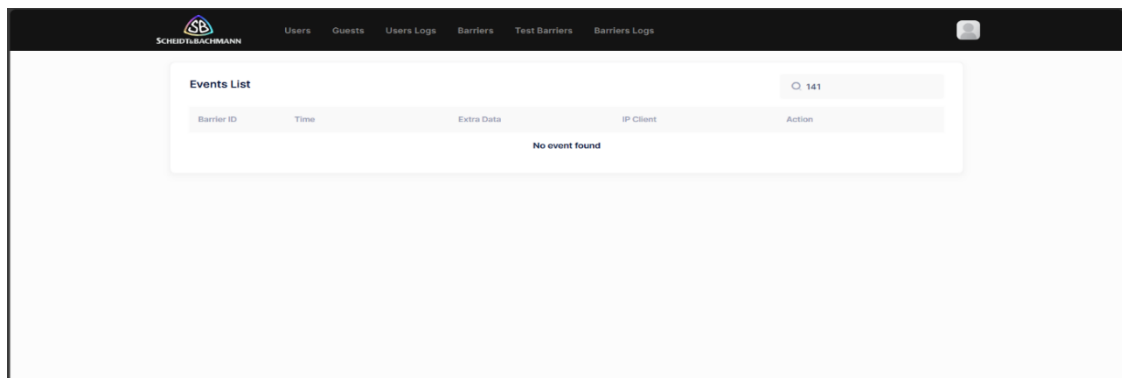


Figure 48. Recherche non trouvée / Sprint 1

- A propos de nous :

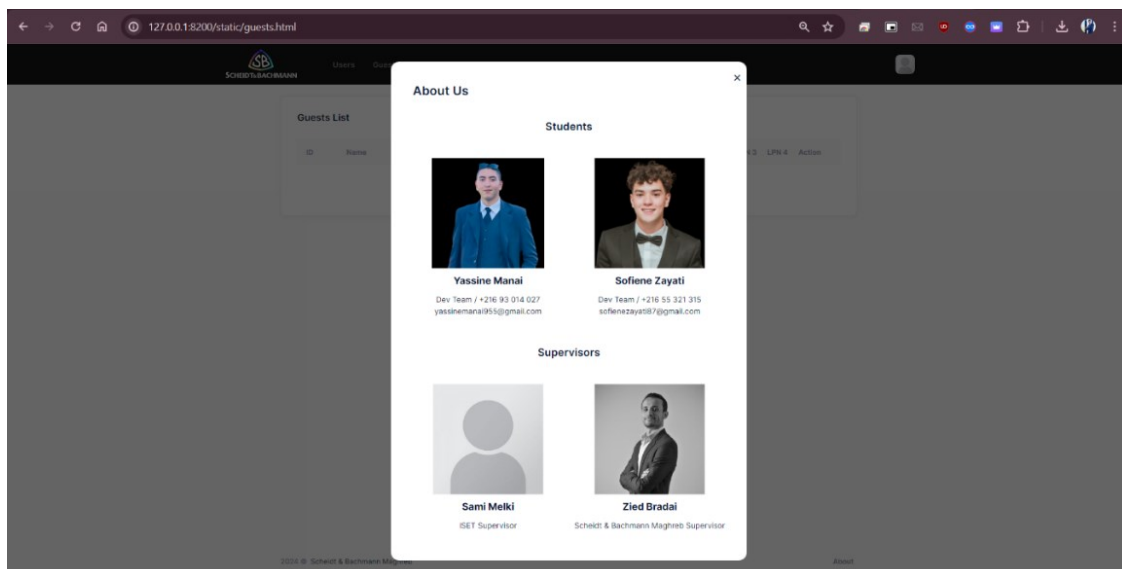


Figure49. A propos de nous / Sprint 1

6.2 Logique et Codes

Pour ce projet, nous utilisons Python et diverses bibliothèques ainsi que Docker pour la gestion des conteneurs. Voici un aperçu des étapes et des commandes nécessaires.

- **Création et Activation d'un Environnement Virtuel**

Pour isoler les dépendances de notre projet, nous utilisons un environnement virtuel Python [6] :

```
# Création de l'environnement virtuel
python -m venv venv

# Activation de l'environnement virtuel sur un système d'exploitation Linux
source ./venv/bin/activate

# exécuter notre programme
python main.py
```

- **Installation des Dépendances**

Pour gérer les dépendances, nous utilisons pip. Voici les commandes pour installer les bibliothèques nécessaires :

```
# Installation d'Uvicorn, un serveur ASGI pour FastAPI
pip install uvicorn

# Installation de FastAPI, un framework web pour construire des APIs
pip install fastapi

# Installation de Jinja2, un moteur de templates pour le rendu HTML
pip install jinja2

# Installation de Motor, un driver asynchrone pour MongoDB
pip install motor

# Installation de python-dotenv, pour gérer les variables d'environnement
pip install python-dotenv
```

- **Gestion des Variables d'Environnement**

Pour charger les variables d'environnement comme l'adresse IP et le port à partir d'un fichier (**.env**), nous utilisons python-dotenv.

Voici un exemple de fichier **(.env)** :

```
#Adresse IP et PORT pour exécuter application
APP_IP=127.0.0.1
APP_PORT=8000
```

Dans notre code Python, nous pouvons charger ces variables et utiliser des valeurs à partir d'un fichier **(config.ini)** de la manière suivante :

```
import configparser
from dotenv import load_dotenv
import os

load_dotenv()
config = configparser.ConfigParser()
config.read('Config/config.ini')

APP_IP = str(os.getenv("APP_IP")) if os.getenv("APP_IP") else config.get('APP',
'APP_IP', fallback='127.0.0.1')
APP_PORT = int(os.getenv("APP_PORT")) if os.getenv("APP_PORT") else
config.getint('APP', 'APP_PORT', fallback=8200)

DB_FILE_PATH_barrier = "./Database/DB/barriers.db"
DB_FILE_PATH_events = "./Database/DB/events.db"
```

- **Mise à Jour des Fichiers de Configuration JavaScript**

Pour garantir que nos fichiers JavaScript utilisent les bonnes valeurs d'adresse IP et de port, nous pouvons utiliser la fonction suivante définie dans le fichier **(config.py)** pour remplacer les lignes appropriées dans le fichier **(config.js)** :

```
def replace_line_with_substring(file_path, substring, new_line):
    with open(file_path, 'r') as file:
        lines = file.readlines()

    for i, line in enumerate(lines):
        if substring in line:
            lines[i] = new_line + '\n'
            break

    with open(file_path, 'w') as file:
        file.writelines(lines)
    print(f"{substring} found and replaced by {new_line}")

replace_line_with_substring('./static/config.js', "var ipAddress", f'var
ipAddress="{APP_IP}"')
replace_line_with_substring('./static/config.js', "var portep", f'var
portep="{APP_PORT}"')
```

Cette fonction remplace les lignes contenant les sous-chaînes spécifiées dans le fichier **config.js** par de nouvelles lignes contenant les valeurs d'IP et de port chargées précédemment.

- **Dockerisation de l'Application**

Pour containeriser notre application, nous utilisons Docker [9]. Voici un fichier Dockerfile exemple :

Avant de préparer notre fichier docker il faut préparer un fichier essentiel pour lister les dépendances utilisées dans notre projet avec la commande :

```
pip freeze > requirement.txt
```

```
# Utilisation de l'image de base Python 3.12 slim  
FROM python:3.12-slim  
  
# Définition du répertoire de travail  
WORKDIR /app  
  
# Copie du contenu de l'application dans le répertoire de travail  
COPY . /app  
  
# Copie des dépendances dans le conteneur  
COPY ./requirements.txt /app/requirements.txt  
  
# Installation des dépendances  
RUN pip install --no-cache-dir -r /app/requirements.txt  
  
# Commande pour lancer l'application  
CMD ["python3", "/app/main.py"]
```

Pour gérer les conteneurs Docker, nous utilisons les commandes suivantes :

```
# Construction de l'image Docker  
docker build .  
  
# Tag de l'image Docker  
docker tag nom_image yassinemanai/mp_back :2.0  
  
# charger l'image vers un registre de conteneurs en ligne (Docker HUB)  
docker push yassinemanai/mp_back :2.0  
  
#Telecharger une image  
docker pull yassinemanai/mp_back :2.0
```


- **Exécution de l'Image Docker**

Pour exécuter notre application dans un conteneur Docker, nous utilisons la commande docker run. Voici comment cette commande fonctionne

```
docker run -d -p 8000:8000 -v /path/to/local/app/database:/etc/database  
--env APP_IP=192.168.25.30 --env APP_PORT=8200 yassinemanai/mp_back:2.0
```

- **Explication de la commande :**

-d : Exécute le conteneur en arrière-plan (détaché).
-p 8000 :8000 : Mappe le port 8000 du conteneur au port 8000 de l'hôte.
-v /path/to/local/app/database:/etc/database : Monte le répertoire local /path/to/local/app/database dans le conteneur à l'emplacement /etc/database. Cela permet au conteneur d'accéder aux fichiers de base de données locaux.
--env APP_IP=192.168.25.30 : Définit la variable d'environnement APP_IP à 192.168.25.30 dans le conteneur extrait de fichier (.env).
--env APP_PORT=8200 : Définit la variable d'environnement APP_PORT à 8200 dans le conteneur.
yassinemanai/mp_back:2.0 : Spécifie l'image Docker à utiliser et son tag

Ces commandes et configurations permettent de créer un environnement de développement structuré et de déployer facilement l'application dans des conteneurs pour une distribution plus simple et une meilleure gestion des dépendances.

7. Sprint Review

Au cours de ce sprint, notre équipe a réalisé des avancées significatives sur la plateforme de gestion de parking. Nous avons revu et amélioré l'interface utilisateur pour une navigation plus intuitive, facilitant ainsi la gestion des utilisateurs et les invités. De plus, une nouvelle fonctionnalité de commande manuelle des barrières a été introduite, permettant à l'administrateur de les contrôler en cas d'urgence (Incendie, Police, Ambulance) avec une traçabilité pour une sécurité optimale.

8. Perspective

En plus de nos efforts pour améliorer l'expérience utilisateur et les performances du système, nous avons spécifiquement investi dans l'intégration de la gestion des places en temps réel. Cette fonctionnalité renforce notre engagement à fournir une plateforme de gestion de parking complète et fiable. Nous sommes confiants que ces développements nous placent sur la voie pour devenir le choix préféré des utilisateurs à la recherche d'une solution de stationnement efficace et pratique.

9. Conclusion :

Dans ce Sprint nous avons réussi à fournir une interface Web complète destinée à l'administrateur. On a commencé par l'élaboration du Backlog du premier sprint, on a présenté par la suite la partie analyse, la partie conception, les technologies utilisées, la description des interfaces, les parties complexes du code et on a terminé par sprint review et une perspective.

CHAPITRE 4

SPRINT 2 : MACROPARK

MOBILE APPLICATION

1. Introduction

Ce chapitre est destiné à décrire les différentes étapes de la réalisation du deuxième sprint. Tout d'abord nous présentons le Backlog du ce sprint, ensuite nous présentons l'analyse et la conception du ce sprint-là, nous allons aussi présenter les différentes interfaces IHM et les parties complexes du code.

2. Backlog sprint 2

Selon la planification de notre méthodologie, le deuxième sprint est principalement sur le cas « **Développement d'une application mobile pour les utilisateurs** » que nous allons repartir par la suite.

Tableau 16. Réparation du Sprint2

Identifiant	Acteur	Tâche	Estimation
Se Connecter	Utilisateurs	T1 : Analyse T2 : Conception T3 : Backend T4 : Frontend	T1 : 01 H T2 : 01 H T3 : 01 H T4 : 03 H
S'inscrire			T1 : 03 H T2 : 02 H T3 : 10 H T4 : 10 H
Page d'accueil / Commande de la barrière			T1 : 03 H T2 : 02 H T3 : 15 H T4 : 20 H
Consulter Profile et mise à jour des données			T1 : 03 H T2 : 02 H T3 : 10 H T4 : 12 H
Consulter Historique			T1 : 03 H T2 : 02 H T3 : 05 H T4 : 07 H
Se déconnecter			T1 : 01 H T2 : 02 H T3 : 01 H T4 : 01H

3 Modélisation UML

3.1 Diagramme de cas d'utilisation du sprint 2

Un diagramme de cas d'utilisation représente les interactions entre les utilisateurs et un système, décomposant les fonctionnalités en cas d'utilisation logiques pour une compréhension facile des besoins des utilisateurs.

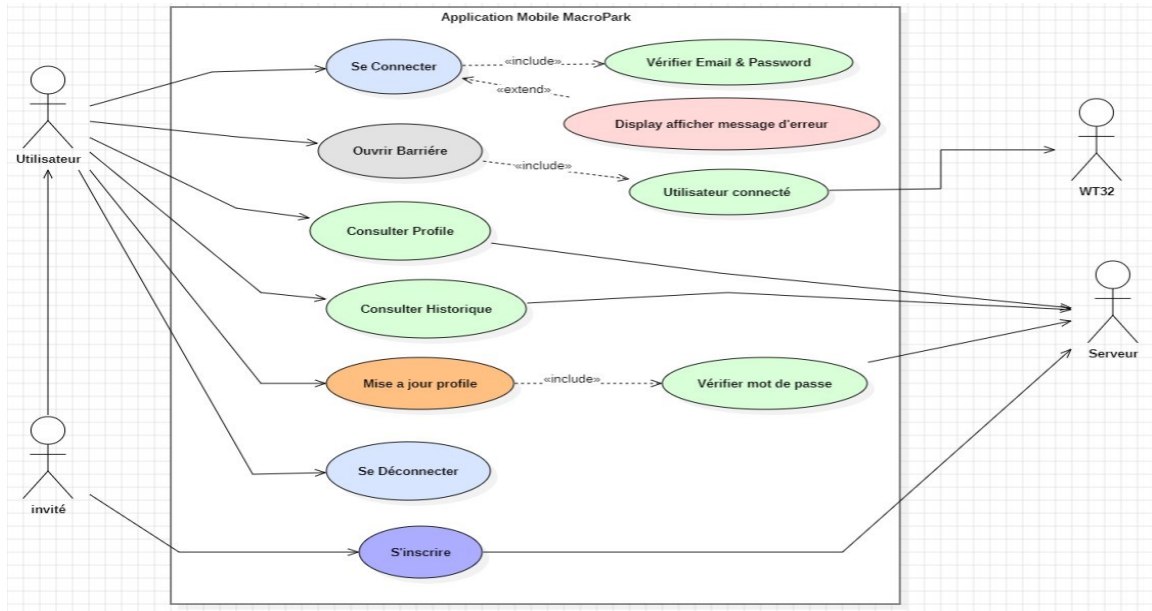


Figure50. Diagramme de cas d'utilisation du sprint 2

3.2 Diagramme de Class du sprint 2

Cette phase est consacrée pour la réalisation de notre diagramme de classes qui décrit la structure des données de notre interface web et l'analyse des informations :

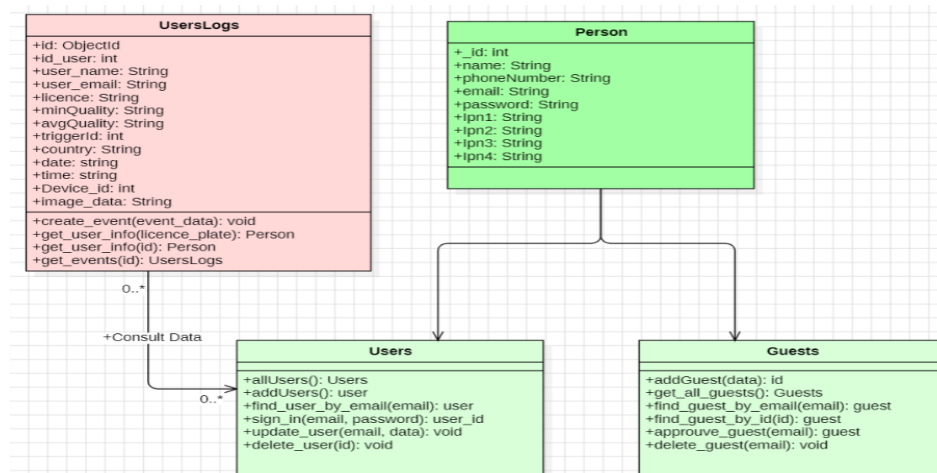


Figure 51. Diagramme de Class du sprint 2

3.3 Diagramme de Séquence du sprint 2

Le diagramme de séquence qui décrit l'interaction entre notre application mobile et l'utilisateur est présenté ci-dessous :

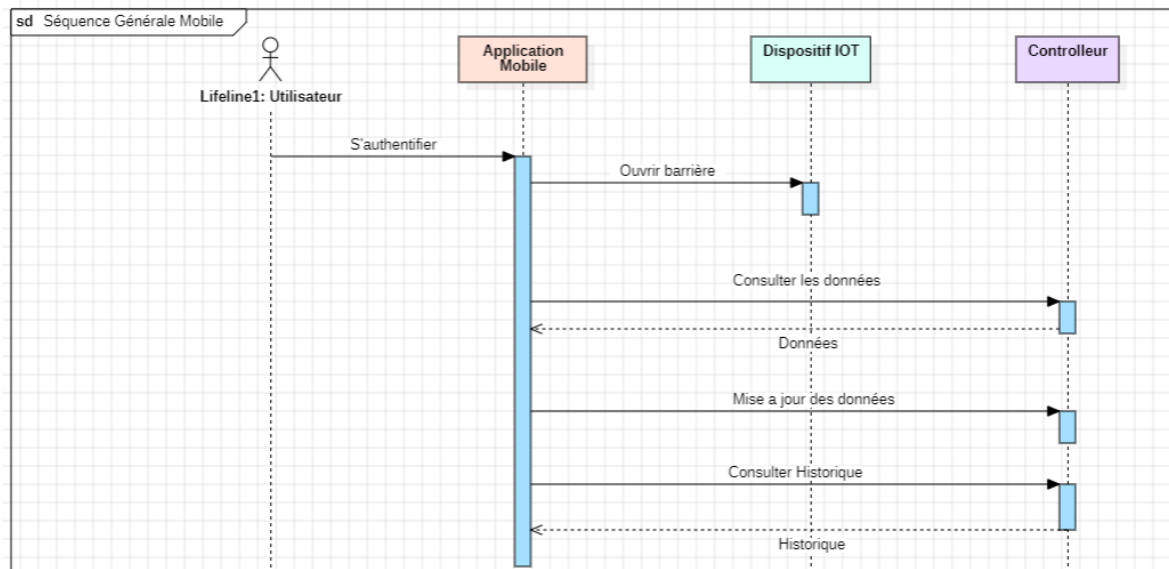


Figure 52. Diagramme de Séquence du sprint 2

4 Implémentation et technologie utilisé

Nous commençons par configurer notre environnement de développement, nous choisissons donc de travailler avec ces outils afin de développer une interface web bien fonctionnelle et bien structurée.

Voici un aperçu de chaque élément utilisé :

4.1 Frontend

- ✓ **Flutter** : Flutter est un Framework open-source de Google pour créer des applications multiplateformes avec un seul code source.

4.2 Backend

- ✓ **Python** : Langage de programmation utilisé pour écrire la logique côté serveur.
- ✓ **Docker** : Outil de virtualisation pour créer des conteneurs légers et portables pour les applications afin de l'exécuter dans différentes machines [9]
- ✓ **Uvicorn** : Serveur ASGI rapide et léger pour exécuter les applications Python.

- ✓ **Environnement Virtuel** : Un espace de développement isolé où nous installons les logiciels et paquets Python nécessaires, isolant ainsi ces paquets du reste de l'environnement global de notre machine.

4.3 Liaison entre Frontend et Backend

- ✓ **FastAPI** : Framework web moderne et rapide pour construire des API avec Python.
- ✓ **Swagger UI** : Interface utilisateur interactive pour tester et documenter les API créées avec FastAPI [7]

4.4 Liaison entre Application mobile et dispositif IOT (contrôleur de la barrière)

- ✓ **iBeacon** : Utilisé pour la communication sans fil à courte distance, iBeacon permet à l'application mobile de détecter la proximité du dispositif IoT et d'interagir avec lui, déclenchant des actions en fonction de la localisation.
- ✓ **BLE** : Ce protocole permet une connexion économe en énergie entre l'application mobile et le dispositif IoT, facilitant le contrôle de la barrière via des commandes sans fil tout en prolongeant la durée de vie de la batterie des appareils mobiles.

4.5 Base de Données

- **MongoDB** : MongoDB est une base de données NoSQL orientée document, flexible et évolutive, idéale pour les données non structurées et les données de taille étendue.

Cette architecture semble bien équilibrée, utilisant des technologies modernes pour développer une application mobile fonctionnelle et bien structurée d'un côté, et un design confortable pour les utilisateurs de l'autre côté.

5 Description détaillée du sprint 2

À ce stade, nous avons détaillé le fonctionnement de notre système avec une description plus approfondie.

5.1 Description ‘Se connecter’

L'utilisateur peut accéder à l'application mobile à l'aide de son compte fournis par l'administrateur. Pour ce faire, il doit saisir son adresse email et son mot de passe.

- Diagramme de cas d'utilisation :

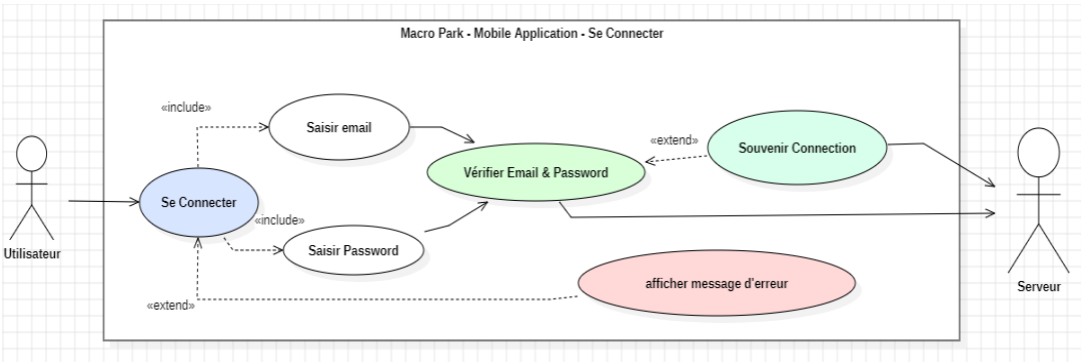


Figure53. Diagramme de cas d'utilisation de « se connecter »

- Description textuelle :

Tableau 17. Description textuelle / se connecter / Sprint 2

Cas	Se Connecter à l'application mobile
Acteurs	Utilisateur
Précondition	Permet à l'utilisateur à se connecter à son compte.
Scénario nominal	- L'utilisateur accède à l'application - Le système fait appel à la page de connexion - L'admin saisit son email et son mot de passe - Le système vérifie les informations saisies - Le système fait appel à la page d'accueil
Scénario alternatif	Si les champs sont erronés ou vides : - Le système les indique dans le formulaire Si l'email ou le mot de passe est invalide : - Le système affiche un message d'erreur Si un invité n'est pas accepté : - Le système affiche un message s'erreur

5.2 Description 'S'inscrire'

- Diagramme de cas d'utilisation :

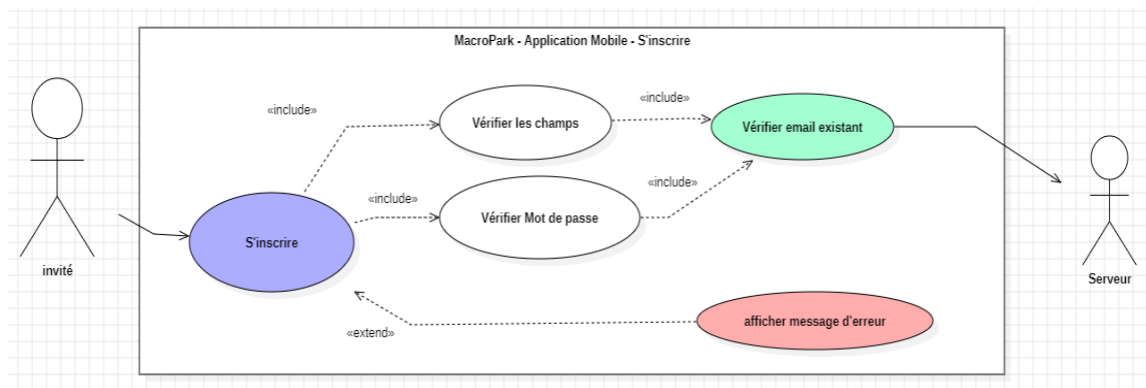


Figure54. Diagramme de cas d'utilisation de « s'inscrire »

- Description textuelle :

Tableau 18. Description textuelle / s'inscrire / Sprint 2

Cas	S'inscrire à l'application mobile
Acteurs	Invité
Précondition	Permet à l'invité s'inscrire à l'application mobile
Scénario nominal	- L'invité accède pour s'inscrire à l'application - Le système fait appel à l'interface d'inscription - L'invité remplit les champs nécessaires - L'invité attend l'acceptation par l'admin pour se connecter - L'invité se connecte si son compte est accepté. - L'admin modifier les informations d'un utilisateur - L'admin supprimer un utilisateur
Scénario alternatif	Si les champs sont erronés ou vides : - Le système les indique dans le formulaire Si l'email est déjà inscrit : - Le système affiche un message d'erreur Si la longueur du mot de passe est inférieure à 8 : - Le système affiche un message d'erreur Si le champ du mot de passe et le champ du confirmer mot de passe n'est pas équivalent : - Le système indique que les deux champs ne sont pas valides.

5.3 Description 'Page d'accueil'

- Diagramme de cas d'utilisation :

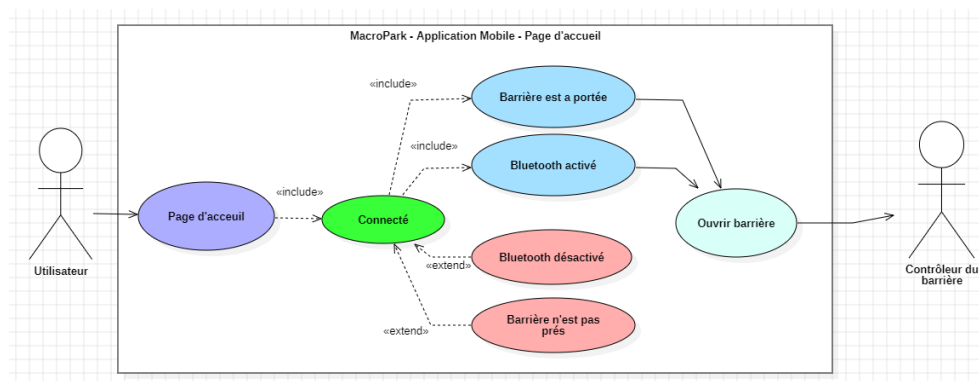


Figure55. Diagramme de cas d'utilisation de « Page d'accueil »

- Description textuelle :

Tableau 19. Description textuelle / page d'accueil / sprint 2

Cas	Page d'accueil
Acteurs	Utilisateur
Précondition	Permet à l'utilisateur à ouvrir la barrière
Scénario nominal	- L'admin accède à la page d'accueil - Le système fait appel à l'interface demandé - L'utilisateur peut ouvrir la barrière avec un simple click - Une fenêtre d'envoi d'action s'affiche
Scénario alternatif	Si le Bluetooth est désactivé la barrière n'ouvre pas. Si la barrière n'est pas proche il n'ouvre pas

5.4 Description 'Consulter Profile et Mise à jour des données'

- Diagramme de cas d'utilisation :

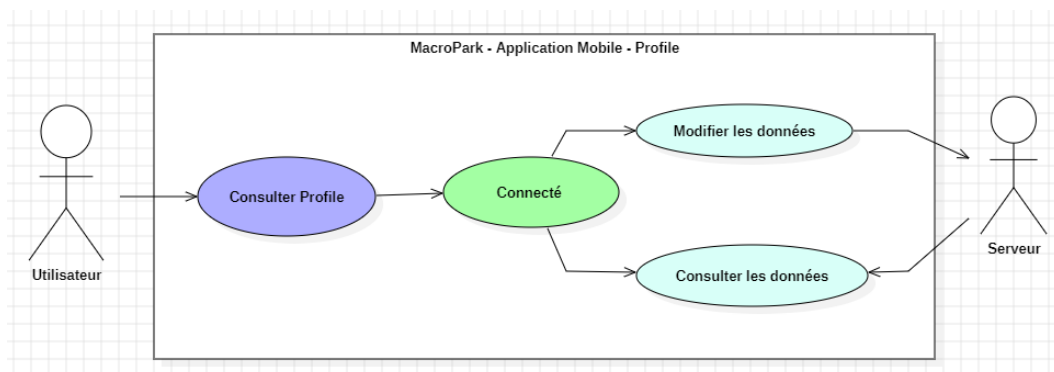


Figure56. Diagramme de cas d'utilisation de « Consulter Profile et Mise à jour des données »

- Description textuelle :

Tableau 20. Description textuelle / profile / Sprint 2

Cas	Consulter le profile et mise à jour des données
Acteurs	Utilisateur
Précondition	Permet l'utilisateur à consulter don profile
Scénario nominal	• L'admin accède à la page du profile • Le système fait appel à l'interface adéquate. • Le système fait appel à les données de la base. • L'utilisateur effectue ses changements aux données. • L'utilisateur consulte ses données.

5.5 Description ' Consulter Historique'

- Diagramme de cas d'utilisation :

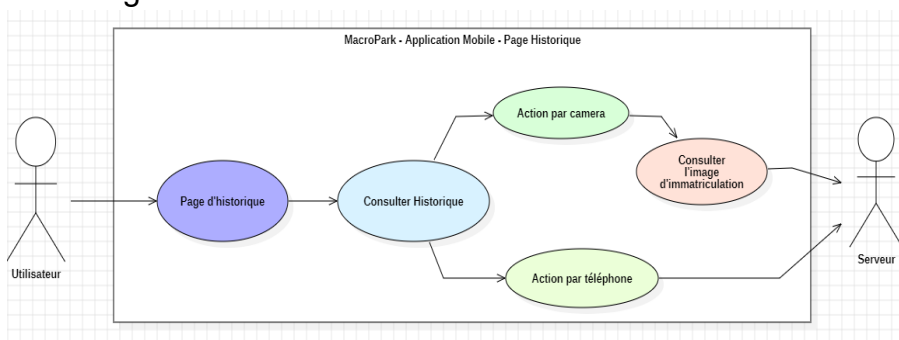


Figure57. Diagramme de cas d'utilisation de « Consulter Historique »

- Description textuelle :

Tableau 21. Description textuelle / Historique / sprint 2

Cas	Consulter Historique
Acteurs	Utilisateur
Précondition	Permet l'utilisateur à consulter son historique du parking
Scénario nominal	• L'utilisateur accède à la page d'historique • Le système fait appel à l'interface • Le système fait appel à la liste des historiques dans la base de données. • L'utilisateur consulte son historique • Si l'historique contient une image l'utilisateur consulte l'image capturé par la caméra lors d'entrer au parking. • Si l'historique contient une action par téléphone il s'affiche un message <u>('Opened with Phone')</u>.
Scénario alternatif	Si la liste d'historique est vide un message s'affiche.

5.6 Description ‘ Se déconnecter’

- Diagramme de cas d'utilisation :

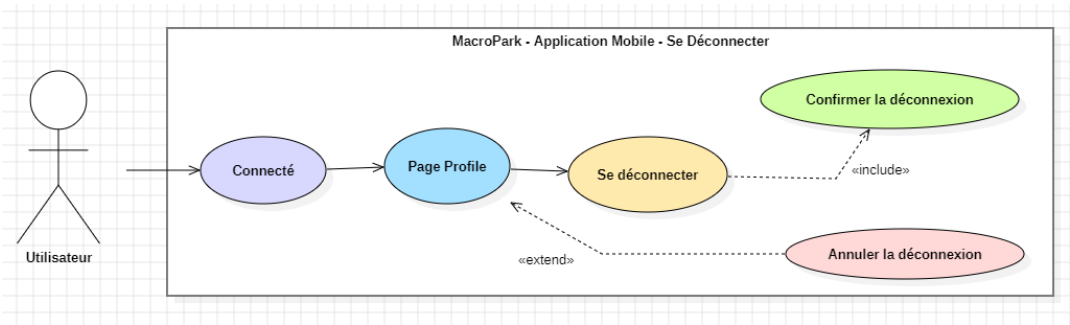


Figure58. Diagramme de cas d'utilisation de « Se déconnecter»

- Description textuelle :

Tableau 22. Description textuelle / Se déconnecter / Sprint 2

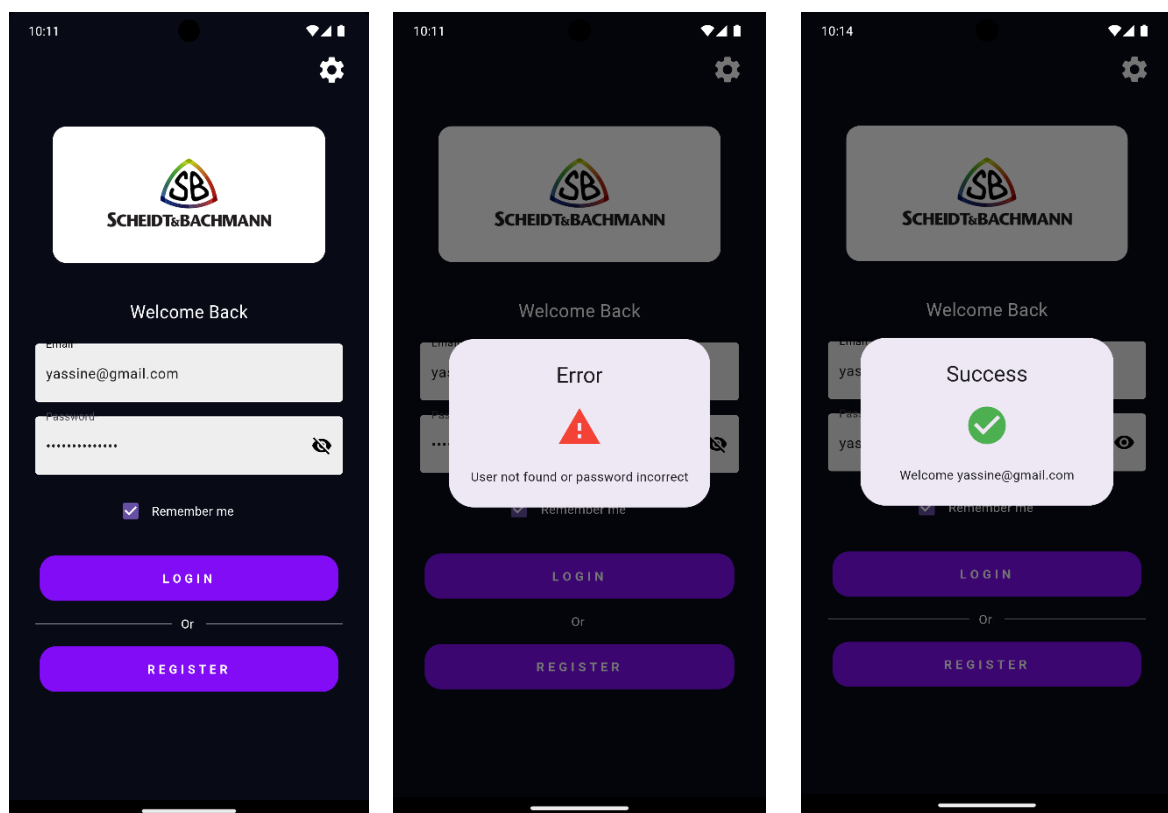
Cas	Se déconnecter
Acteurs	Utilisateur
Précondition	Permet à l'utilisateur se déconnecter de l'application
Scénario nominal	• L'utilisateur accédé page profile • L'utilisateur demande à se déconnecter. • L'utilisateur se confirme à se déconnecter
Scénario alternatif	Si l'utilisateur ne confirme pas il se retour à son page de profile

6. Réalisation

6.1 Interfaces graphiques

- Page de connexion

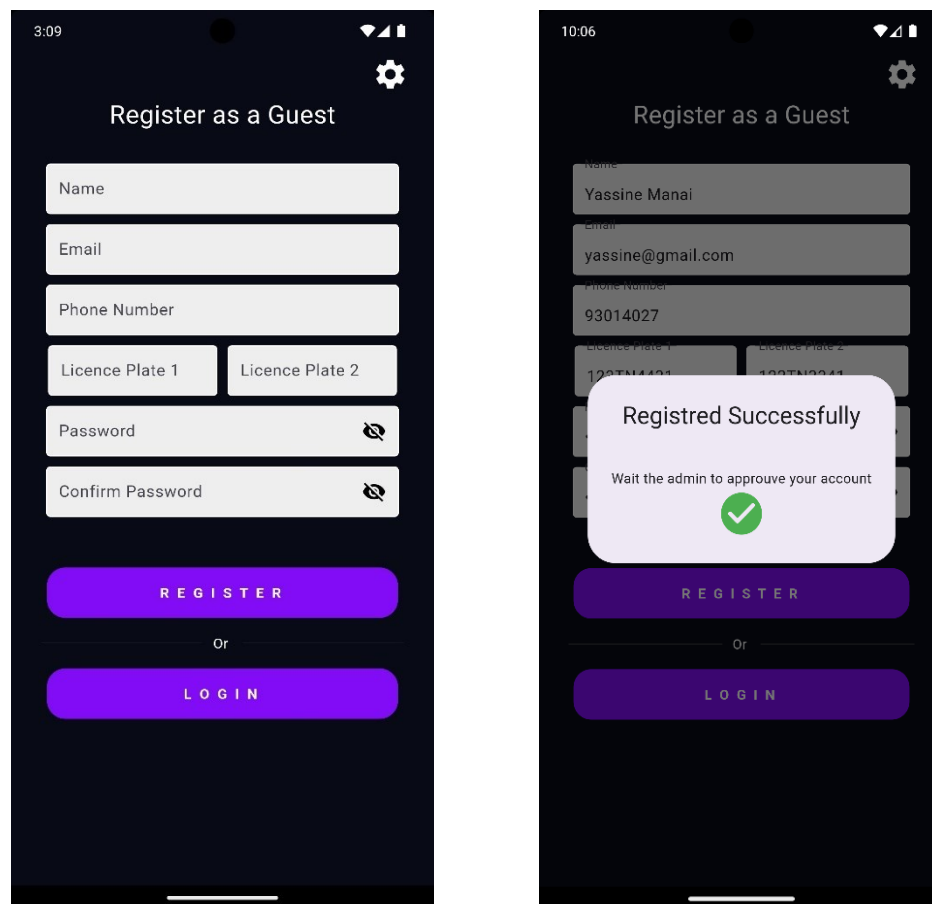
La page de connexion pour l'utilisateur offre une interface simple et intuitive pour accéder à l'espace de l'utilisateur, elle se compose de deux champs (2) essentiels 'Email' & 'Password' qui permettent de se connecter en toute sécurité et un bouton 'Remember Me' pour enregistrer les données afin de se connecter automatiquement à chaque fois l'utilisateur s'accède à l'application.



Figures 59. Page de connexion

- **Page d'inscription**

La page d'inscription est conçue pour les invités externes de la société, avec un design simple pour utiliser notre application. Elle se compose de six champs essentiels **'Full Name'** & **'Email'** & **'Phone Number'** & **'Licence Plate Number 1'** & **'Password'** & **'Confirm Password'** ainsi q'un champ optionnel **'Licence Plate Number 2'**. Un bouton **'REGISTER'** permet d'enregistrer les données pour permettre à l'invité de s'inscrire. Une fenêtre s'affiche si toutes les données sont valides et si l'email n'existe pas dans la base de données.



Figures 60. Page d'inscription

- **Page d'accueil**

La page d'accueil est prioritairement développée pour l'ouverture de la barrière à distance à travers Bluetooth en utilisant la technologie BLE (Bluetooth Low Energy / Beacon) et la trame envoyée, porteuse d'un UUID (validé par une expression régulière), est répartie comme suit :

UUID	'F826'+id+'0-0122-3344-0509-000000000000'												
Validation du trame	Regular Expression												
	r'^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{12}\$'												
Répartition du trame	Identifiant du scan	ID organisme	ID User	ID User	Sortie Relais 1	Sortie Relais 2	Sortie Relais 3	Sortie Relais 4	Temps Relais 1	Temps Relais 2	Temps Relais 3	Temps Relais 4	Non définis
	F8	26	Id		01	22	33	44	05 s	09 s	0 s	0 s	000000000000

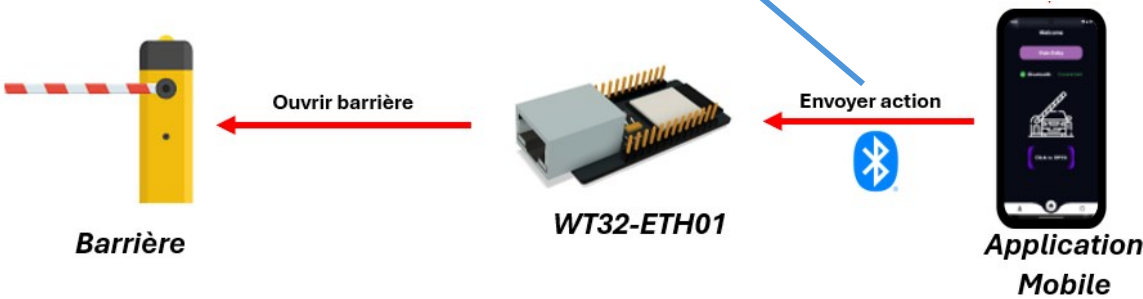
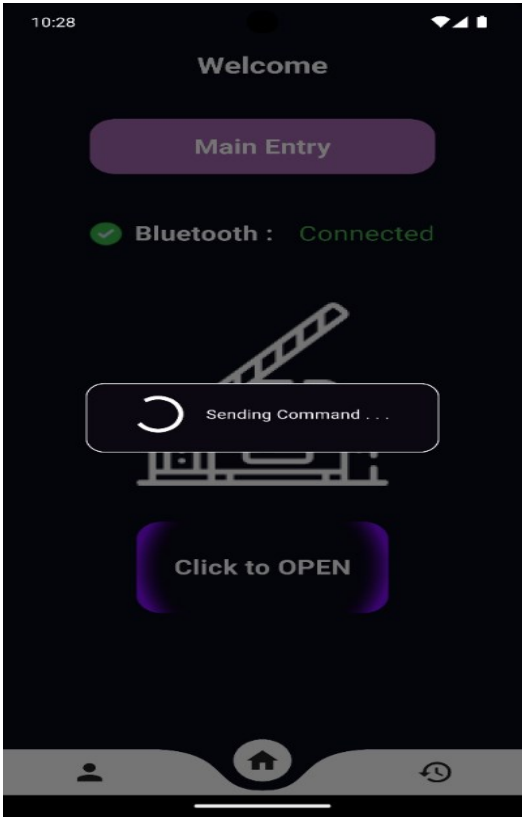
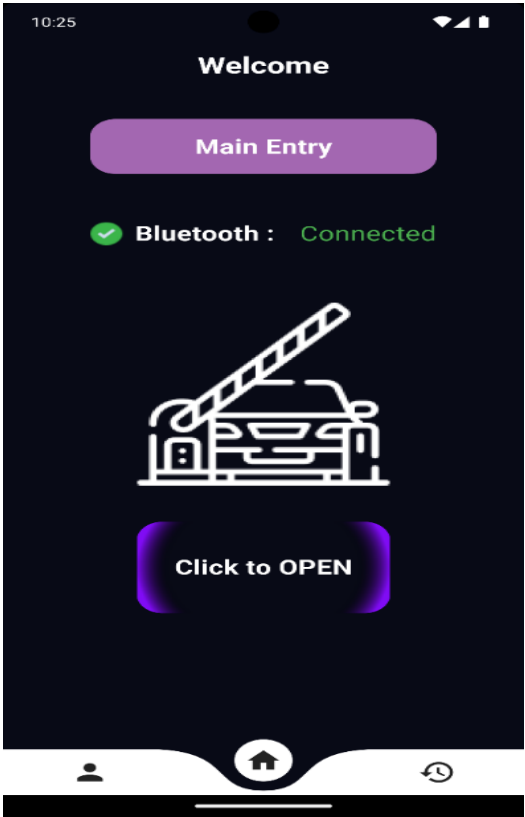
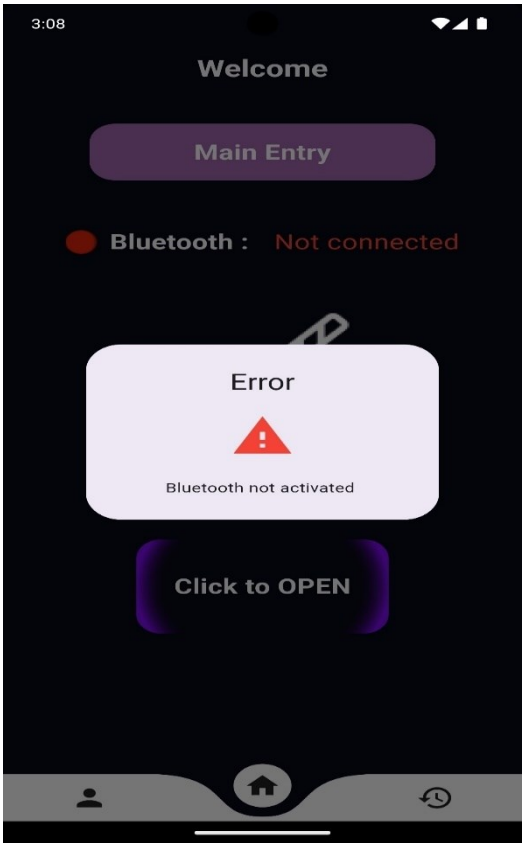
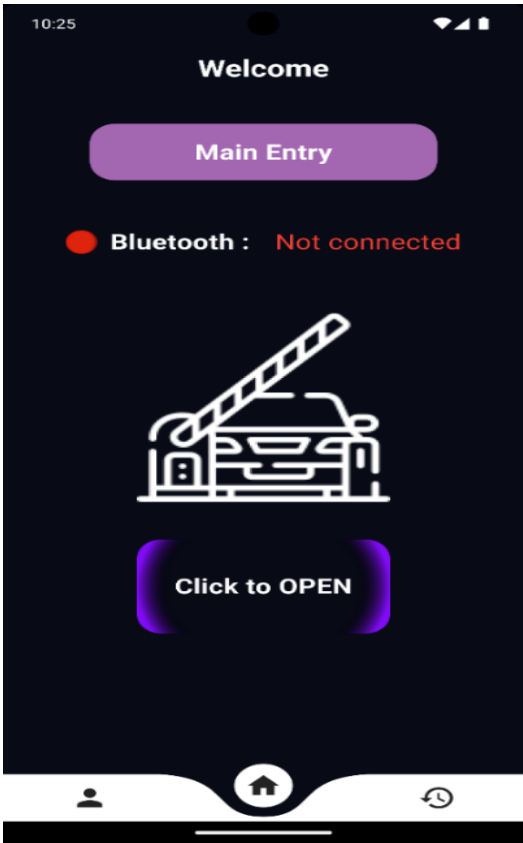


Figure 61. UUID

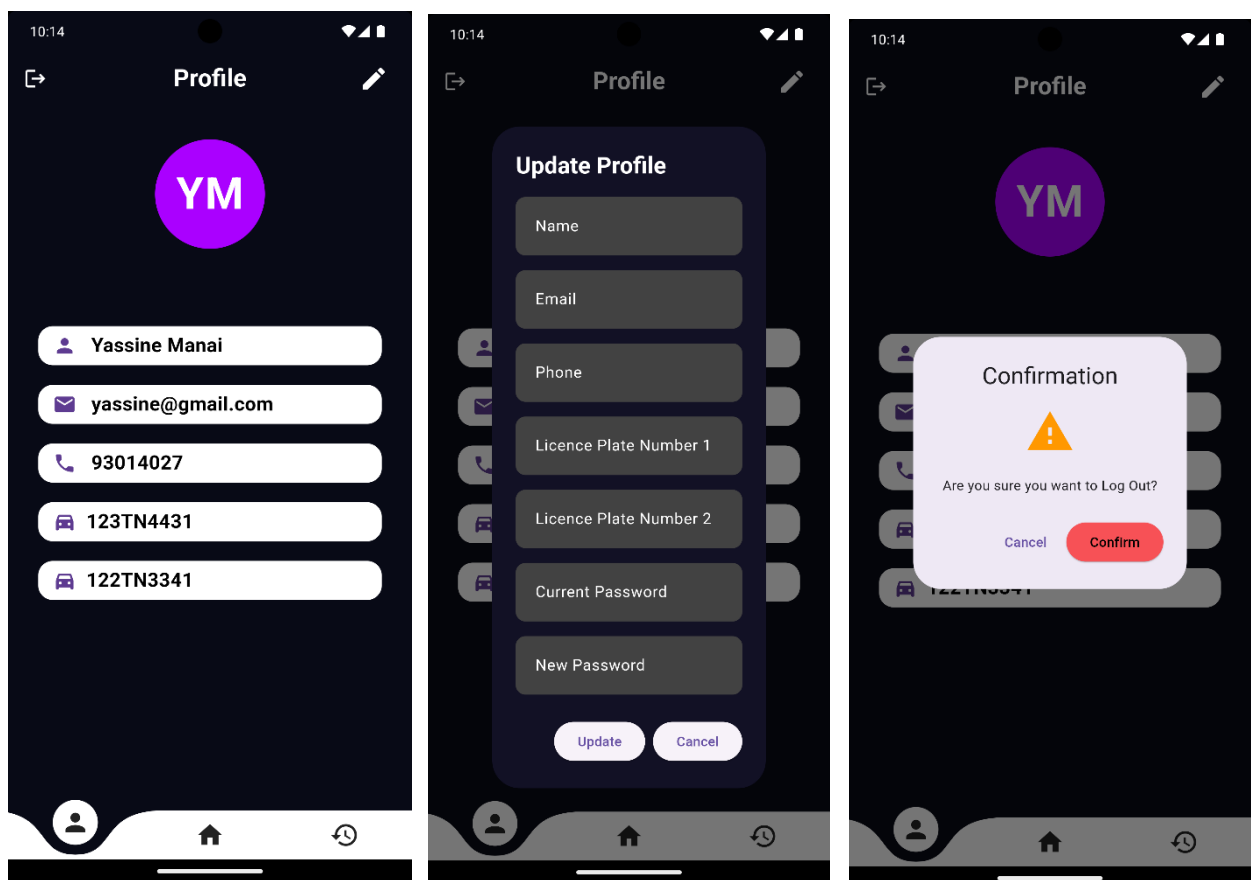


Figures62. Page d'accueil

• Page de Profile

La Page Profile est conçue essentiellement pour les informations d'utilisateur tels que le Nom & Prénom, l'email, le numéro du téléphone et les immatriculations liées avec cet utilisateur.

Dans ce page aussi on a un bouton modifier qui affiche une fenêtre des champs pour modifier les données d'une manière sécurisée et aussi un bouton 'Se déconnecter'



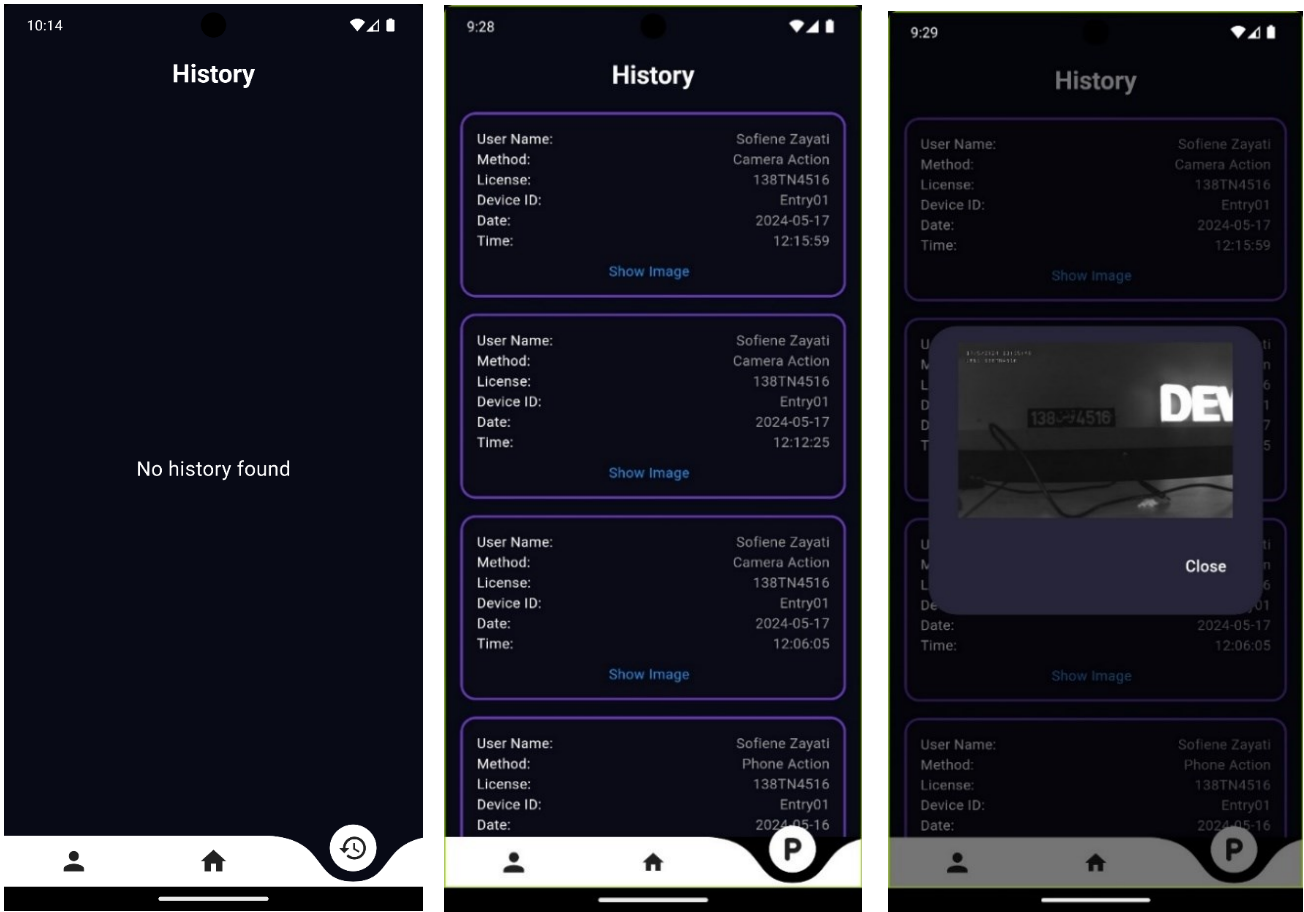
Figures 63. Page de Profile

• Page d'historique

Cette page est principalement élaborée pour afficher l'historique d'entrée au parking de chaque utilisateur.

Si l'entrée est exécutée par l'immatriculation donc il s'affiche un bouton pour afficher l'image capturée et toutes les informations telles que l'heure, date, méthode, identifiant et immatriculation.

Si l'entrée est exécutée par le téléphone donc il s'affiche un texte « **Opened with phone** » et toutes les informations telles que l'heure, date, méthode, identifiant.



Figures 64. Page historique

6.1 Logique et Codes

Pour ce projet, nous utilisons Flutter et diverses bibliothèques telles que `shared_preferences`, `beacon_broadcast`, et `permission_handler`.

Voici les étapes et les commandes nécessaires pour créer et exporter une application Flutter, ainsi que quelques extraits de code pertinents.

- **Création d'une Application Flutter**

Pour créer une nouvelle application Flutter, utilisez les commandes suivantes :

```
// Création d'un nouveau projet Flutter
flutter create macropark

// Accédez au répertoire du projet
cd macropark
```

- **Exportation d'une Application Flutter en APK :**

Pour exporter votre application Flutter en APK, suivez ces étapes :

```
// Assurez-vous d'être dans le répertoire de votre projet Flutter
cd macropark

//Compilez l'application pour Android
flutter build apk

// Le fichier APK se trouvera dans le répertoire ./build/app/outputs/flutter-apk/
```

- **Exemple de Code**

Voici un exemple de code montrant comment utiliser `beacon_broadcast` pour diffuser un signal BLE avec un UUID généré et vérifier sa validité :

```
Future<void> _broadcastBeacon(String uuid) async
{
  String uuid = 'F826'+'$id'+'0-0122-3344-0509-9C398FC199D2'.
  bool isValid = isValidUUID(uuid);
  if(isValid){
    try {
      _showLoadingDialog("Sending Command . . .");

      beaconBroadcast .setUUID(uuid)
        .setMajorId(1) .setMinorId(100)
        .start();
    }
  }
}
```

```

        Navigator.of(context).pop();
        await Future.delayed(Duration(seconds: 1));
        beaconBroadcast .setUUID(uuid).stop();
    }
    catch (e)
    {
        print('Error: $e');
    }
}

```

- **UUID Généré** : Le UUID est généré et imprimé.

Par exemple, uuid = 'F826123456780122334405099C398FC199D2'.

- **Validation de l'UUID** : La fonction isValidUUID vérifie si l'UUID est valide.

```

bool isValidUUID(String uuid)
{
    if (uuid == null) return false;
    final pattern = RegExp(
        r'^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$',
        caseSensitive: false,
    );
    return pattern.hasMatch(uuid);
}

```

Cette fonction utilise une expression régulière (RegExp) pour vérifier que le format de l'UUID correspond au standard : huit caractères hexadécimaux suivis de trois groupes de quatre caractères hexadécimaux, puis un groupe de douze caractères hexadécimaux cette expression est fixée par défaut auprès de la bibliothèque du **beacon_broadcast**.

- **Diffusion du Signal BLE :**

La méthode `_broadcastBeacon` utilise la bibliothèque **beacon_broadcast** pour diffuser un signal BLE avec l'UUID spécifié.

Elle affiche un dialogue de chargement pendant l'envoi de la commande.

La diffusion commence avec :

beaconBroadcast.setUUID(uuid).setMajorId(1).setMinorId(100).start().

Après un délai de 1 seconde, la diffusion s'arrête avec : **beaconBroadcast.stop().**

7. Sprint Review

Ce sprint a été axé sur le développement de notre application mobile avec Flutter et BLE. Nous avons réussi à intégrer efficacement Flutter pour une interface utilisateur réactive et attrayante. De plus, nous avons mis en place les fonctionnalités BLE, ouvrant ainsi de nouvelles possibilités pour des interactions sans fil. Notre équipe a également travaillé sur l'aspect visuel de l'interface utilisateur, en se concentrant sur son esthétique et son ergonomie. En résumé, ce sprint a été un succès dans la construction d'une application mobile moderne et fonctionnelle.

8. Perspective Sprint 2

- **Version iOS** : Nous envisageons de développer une version iOS de notre application pour élargir notre base d'utilisateurs.
- **Réservation en Ligne** : Nous travaillons sur un système de réservation en ligne pour offrir plus de commodité aux utilisateurs.
- **Surveillance en Temps Réel** : Nous nous efforçons d'intégrer un système de surveillance en temps réel dans le parking pour améliorer la sécurité.

9. Conclusion

Au cours de ce sprint, nous avons réussi à fournir une application mobile pour les utilisateurs. Nous avons commencé par élaborer le backlog ,ensuite, nous avons présenté l'analyse, la conception, les technologies utilisées, ainsi qu'une description des interfaces.

CHAPITRE 5

SPRINT 3 : MACROPARK

BARRIER CONTROLLER

1. Introduction

Durant notre stage, nous avons développé un système de contrôle pour gérer des barrières en utilisant le microcontrôleur WT32. Le système est conçu pour ouvrir et fermer les barrières en fonction des signaux Bluetooth Low Energy (BLE) ou de la reconnaissance de plaques d'immatriculation via MQTT (Message Queuing Telemetry Transport). Il inclut également un WiFiManager pour gérer les connexions WiFi, enregistrer les configurations et définir des adresses IP statiques. Ce rapport fournit une vue d'ensemble du projet, des détails de l'implémentation et des fonctionnalités du code.

2. Backlog Sprint 3

Selon la planification de notre méthodologie, le premier sprint est principalement sur les cas d'utilisation « **Codage et conception du contrôleur de la barrière** » que nous allons décortiquer par la suite.

Tableau 23. Backlog Sprint 3

Identifiant	Acteur	Tâche	Estimation
Identification du matériel	Barrière	T1 : Analyse T2 : Codage T3 : Test	T1 : 04 H T3 : 10 H
Découvrir le fonctionnement du système			T1 : 10 H T2 : 60 H T3 : 10 H
Gestion des configurations MQTT			T1 : 03 H T2 : 25 H T3 : 03 H
Gestion des configurations Wifi - Ethernet			T1 : 01 H T2 : 50 H T3 : 02 H
Interface Web			T1 : 01 H T2 : 30 H T3 : 10 H
Câblage			T1 : 05 H T3 : 03 H
Schéma et Housing			T1 : 05 H T2 : 20 H T3 : 02 H

3. Modélisation UML

- Diagramme de classe :

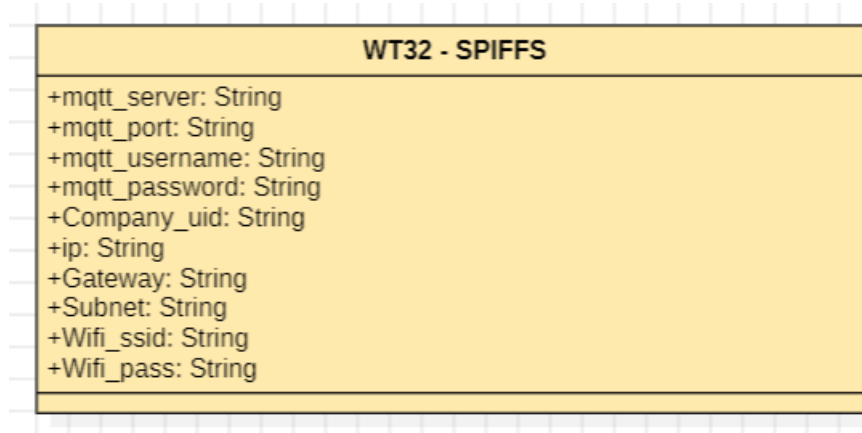


Figure 65. Diagramme de classe

- Séquence :

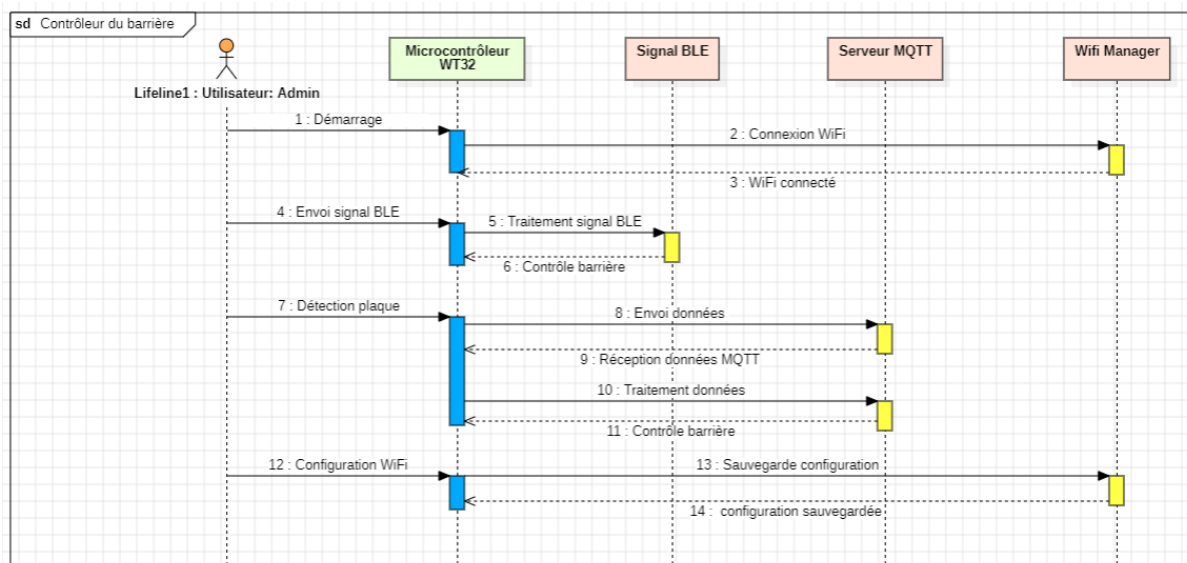


Figure 66. Diagramme de séquence

4. Description détaillée du sprint 3

Dans cette partie, nous avons précisé comment fonctionne notre système avec une description plus détaillée.

4.1 Description 'Identification du matérielle '

4.1.1 Qu'est-ce que le WT32 ?

Le WT32-ETH01 [2] est un module microcontrôleur basé sur l'ESP32, développé par la société "Wireless-Tag" (WT) [4]. Il s'agit essentiellement d'une petite carte de développement ESP32 bon marché avec Ethernet, Wi-Fi et des broches GPIO. À environ 15 \$ chez JacobsParts et 7 \$ environ sur AliExpress au moment de la rédaction, il offre une solution économique pour les projets nécessitant une connectivité réseau filaire.

4.1.2 Caractéristiques Clés du WT32

- **Processeur à Double Cœur** : Équipé d'un CPU Xtensa LX6 à double cœur 32 bits, offrant une puissance de traitement substantielle pour gérer des tâches complexes.
- **Communication Sans Fil** :
 - **Wi-Fi** : Supporte les normes IEEE 802.11 b/g/n/e/i, permettant un réseau sans fil fiable et à haute vitesse.
 - **Bluetooth** : Comprend à la fois le Bluetooth Classique et le Bluetooth Low Energy (BLE), facilitant divers besoins de communication sans fil.
- **Connectivité Ethernet** :
 - **Ethernet Intégré** : Fournit une connectivité réseau filaire, souvent plus stable et sécurisée par rapport aux connexions sans fil.
- **Mémoire et Stockage** :
 - **RAM** : Typiquement 520 KB de SRAM, permettant un traitement efficace des données.
 - **Mémoire Flash** : Supporte jusqu'à 16 MB de flash, offrant un espace de stockage ample pour le firmware et le code applicatif.
- **Interfaces Périphériques** :

- **Pins GPIO** : Plusieurs pins d'Entrée/Sortie générales pour l'interfaçage avec divers capteurs et actionneurs.
- **SPI, I2C, UART** : Interfaces de communication pour connecter des périphériques externes.
- **Gestion de l'Énergie** : Fonctionnalités avancées de gestion de l'énergie, y compris les modes de sommeil profond, cruciaux pour les applications IoT écoénergétiques.
- **Environnement de Développement Intégré (IDE)** : Supporte des environnements de développement comme l'Arduino IDE et l'ESP-IDF d'Espressif, simplifiant le processus de développement.

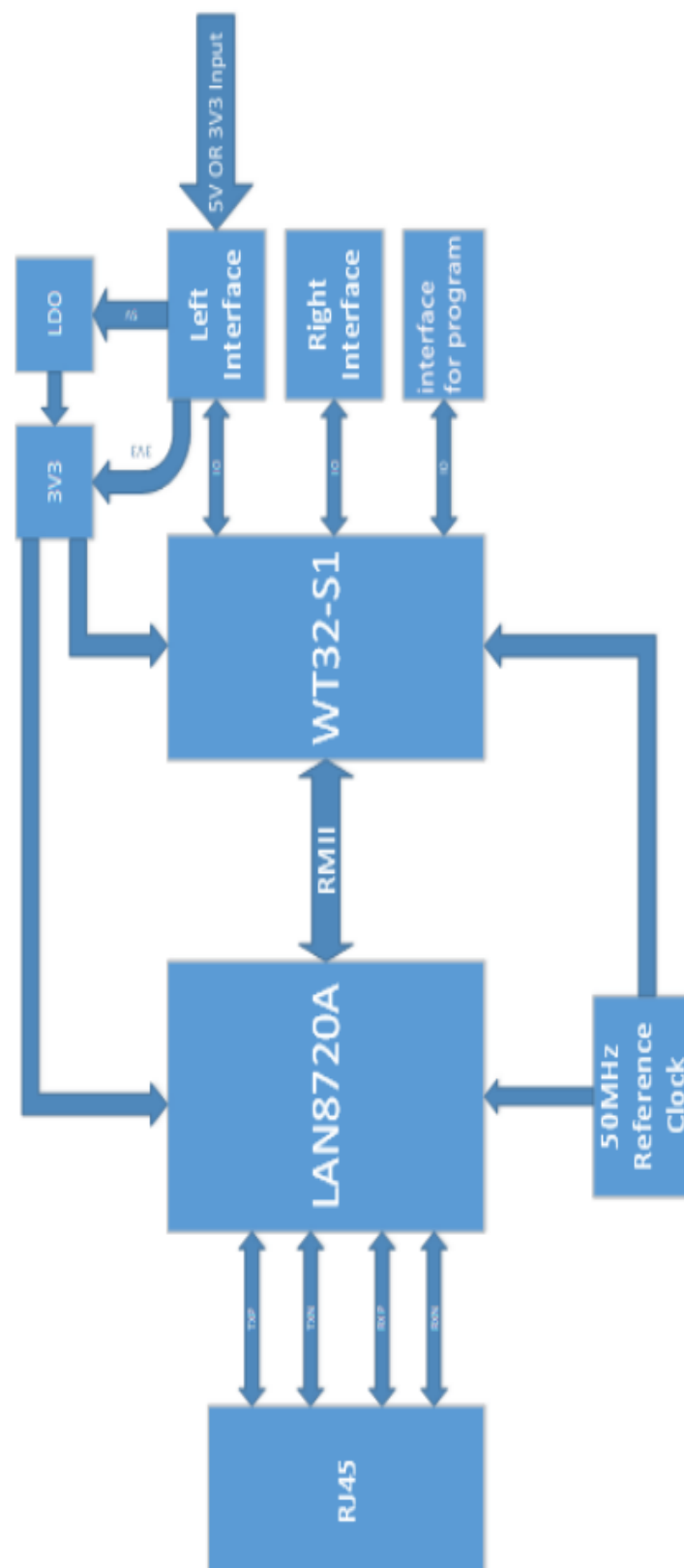


Figure 67. system block Diagram [1]

Tableau 24. Caractéristique WT32 – ETH01[1]

Catégories	Items	Spécifications du produit
Wi-Fi	RF certification	FCC/CE/RoHS
	Protocols	802.11 b/g/n/e/i (802.11n, up to 150 Mbps) A-MPDU and A-MSDU aggregation and 0.4 μ s guard interval support
	Frequency Range	2.4~2.5 GHz
Bluetooth	Protocols	Bluetooth v4.2 BR/EDR and BLE specification
	Radio	NZIF receiver with -97 dBm sensitivity
Hardware	Network port specification	RJ45, 10/100Mbps, cross straight adaptive
	Serial port baud rate	80~5000000
	On-board Flash Memory	32Mbit
	Operating voltage	5V or 3.3V power supply (choose one of them)
	Operating current	Average : 80mA
	Power supply current	Min : 500 mA
	Operating temperature range	-40°C~+85°C
	Ambient temperature range	Normal temperature
	Package	Half pad / connector through hole connection(optional)
Software	Wi-Fi mode	Station/softAP/SoftAP+station/P2P
	Wi-Fi Security	WPA/WPA2/WPA2-Enterprise/WPS
	Encryption	AES/RSA/ECC/SHA
	Firmware upgrade	Remote OTA upgrade over the network
	Software development	SDK for user secondary development
	Network protocols	IPv4, TCP/UDP
	IP acquisition method	Static IP, DHCP(Default)
	Simple transmission mode	TCP Server/TCP Client/UDP Server/UDP Client
	User configuration	AT instruction set

Pin	Name	Description
1	EN1	Reserved debug burning interface; Enable signal, active high
2	GND	Reserved debug burning interface; GND
3	3V3	Reserved debug burning interface; 3V3
4	TXD	Reserved debug burning interface; IO1, TXD0
5	RXD	Reserved debug burning interface; IO3, RXD0
6	IO0	Reserved debug burning interface; IO0

Pin	Name	Description
1	EN1	Enable signal, active high
2	CFG	IO32, CFG
3	485_EN	Enable pin of IO33, RS485
4	RXD	IO5, RXD2
5	TXD	IO17, TXD2
6	GND	GND
7	3V3	3V3 power supply
8	GND	GND
9	5V	5V power supply
10	LINK	Network connection indicator pin
11	GND	GND
12	IO39	IO39, only supports input
13	IO36	IO36, only supports input
14	IO15	IO15
15	IO14	IO14
16	IO12	IO12
17	IO35	IO35, only supports input
18	IO4	IO4
19	IO2	IO2
20	GND	GND

4.2 Pourquoi Utiliser le WT32 pour le Système de Contrôle de Barrière ?

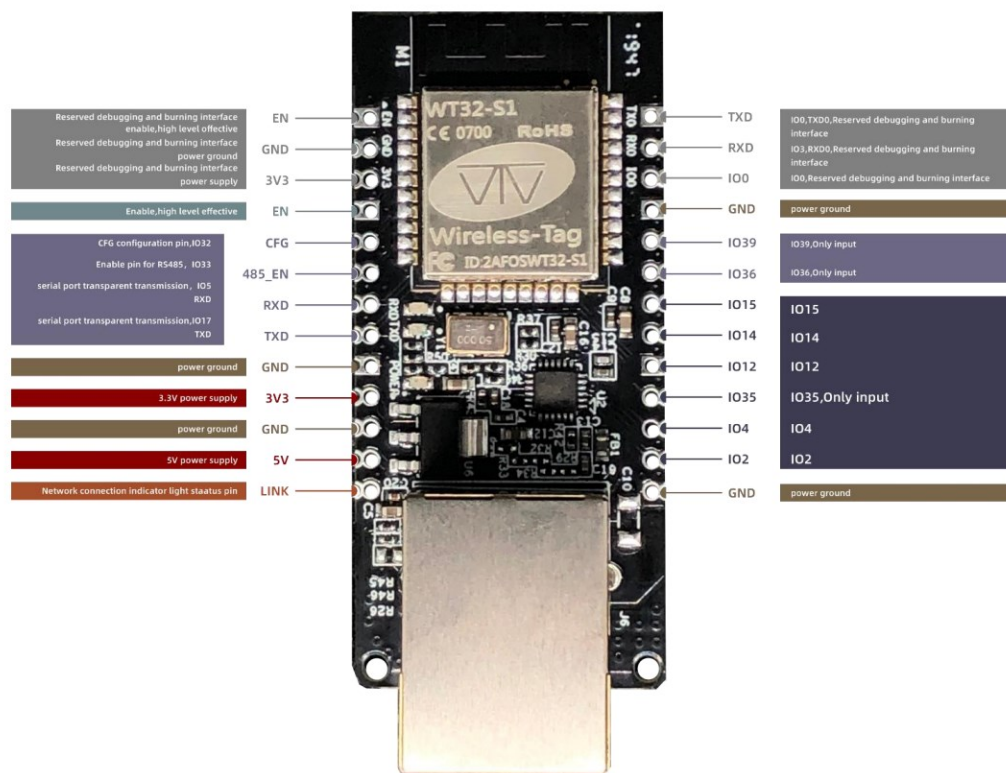


Figure68. WT32 [1]

4.2.1 Connectivité Complète

- **Intégration Wi-Fi, BLE et Ethernet** : Les capacités sans fil et Ethernet du WT32 permettent une intégration transparente avec les réseaux locaux et les appareils compatibles Bluetooth. Ceci est essentiel pour un système de contrôle de barrière qui repose sur des signaux BLE pour détecter les véhicules à proximité, le Wi-Fi pour la communication MQTT, et l'Ethernet pour une connectivité réseau stable et sécurisée.

- **Communication MQTT** : Les capacités Wi-Fi et Ethernet facilitent l'utilisation de MQTT, un protocole de messagerie léger idéal pour la communication en temps réel IoT, assurant une livraison rapide et fiable des messages entre le système de contrôle de barrière et le système de reconnaissance des plaques d'immatriculation.

4.2.2 Puissance de Traitement

- **CPU à Double Cœur** : Le processeur à double cœur gère efficacement les tâches concurrentes telles que la numérisation des signaux BLE, le traitement des messages MQTT et le contrôle des sorties de barrière sans goulots d'étranglement de performance.
- **Opérations en Temps Réel** : La puissance de traitement supporte les opérations en temps réel, cruciales pour des actions de contrôle de barrière opportunes basées sur les signaux et messages entrants.

4.2.3 Rentabilité et Intégration Simplifiée

- **Solution Tout-en-Un** : Le WT32 intègre des capacités Wi-Fi, Bluetooth et Ethernet sur un seul module, éliminant le besoin d'acheter et d'intégrer des modules Ethernet séparés. Cela réduit non seulement le coût global mais simplifie également le processus de développement et réduit les points de défaillance potentiels.
- **Choix Économique** : Comparé à l'achat d'un ESP32 et d'un module Ethernet supplémentaire séparément, le WT32 offre une solution plus économique, offrant toutes les fonctionnalités nécessaires dans un package compact et rentable.

4.2.4 Efficacité Énergétique

- **Faible Consommation d'Énergie** : Les fonctionnalités de gestion de l'énergie du WT32 sont bénéfiques pour maintenir une faible consommation d'énergie, particulièrement importante pour les installations où l'efficacité énergétique est une priorité.
- **Efficacité du BLE** : Le Bluetooth Low Energy est optimisé pour une faible consommation d'énergie, ce qui le rend adapté à une utilisation continue dans les systèmes de contrôle d'accès.

4.2.5 Flexibilité et Scalabilité

- **Interfaces Multiples** : La disponibilité de diverses interfaces de communication (SPI, I2C, UART) offre une flexibilité pour connecter des capteurs ou appareils supplémentaires, améliorant les capacités du système.
- **Stockage Extensible** : Une mémoire flash suffisante garantit que le firmware peut être facilement mis à jour et étendu avec de nouvelles fonctionnalités sans modifications matérielles.

4.2.6 Développement et Maintenance

- **Mises à Jour OTA** : Le support pour les mises à jour Over-the-Air (OTA) simplifie la maintenance du firmware, permettant des mises à jour à distance et réduisant le besoin d'accès physique aux appareils.
- **SPIFFS** : Le SPI Flash File System (SPIFFS) permet un stockage et une récupération faciles des fichiers de configuration, garantissant que le système peut être rapidement reconfiguré selon les besoins.

4.2.7 Utilisation de l'Adaptateur CP2102

Pour la programmation et la communication avec notre microcontrôleur WT32 ETH01, nous avons utilisé l'adaptateur CP2102. Cet adaptateur USB vers UART (Universal Asynchronous Receiver-Transmitter) est essentiel pour interfacer notre microcontrôleur avec un ordinateur. Il permet de transférer les données et les instructions entre l'ordinateur et le WT32 ETH01 de manière fiable.

Le CP2102 est connu pour sa facilité d'utilisation et sa compatibilité avec divers systèmes d'exploitation, incluant Windows, MacOS et Linux. Grâce à cet adaptateur, nous pouvons facilement charger le firmware, surveiller les messages de debug, et effectuer des mises à jour de configuration sur notre microcontrôleur.

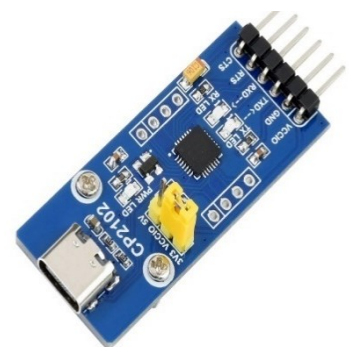


Figure 69. L'Adaptateur CP2102

Le système de contrôle des barrières WT32 comprend plusieurs composants clés :

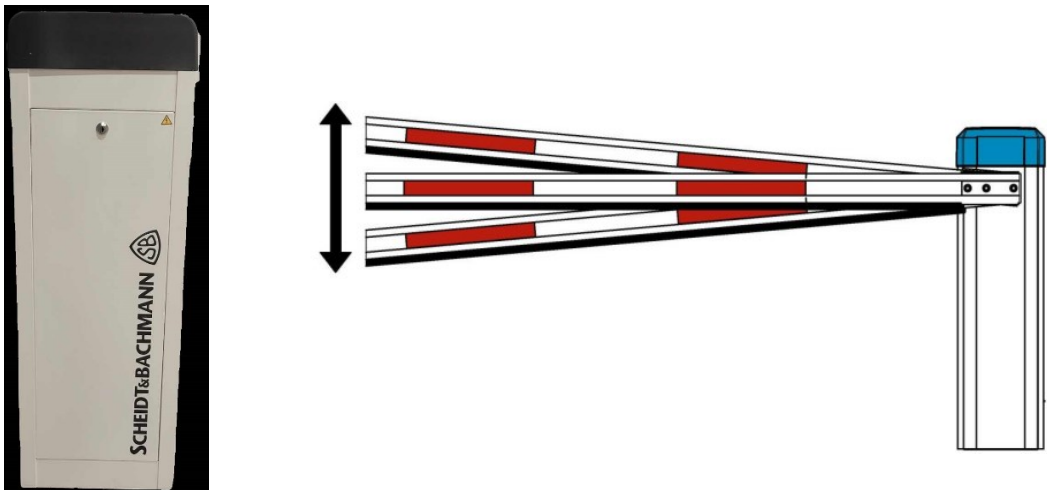
Tableau 25. Les composants de WT32

Technologies / Bibliothèques	Description	Utilisation
WT32-ETH01	Le WT32-ETH01 est un module basé sur l'ESP32, spécialement conçu pour fournir une connectivité Ethernet et Wi-Fi dans un seul appareil. Il est couramment utilisé dans les applications IoT pour sa polyvalence et sa capacité à gérer plusieurs interfaces réseau.	Fournit la connectivité Ethernet et Wi-Fi pour le projet. Les broches GPIO du WT32-ETH01 sont utilisées pour contrôler les mécanismes de barrière.
Relai	Un relai est un interrupteur électromécanique qui permet de contrôler un circuit haute puissance avec un signal basse puissance.	: Les relais sont utilisés pour activer ou désactiver les mécanismes de barrière en fonction des signaux reçus par le WT32-ETH01. Les broches GPIO contrôlent les relais pour ouvrir ou fermer les barrières.
BLE (Bluetooth Low Energy)	Le Bluetooth Low Energy est une technologie sans fil conçue pour une faible consommation d'énergie, idéale pour les appareils alimentés par batterie et les applications IoT. Contrairement au Bluetooth classique, le BLE est optimisé pour des transmissions de données périodiques et de faible volume, ce qui le rend parfait pour les capteurs, les dispositifs de suivi et les interactions à courte portée.	Le BLE est utilisé pour scanner les appareils à proximité et récupérer des données spécifiques, telles que l'UUID (Universal Unique Identifier) de l'entreprise. Cette fonctionnalité permet de détecter la présence d'appareils autorisés dans la zone de stationnement et d'activer les barrières en conséquence.
WiFiManager	Une bibliothèque pour ESP32 qui permet de configurer Wi-Fi via un portail web (Web Portal).	Permettre la connexion aux réseaux Wi-Fi et de gestion des identifiants réseau.

SPIFFS (SPI Flash File System)	Un système de fichiers léger pour les dispositifs flash SPI NOR sur les microcontrôleurs, tels que le WT32-ETH01.	Stocke les fichiers de configuration dans la mémoire flash du WT32-ETH01, permettant des paramètres persistants entre les redémarrages.
ArduinoJson	Une bibliothèque JSON pour Arduino et projets C++ embarqués.	Analyse et génère des fichiers de configuration JSON, permettant un stockage et une récupération facile des données de configuration.
ArduinoOTA	Permet les mises à jour par voie aérienne (OTA) pour les appareils ESP32.	Permet de mettre à jour le firmware du WT32-ETH01 à distance sans accès physique.
ETH	Bibliothèque Ethernet pour le WT32-ETH01.	Fournit une connectivité Ethernet, permettant la communication réseau filaire.
PubSubClient	Une bibliothèque client MQTT simple pour Arduino, ESP8266 et ESP32.	Gère la communication avec un broker MQTT, permettant l'échange de messages avec d'autres appareils et systèmes.

5. Barrière

ELKA fabrique des systèmes de barrières automatiques, de bornes escamotables et de solutions de contrôle d'accès, utilisés pour sécuriser parkings, entrées d'immeubles et zones industrielles. Ces barrières sont rapides, robustes, sûres, et équipées de détection d'obstacles. Faciles à installer et entretenir, elles existent en diverses configurations pour répondre à différents besoins.



Figures 70. Systèmes de barrières automatiques

- Caractéristique :

Tableau 26. Les caractéristiques

Caractéristique	Détail
Marque barrière	ELKA
Modèle	P 2500-5000
Longueur du flèche (Boom)	2500 mm (2.5 m)
Type de communication	TCP / IP, Cable RJ45 / Direct

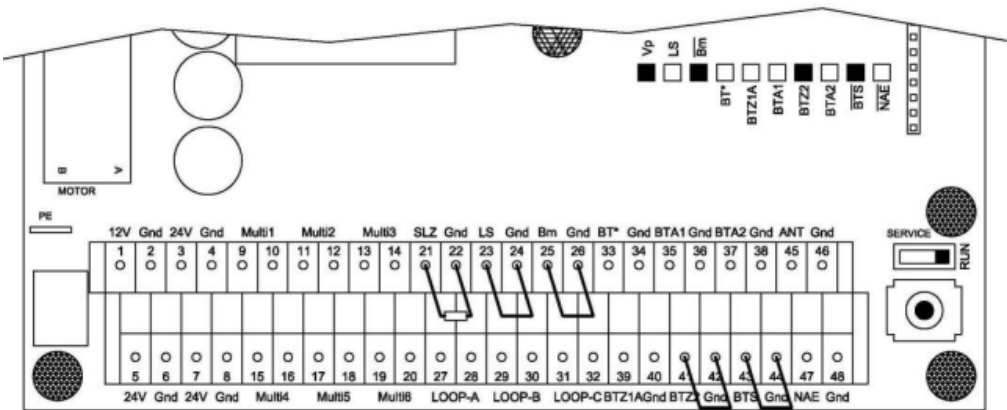


Figure 71 : Système d'I/O de la barrière

5.1 Description du ‘ Découvrir le fonctionnement du système’

- **Détection et filtrage des signaux BLE :**

- Le microcontrôleur ESP32 scanne en continu les signaux BLE.
- Les signaux BLE détectés sont filtrés en fonction d'un UUID spécifique de l'entreprise pour s'assurer que seuls les signaux autorisés sont traités.
- Les données pertinentes telles que le numéro de sortie (indiquant quelle barrière à contrôler) et la durée pendant laquelle la barrière doit rester ouverte sont extraites du signal BLE.

- **Réglage des états et de la durée des sorties :**

- Le système règle l'état de la sortie correspondante de la barrière (HIGH pour ouvrir, LOW pour fermer) en fonction des données extraites.
- Un minuteur est lancé pour contrôler la durée pendant laquelle la barrière reste ouverte. Par exemple, si un signal indique que la barrière doit rester ouverte pendant 5 secondes, le système règle la sortie sur HIGH et lance un minuteur de 5 secondes, après quoi la barrière est fermée en réglant la sortie sur LOW.

- **Communication MQTT :**

- Le système se connecte à un serveur MQTT pour faciliter la communication avec le système de reconnaissance de plaques d'immatriculation.
- Il s'abonne aux sujets MQTT pertinents et traite les messages entrants pour contrôler les opérations de la barrière en fonction des données de la plaque d'immatriculation.

- **Initialisation et configuration :**

- Lors de la configuration, le système initialise la connectivité Wi-Fi/Ethernet, le scan BLE et lit la configuration depuis SPIFFS.
- WiFiManager [3] est utilisé pour gérer les identifiants réseau et fournir un portail web pour la configuration.

- **Mises à jour OTA :**

- Les mises à jour Over-the-Air (OTA) sont gérées à l'aide de la bibliothèque ArduinoOTA, permettant l'application de mises à jour du firmware à distance.
- Le portail web du système facilite la configuration à distance des paramètres réseau et MQTT.

5.2 Description du 'Gestion des configurations'

Le système de contrôle des barrières de parking permet une gestion complète de la configuration via une interface web accessible à l'adresse IP locale 192.168.4.1. Cette interface offre plusieurs fonctionnalités essentielles :

- **Configuration WiFi :** Connectez et gérez les paramètres WiFi pour assurer une communication réseau fiable.
- **Paramètres Réseau :** Définissez des adresses IP statiques, des passerelles et des masques de sous-réseau pour maintenir des configurations réseau cohérentes.
- **Configuration du Serveur MQTT :** Gérez les paramètres pour le serveur MQTT, y compris l'adresse du serveur, le port, le nom d'utilisateur et le mot de passe. Ces paramètres facilitent une communication sécurisée et efficace avec le serveur MQTT pour la gestion des messages et les opérations de contrôle à distance.

L'interface web est accessible uniquement dans des situations spécifiques pour des raisons de sécurité et de praticité :

- **Absence de Réseau Wi-Fi Méorisé :** Si le système ne trouve pas le réseau Wi-Fi mémorisé dans la mémoire, il ouvre automatiquement un point d'accès pour permettre la configuration.
- **Réinitialisation Manuelle :** Si vous appuyez sur le bouton dédié pour effacer la mémoire du WT32, le système ouvre également un point d'accès, permettant ainsi de reconfigurer le WiFi et d'autres paramètres réseau.

5.3 Description du ' Implémentation et réalisation '

5.3.1 Logiciel Utilisé

Pour la réalisation de ce projet, plusieurs programmes ont été utilisés pour faciliter la communication, la gestion et la simulation des dispositifs. Voici une brève description de chacun de ces programmes, accompagnée de captures d'écran pour illustrer leur utilisation et leur interface.

• Mosquitto

Mosquitto est un broker MQTT léger et open-source, essentiel pour la gestion de la communication entre les appareils IoT. Il facilite la publication, la réception et la distribution des messages MQTT, assurant une transmission de données fiable et rapide entre le microcontrôleur WT32-ETH01 et le serveur central.mq

```
#  
# listener port-number [ip address/host name/unix socket path]  
listener 1883 0.0.0.0  
allow_anonymous true
```

• MQTT Explorer

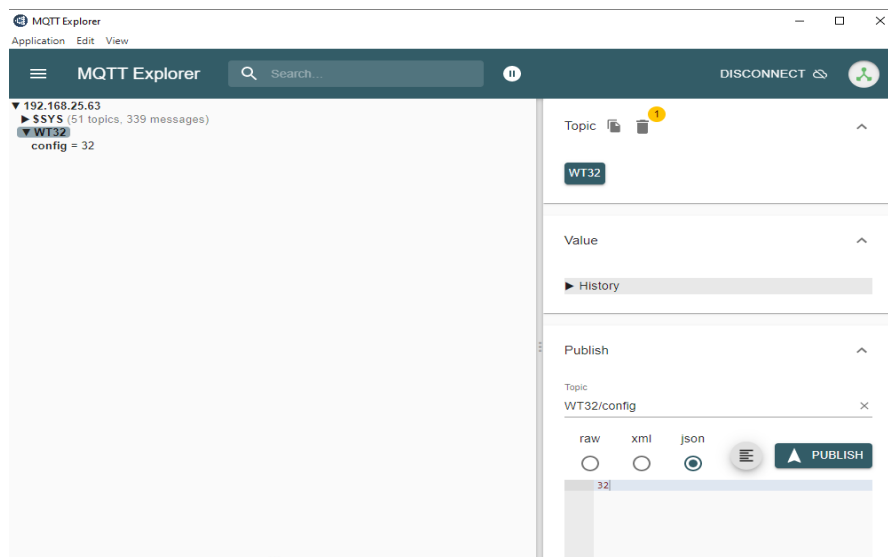


Figure 72. MQTT Explorer

MQTT Explorer est un outil graphique qui permet de visualiser et de gérer les messages MQTT. Il offre une interface intuitive pour explorer les topics MQTT, examiner les messages échangés, et surveiller l'état des connexions. Cela simplifie le débogage et l'analyse des communications MQTT dans le système.

• Beacon Simulator

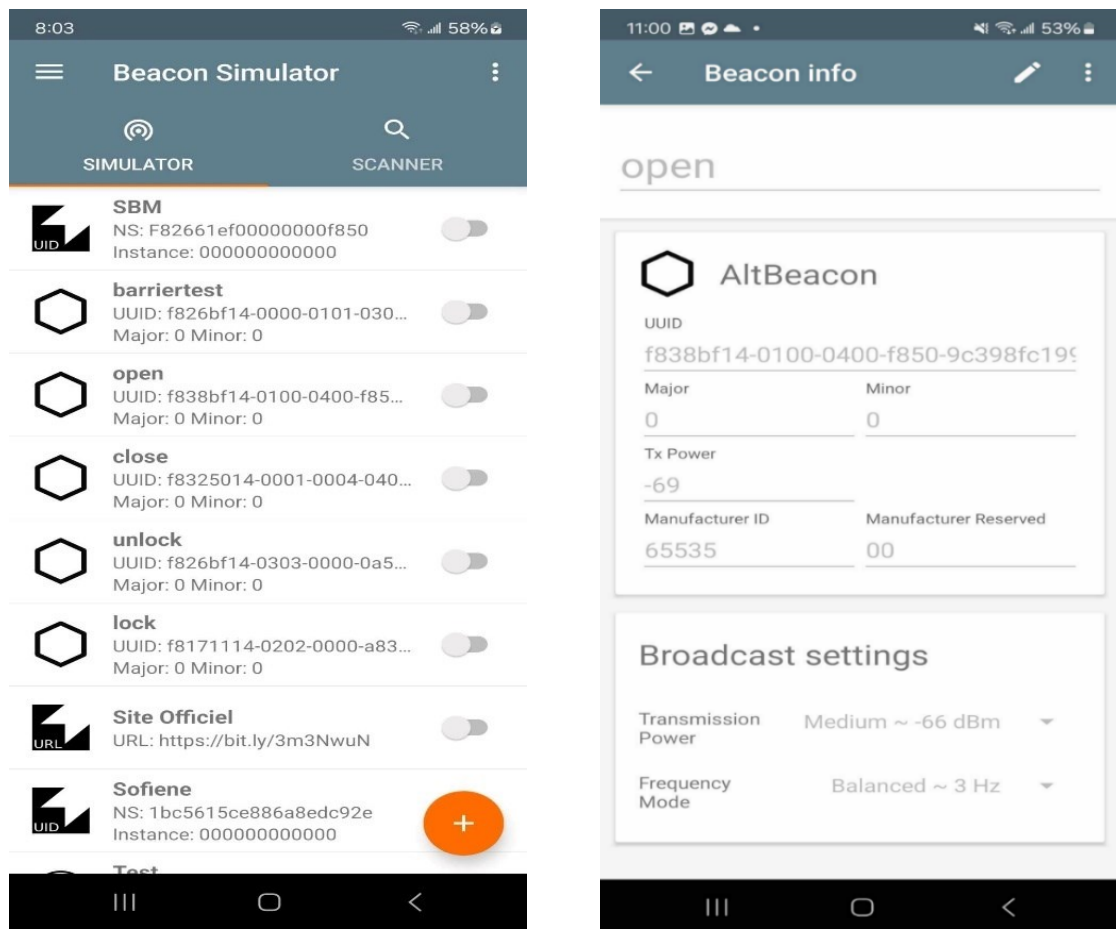


Figure73. Beacon Simulator

Beacon Simulator est une application mobile qui émule les balises BLE (Bluetooth Low Energy). Elle permet de tester la détection et la communication des balises avec le microcontrôleur WT32-ETH01, assurant ainsi une interaction fluide entre les appareils mobiles et le système de contrôle des barrières. L'application offre une interface intuitive et de nombreuses options de personnalisation pour simuler divers scénarios et configurations de balises. Cela facilite le développement, le débogage et l'optimisation des systèmes de contrôle des barrières.

5.3.2 Interface Graphique

L'interface graphique de gestion du système de contrôle des barrières de parking, accessible à l'adresse IP locale 192.168.4.1, offre plusieurs fonctionnalités organisées en différentes sections pour une gestion complète et intuitive :

- **Configurer WiFi**
 - Permet de connecter le système à un réseau WiFi spécifique en saisissant les paramètres requis.
- **Info**
 - **Uptime** : Affiche la durée de fonctionnement continue du dispositif.
 - **Chip ID** : Identifiant unique du microcontrôleur.
 - **Chip Revision** : Version du microcontrôleur utilisé.
 - **Flash Size** : Taille de la mémoire flash disponible.
 - **PSRAM Size** : Taille de la mémoire PSRAM disponible.
 - **CPU Frequency** : Fréquence du processeur.
 - **Memory Free Heap** : Quantité de mémoire libre dans le tas.
 - **Memory Sketch Size** : Taille de la mémoire utilisée par le programme.
 - **Temperature** : Température actuelle du dispositif.
 - **WiFi Status** : Statut actuel de la connexion WiFi.
 - **Effacer Configuration** : Option pour réinitialiser les paramètres de configuration du dispositif.
- **Update**
 - **Télécharger Nouveau Firmware** : Permet de mettre à jour le firmware du dispositif en téléchargeant une nouvelle version. Cette fonctionnalité assure que le système peut être facilement mis à jour avec les dernières améliorations et corrections de bugs.
- **Exit**
 - Ferme l'interface graphique et retourne à l'écran précédent ou au mode normal de fonctionnement.

Ces sections de l'interface graphique sont conçues pour offrir une gestion efficace et facile du système de contrôle des barrières de parking, permettant aux utilisateurs de configurer, surveiller et mettre à jour le dispositif en toute simplicité

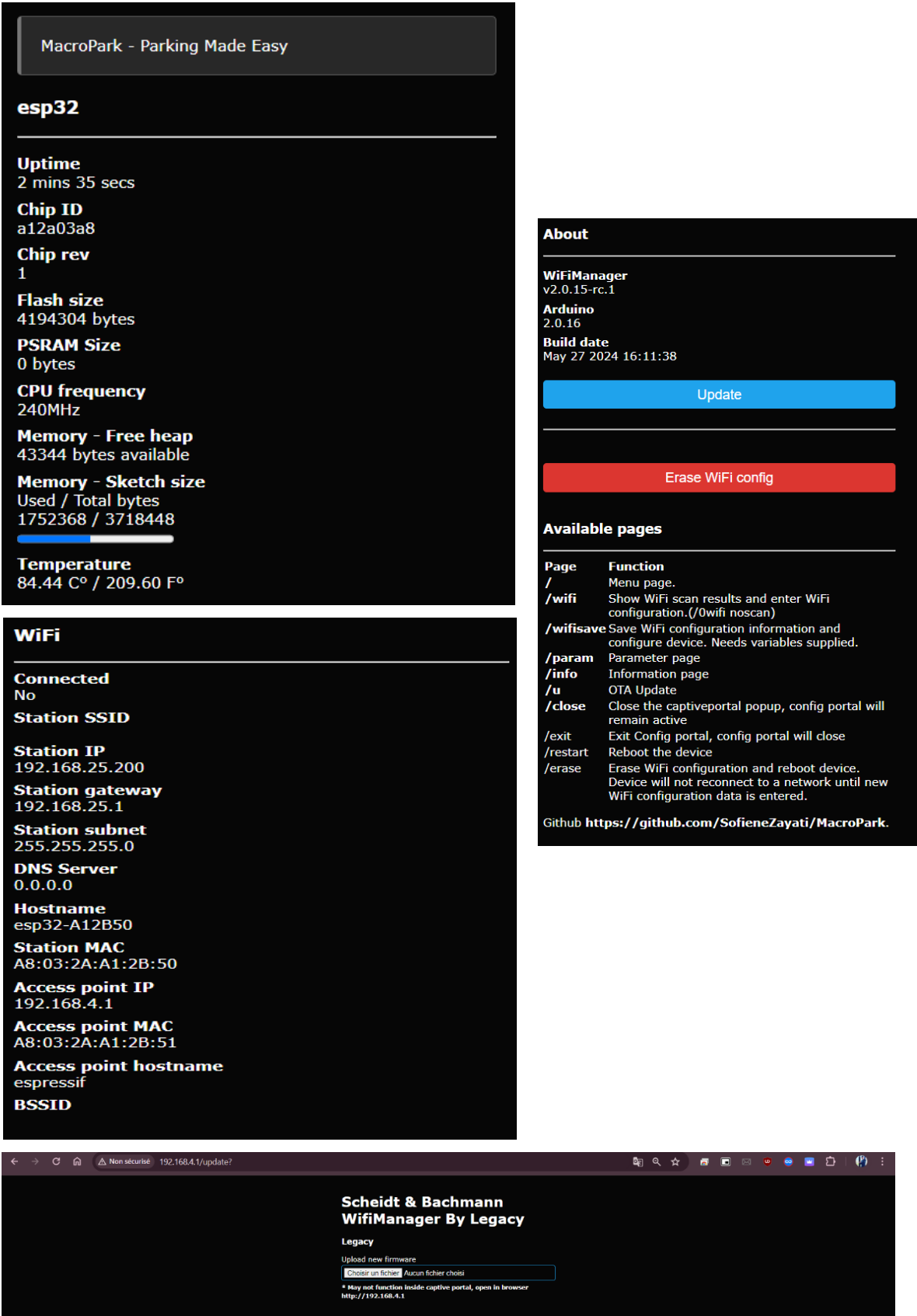


Figure 74. Interfaces Graphiques

5.4 Description du 'Schéma et Housing'

5.4.1 Housing

Pour protéger et organiser le matériel électronique de notre système de contrôle des barrières de parking, nous avons conçu et imprimé en 3D un boîtier personnalisé. Ce boîtier, réalisé avec SolidWorks, est optimisé pour garantir une installation propre et fonctionnelle des composants.

Le boîtier comprend les caractéristiques suivantes :

- **Port Ethernet** : Permet la connexion filaire pour une communication réseau stable et fiable.
- **Port d'Alimentation** : Interface pour connecter l'alimentation électrique nécessaire au fonctionnement du dispositif.
- **Sorties Relais** : Connecteurs pour les commandes de relais, permettant de contrôler les barrières de manière efficace.
- **Bouton de Réinitialisation** : Un bouton dédié pour réinitialiser le système, utile pour effacer les paramètres de configuration ou redémarrer l'appareil.

L'intégration de ces éléments dans un boîtier imprimé en 3D assure non seulement la protection des composants électroniques contre les dommages physiques et les interférences, mais facilite également l'installation et la maintenance du système sur le terrain.

Voici quelques captures d'écran de la conception et de la réalisation de ce boîtier, créées avec SolidWorks :

- **Type de vue : en haut complète**

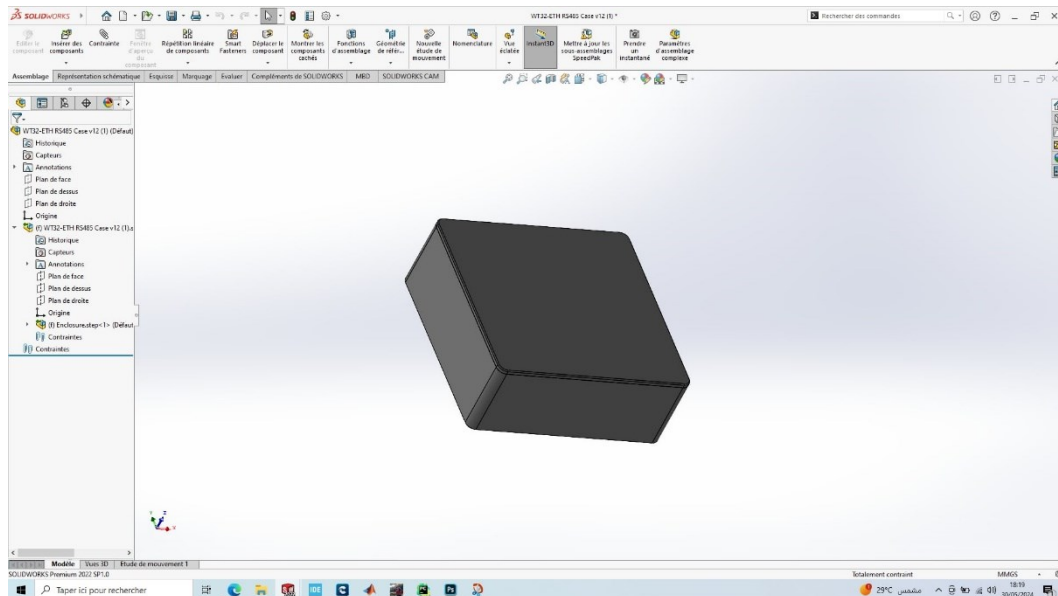


Figure 75. Type de vue : en haut complète

- **Type de vue : en haut sans cache**

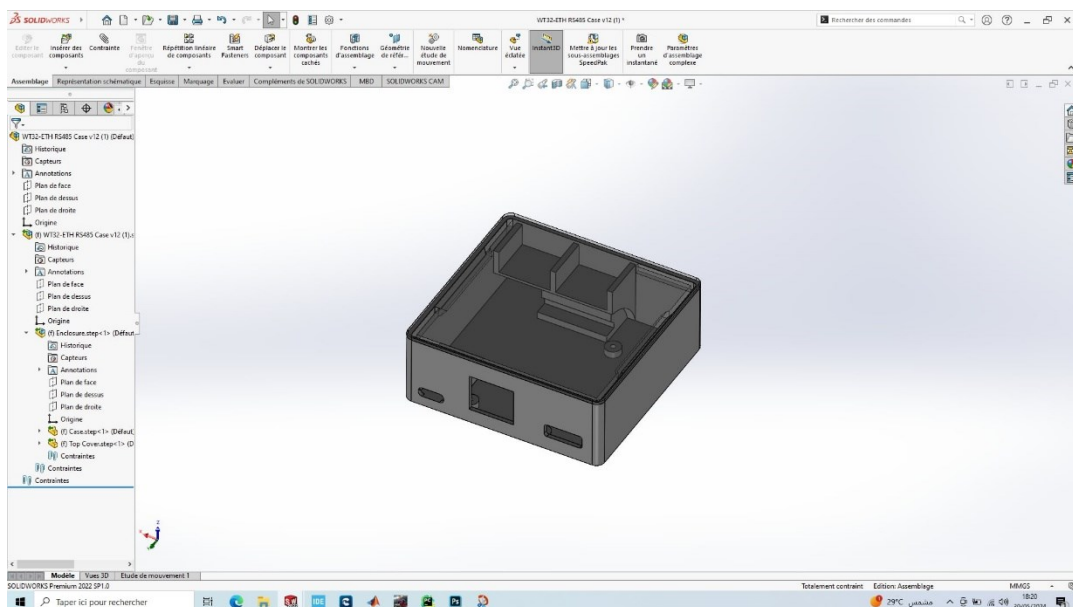


Figure 76. Type de vue : en haut sans cache

- **Type du vue : droite**

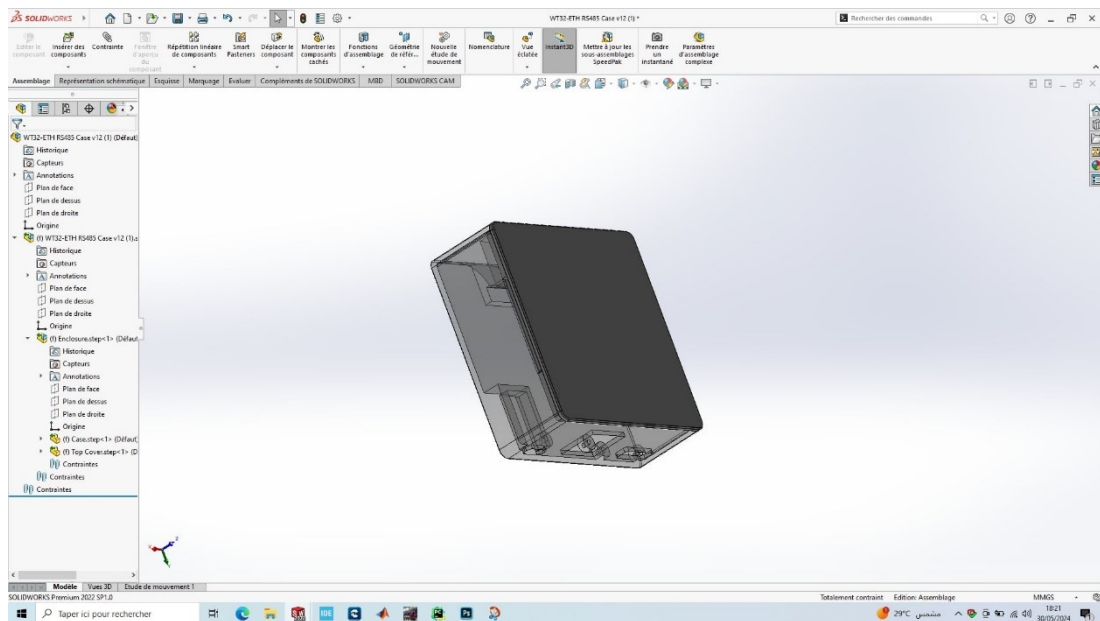


Figure 77. Type de vue : en droite

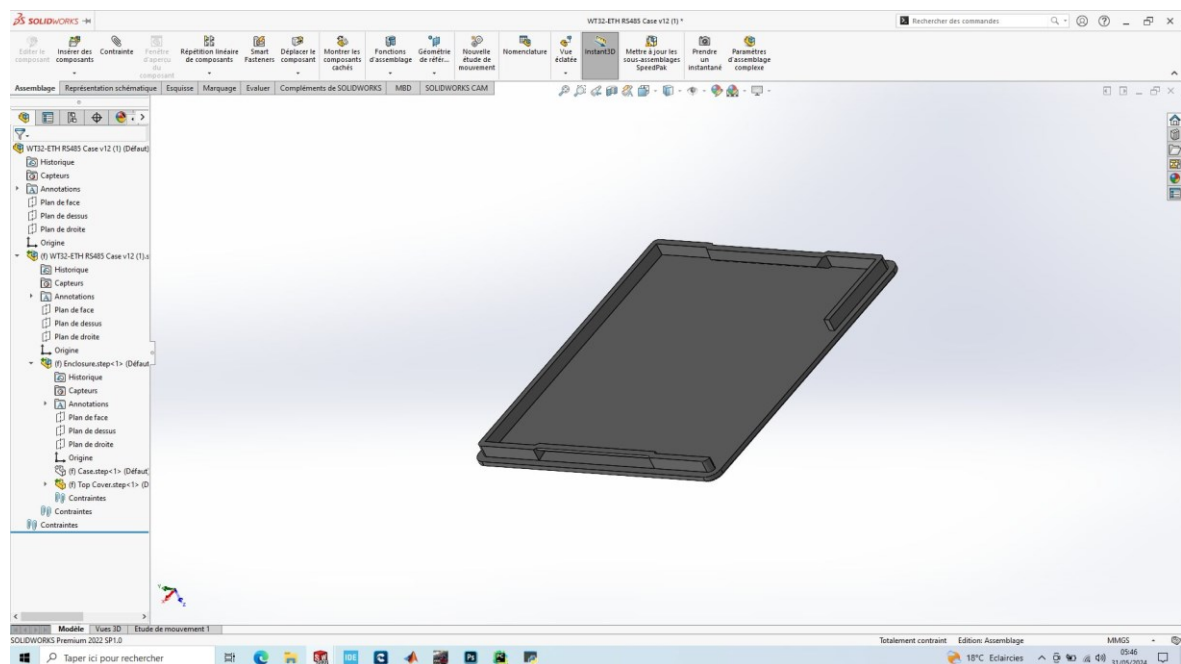


Figure 78. Type de vue : Couvercle

5.4.2 Impression 3d

- **Résumé sur l'Imprimante 3D Ender :**

L'impression 3D, ou fabrication additive, permet de créer des objets complexes à partir de modèles numériques. L'imprimante 3D Ender de **Creality** est appréciée pour son rapport qualité-prix et sa fiabilité.

- **Applications dans le cadre du projet :**

- **Prototypage Rapide** : Création rapide de prototypes pour valider des concepts.
- **Recherche et Développement** : Permet des expérimentations et itérations rapides.
- **Éducation** : Support d'apprentissage pratique des principes de la fabrication additive.
- **Projets Innovants** : Idéale pour des projets variés, de l'ingénierie à l'art.

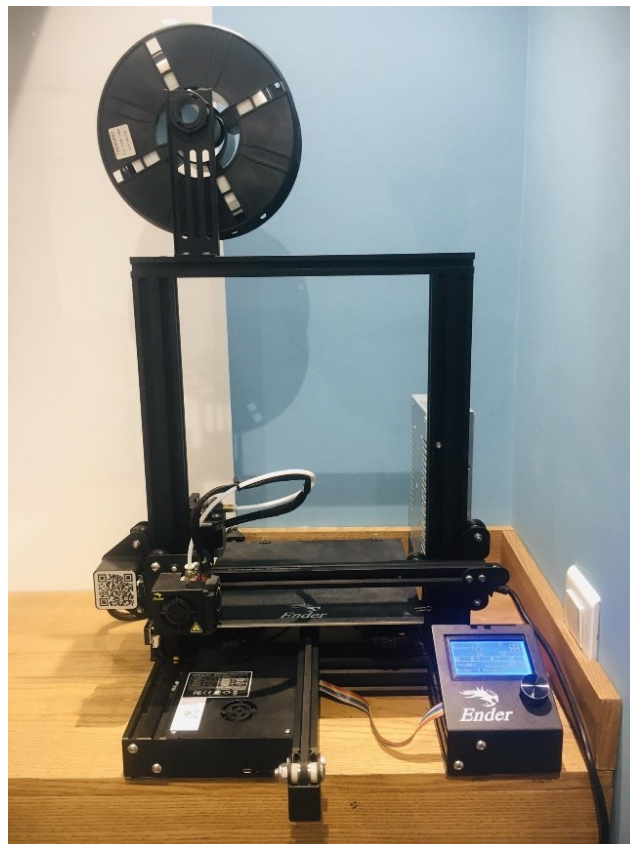


Figure 78. L'imprimante 3D - ENDER

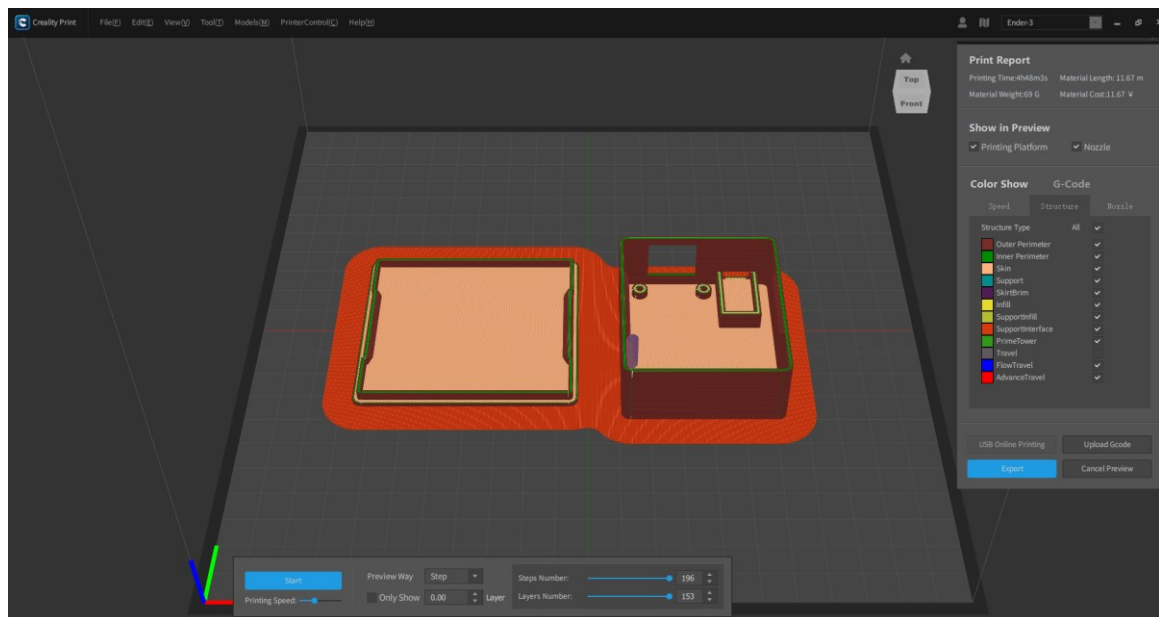


Figure 79. Modèle numérique

5.4.3 Câblage

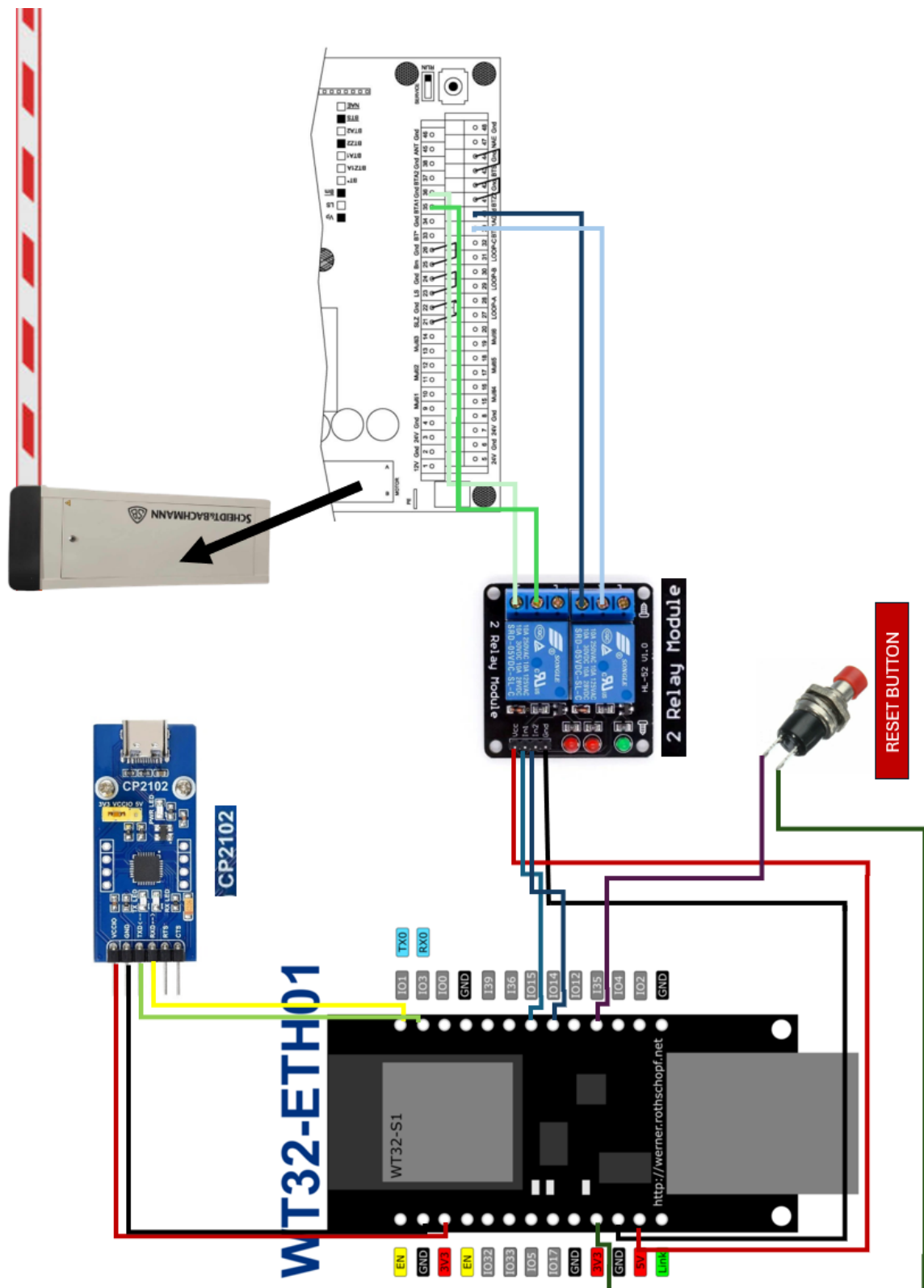


Figure 80. Câblage

5.5 Explication du code

• Bibliothèques

Ces bibliothèques fournissent des fonctionnalités pour la gestion du système de fichiers, la connexion WiFi, la mémoire flash SPI, la manipulation des données JSON, les mises à jour Over-The-Air (OTA), la gestion Ethernet, la communication MQTT et les opérations Bluetooth Low Energy (BLE).

```
#include <FS.h>           // Gestion du système de fichiers
#include <WiFiManager.h>   // Gestion de la connexion WiFi
#include <SPIFFS.h>        // Système de fichiers Flash SPI
#include <ArduinoJson.h>   // Manipulation des données JSON
#include <ArduinoOTA.h>    // Mises à jour Over-The-Air
#include <ETH.h>           // Gestion Ethernet pour WT32-ETH01
#include <PubSubClient.h>  // Communication MQTT
#include <BLEDevice.h>      // Gestion des appareils BLE
#include <BLEUtils.h>      // Utilitaires BLE
#include <BLEScan.h>       // Fonctionnalités de scan BLE
#include <BLEAdvertisedDevice.h> // Gestion des appareils BLE annoncés
#include "BLEBeacon.h"     // Gestion des balises BLE
#include "BLEEddystoneTLM.h" // Format Eddystone TLM
#include "BLEEddystoneURL.h" // Format Eddystone URL
```

Figure 81. Les Bibliothèques

• Variables globales et constantes

◦ Définitions des broches (**Pin**)

Les broches sont définies pour diverses barrières et un bouton pour supprimer les paramètres enregistrer sur le SPIFFS.

```
#define TRIGGER_PIN 35
#define B1OPEN 15 // Barrier 1 Open Command 2322
#define B1CLOSE 14 // Barrier 1 Close Command
#define B2OPEN 4 // Barrier 2 Open Command
#define B2CLOSE 2 // Barrier 2 Close Command

#define B3OPEN 33 // Barrier 3 Open Command
#define B3CLOSE 32 // Barrier 3 Close Command
#define B4OPEN 17 // Barrier 4 Open Command
#define B4CLOSE 5 // Barrier 4 Close Command
#define ENDIAN_CHANGE_U16(x) (((x)&0xFF00) >> 8) + (((x)&0xFF) << 8))
```

Figure 82. Variables globales et constantes

○ Contrôle du timing et de l'état des barrières

Variables pour contrôler le timing et l'état des barrières.

```
unsigned long startTime1 = 0, stopTimeS1 = 0;
unsigned long startTime2 = 0, stopTimeS2 = 0;
unsigned long startTime3 = 0, stopTimeS3 = 0;
unsigned long startTime4 = 0, stopTimeS4 = 0;
bool actionInProgress1 = false, actionInProgress2 = false, actionInProgress3 = false,
actionInProgress4 = false;
```

Figure 83. Contrôle du timing et de l'état des barrières

○ Configuration BLE et réseau

UUID et temps de scan pour BLE, configuration réseau pour MQTT.

```
uint16_t beconUUID = 0xFEAA;
uint8_t companyUUID = 38;
int scanTime = 1; //In seconds
static bool eth_connected = false;

// Serveur MQTT et configuration réseau
char mqtt_server[40] = "mqtt-server";
char mqtt_port[6] = "1883";
char mqtt_username[40] = "username";
char mqtt_password[40] = "my_password";
char static_ip[16] = "192.168.1.32";
char static_gw[16] = "192.168.1.1";
char static_sn[16] = "255.255.255.0";
const char* auth_topic = "WT32/auth";
const char* config_topic = "WT32/config";
const char* access_topic = "WT32/access";
bool shouldSaveConfig = false;
uint8_t receivedCompanyID = 0;

// Configurations IP réseau
IPAddress _ip, _gw, _sn;
String accessMessage, msg, SavedIP, SavedGateway, SavedSubnet;

// Objet de scan BLE et clients WiFi/MQTT
BLEScan* pBLEScan;
WiFiManager wifiManager;
WiFiClient espClient;
PubSubClient client(espClient);
```

Figure 84. Configuration BLE et réseau

- **Classes de rappel et fonctions**
 - Rappels des appareils annoncés BLE
 - Gère les annonces BLE.

```
class MyAdvertisedDeviceCallbacks : public BLEAdvertisedDeviceCallbacks {
  void onResult(BLEAdvertisedDevice advertisedDevice) {
    std::string strServiceData = advertisedDevice.getManufacturerData();
    uint8_t cServiceData[100];
    strServiceData.copy((char*)cServiceData, strServiceData.length(), 0);
    Serial.printf("Advertised Device: %s \n", advertisedDevice.toString().c_str());
  }
};
```

- Gestionnaire d'événements Ethernet
- Gère divers événements Ethernet.

```
void WiFiEvent(WiFiEvent_t event) {
  switch (event) {
    case ARDUINO_EVENT_ETH_START:
      Serial.println("ETH Started");
      //set eth hostname here
      ETH.setHostname("Barrier Control Device");
      break;
    case ARDUINO_EVENT_ETH_CONNECTED:
      Serial.println("ETH Connected");
      break;
    case ARDUINO_EVENT_ETH_GOT_IP:
      Serial.print("ETH MAC: ");
      Serial.print(ETH.macAddress());
      Serial.print(", IPv4: ");
      Serial.print(ETH.localIP());
      if (ETH.fullDuplex()) {
        Serial.print(", FULL_DUPLEX");
      }
      Serial.print(", ");
      Serial.print(ETH.linkSpeed());
      Serial.println("Mbps");
      eth_connected = true;
      break;
    case ARDUINO_EVENT_ETH_DISCONNECTED:
      Serial.println("ETH Disconnected");
      eth_connected = false;
      break;
    case ARDUINO_EVENT_ETH_STOP:
      Serial.println("ETH Stopped");
      eth_connected = false;
      break;
    default:
      break;
  }
```

Figure 85. Classes de rappel et fonctions

- **Fonctions supplémentaires**

- Fonctions diverses pour arrêter le scan BLE « stop Scan() »
- Enregistrer la configuration « saveConfigCallback() »
- Vérifier l'état du bouton qui réinitialise la configuration et ouvre le Point D'accès pour configurer le WiFi

```
// Fonction pour arrêter le scan BLE
void stopScan() {
    pBLEScan->stop();
    delay(5000);
}

// Fonction de rappel pour enregistrer la configuration
void saveConfigCallback() {
    shouldSaveConfig = true;
}

// Fonction pour vérifier l'appui sur le bouton et réinitialiser si maintenu
void checkButton() {
    if (digitalRead(TRIGGER_PIN) == HIGH) {
        Serial.println("Button Pressed");
        delay(5000); // Reset delay hold
        if (digitalRead(TRIGGER_PIN) == HIGH) {
            Serial.println("Button Held");
            Serial.println("Erasing Config, restarting");
            wifiManager.resetSettings();
            ESP.restart();
        }
    }
}
```

Figure 86. Fonctions supplémentaires

- **Fonction de rappel MQTT**

- Gère les messages MQTT entrants.

```
void callback(char* topic, byte* payload, unsigned int length) {
    if (strcmp(topic, config_topic) == 0) {
        Serial.println("Received Company ID");
        receivedCompanyID = atoi((char*)payload);
        // Convert the payload to an integer
        Serial.print("Company ID: ");
        Serial.println(receivedCompanyID);
        companyUUID = receivedCompanyID;
        shouldSaveConfig = true;
        saveConfigCallback();
    }
}
```

```

else if (strcmp(topic, access_topic) == 0)
{
    // Received message on access topic
    Serial.println("Received order to open ");
    for (int i = 0; i < length; i++) {
        msg = msg + (char)payload[i]; // convert *byte to string
    }
    accessMessage = msg;
    Serial.print("Received access message: ");
    Serial.println(accessMessage);
}
}

```

Figure 87. Fonction de rappel MQTT

- **Fonction de reconnexion**

- Reconnecte au serveur MQTT si déconnecté.

```

void reconnect() {
    while (!client.connected()) {
        Serial.print("Attempting MQTT connection...");
        if (client.connect("ESP32Client")) {
            Serial.println("connected");
            client.subscribe(auth_topic);
            client.subscribe(config_topic);
            client.subscribe(access_topic);
        } else {
            checkButton();
            Serial.print("failed, rc=");
            Serial.print(client.state());
            Serial.println(" try again in 5 seconds");
            Serial.print("MQTT Server: ");
            Serial.println(mqtt_server);
            Serial.print("MQTT Port: ");
            Serial.println(mqtt_port);
            delay(5000);}
    }
}

```

Figure 88. Fonction de reconnexion

- **Fonction setup**

- Configuration initiale

Cette section configure les paramètres initiaux tels que le mode WiFi, le débogage série, les broches GPIO et les paramètres de la bibliothèque WiFiManager.

```
WiFi.mode(WIFI_STA);
Serial.begin(115200);
WiFi.onEvent(WiFiEvent);
Serial.setDebugOutput(true);
pinMode(TRIGGER_PIN, INPUT_PULLUP);
pinMode(B1OPEN, OUTPUT);
pinMode(B1CLOSE, OUTPUT);
pinMode(B2OPEN, OUTPUT);
pinMode(B2CLOSE, OUTPUT);
pinMode(B3OPEN, OUTPUT);
pinMode(B3CLOSE, OUTPUT);
pinMode(B4OPEN, OUTPUT);
pinMode(B4CLOSE, OUTPUT);

digitalWrite(B1OPEN, LOW);
digitalWrite(B1CLOSE, LOW);
digitalWrite(B2OPEN, LOW);
digitalWrite(B2CLOSE, LOW);
digitalWrite(B3OPEN, LOW);
digitalWrite(B3CLOSE, LOW);
digitalWrite(B4OPEN, LOW);
digitalWrite(B4CLOSE, LOW);

wifiManager.setDarkMode(true);
wifiManager.setTitle("Scheidt & Bachmann WifiManager By Legacy");
```

Figure89. Fonction setup

- Initialisation des périphériques BLE

Cette section initialise le scanneur BLE et monte le système de fichiers SPIFFS.

```
Serial.println("Scanning...");
BLEDevice::init("");
pBLEScan = BLEDevice::getScan();
pBLEScan->setAdvertisedDeviceCallbacks(new MyAdvertisedDeviceCallbacks());
pBLEScan->setActiveScan(true);
Serial.println("mounting FS...");
if (SPIFFS.begin()) {
    Serial.println("mounted file system");
    if (SPIFFS.exists("/park.json")) {
        Serial.println("reading config file");
        File configFile = SPIFFS.open("/park.json", "r");
        if (configFile) {
            Serial.println("opened park file");
            size_t size = configFile.size();
            std::unique_ptr<char[]> buf(new char[size]);
            configFile.readBytes(buf.get(), size);
#ifdef ARDUINOJSON_VERSION_MAJOR && ARDUINOJSON_VERSION_MAJOR >=6
            DynamicJsonDocument json(1024);
            auto deserializeError = deserializeJson(json, buf.get());
            serializeJson(json, Serial);
            if (!deserializeError) {
#else
            DynamicJsonBuffer jsonBuffer;
            JsonObject& json = jsonBuffer.parseObject(buf.get());
            json.printTo(Serial);
            if (json.success()) {
#endif
                Serial.println("\nparsed json");
                strcpy(mqtt_server, json["mqtt_server"]);
                strcpy(mqtt_port, json["mqtt_port"]);
                strcpy(mqtt_username, json["mqtt_username"]);
                strcpy(mqtt_password, json["mqtt_password"]);
                if (json["ip"]) {
                    Serial.println("setting custom ip from config");
                    strcpy(static_ip, json["ip"]);
                    strcpy(static_gw, json["gateway"]);
                    strcpy(static_sn, json["subnet"]);
                    Serial.println(static_ip);
                } else {
                    Serial.println("failed to load json config");
                }
                companyUUID = json["companyUUID"];
                configFile.close();
            } else {
                Serial.println("failed to mount FS");
            }
        }
    }
}
```

Figure 90. Initialisation des périphériques BLE

- Config Réseau

Cette section nous permet de voir le config de WiFi sur le serial (IP, Gateway, Subnet)

```
Serial.println(static_ip);  
Serial.println(mqtt_server);  
Serial.println(mqtt_port);
```

Figure 91. Config Réseau

- Ajouter des paramètres au WebServer du WifiManager

Cette section nous permet d'entrer MQTT Server, Port, Username, Password et config WiFi

```
WiFiManagerParameter custom_mqtt_server("server", "MQTT server", mqtt_server, 40);  
WiFiManagerParameter custom_mqtt_port("port", "MQTT port", mqtt_port, 6);  
WiFiManagerParameter custom_mqtt_username("username", "MQTT username",  
mqtt_username, 40);  
WiFiManagerParameter custom_mqtt_password("password", "MQTT password",  
mqtt_password, 40);  
wifiManager.setSaveConfigCallback(saveConfigCallback);  
wifiManager.setSTAStaticIPConfig(_ip, _gw, _sn);  
wifiManager.addParameter(&custom_mqtt_server);  
wifiManager.addParameter(&custom_mqtt_port);  
wifiManager.addParameter(&custom_mqtt_username);  
wifiManager.addParameter(&custom_mqtt_password);
```

Figure92. Ajouter des paramètres au WebServer du WifiManager

- Config ETH

Cette section nous permet d'entrer le config Ethernet (IP, Gateway, Subnet)

```
ETH.begin(1, 16, 23, 18, ETH_PHY_LAN8720);  
ETH.config(_ip, _gw, _sn);
```

Figure93. Config ETH

- Connection

Cette section nous permet de connecter sur Ethernet si disponible sinon connecté sur WiFi

```
if (eth_connected) {
  Serial.println("Connected to Ethernet");
  Serial.print("IP Address: ");
  Serial.println(ETH.localIP());
  SavedIP = ETH.localIP().toString();
  Serial.println(ETH.gatewayIP());
  SavedGateway = ETH.gatewayIP().toString();
  Serial.println(ETH.subnetMask());
  SavedSubnet = ETH.subnetMask().toString(); } else {
  wifiManager.autoConnect("Legacy");
  Serial.println("Connected to WiFi");
  Serial.println("local ip");
  Serial.println(WiFi.localIP());
  SavedIP = WiFi.localIP().toString();
  Serial.println(WiFi.gatewayIP());
  SavedGateway = WiFi.gatewayIP().toString();
  Serial.println(WiFi.subnetMask());
  SavedSubnet = WiFi.subnetMask().toString();}
```

Figure 94. connexion

- ArduinoOTA

Cette section du code configure et initialise les mises à jour Over-The-Air (OTA) pour la WT32-ETH01. Les mises à jour OTA permettent de téléverser un nouveau firmware sur l'WT32 sans avoir besoin de le connecter physiquement à un ordinateur.

```
ArduinoOTA.begin();
ArduinoOTA.onStart([]() {
  Serial.println("OTA Update Started"); });
ArduinoOTA.onEnd([]() {
  Serial.println("\nOTA Update Finished"); });
ArduinoOTA.onProgress([](unsigned int progress, unsigned int total) {
  Serial.printf("OTA Progress: %u%%\r", (progress / (total / 100))); });
ArduinoOTA.onError([](ota_error_t error) {
  Serial.printf("OTA Error[%u]: ", error);
  if (error == OTA_AUTH_ERROR) Serial.println("Auth Failed");
  else if (error == OTA_BEGIN_ERROR) Serial.println("Begin Failed");
  else if (error == OTA_CONNECT_ERROR) Serial.println("Connect Failed");
  else if (error == OTA_RECEIVE_ERROR) Serial.println("Receive Failed");
  else if (error == OTA_END_ERROR) Serial.println("End Failed"); });
Serial.println("Ready for OTA Updates");
Serial.println("Connected...");
```

Figure 95. ArduinoOTA

- Affichages donnés

Cette section du code extrait les valeurs des paramètres MQTT (serveur, port, nom d'utilisateur, mot de passe) des objets **WiFiManagerParameter** et les stocke dans les variables correspondantes. Ensuite, elle affiche ces valeurs sur le moniteur série.

```
strcpy(mqtt_server, custom_mqtt_server.getValue());
strcpy(mqtt_port, custom_mqtt_port.getValue());
strcpy(mqtt_username, custom_mqtt_username.getValue());
strcpy(mqtt_password, custom_mqtt_password.getValue());
Serial.println("Values in the file:");
Serial.println("\tmqtt_server : " + String(mqtt_server));
Serial.println("\tmqtt_port : " + String(mqtt_port));
Serial.println("\tmqtt_username : " + String(mqtt_username));
Serial.println("\tmqtt_password : " + String(mqtt_password));
```

Figure96. Affichage données

- MQTT Server

Cette section du code configure le client MQTT avec le serveur et le port appropriés, et définit un rappel (callback) pour gérer les messages entrants.

```
uint16_t port = atoi(mqtt_port);
client.setServer(mqtt_server, port);
client.setCallback(callback);
```

Figure97. MQTT server

- **Fonction Loop :**
 - Gestion des barrières

Cette section vérifie si une action sur une barrière est en cours et si le temps écoulé depuis le début de cette action dépasse une durée définie. Si c'est le cas, l'action est terminée, et la barrière est fermée.

```
unsigned long currentTime1 = millis();

if (actionInProgress1 && currentTime1 - startTime1 >= stopTimeS1) {
  actionInProgress1 = false;
  digitalWrite(B1OPEN, LOW);

  digitalWrite(B1CLOSE, HIGH);
  delay(1000);
  digitalWrite(B1CLOSE, LOW);
  Serial.println("Barrier 1 Closed after Opening");
}
unsigned long currentTime2 = millis();
if (actionInProgress2 && currentTime2 - startTime2 >= stopTimeS2) {
  actionInProgress2 = false;
  digitalWrite(B2OPEN, LOW);
  digitalWrite(B2CLOSE, HIGH);
  delay(1000);
  digitalWrite(B2CLOSE, LOW);
  Serial.println("Barrier 2 Closed after Opening");
}
unsigned long currentTime3 = millis();

if (actionInProgress3 && currentTime3 - startTime3 >= stopTimeS3) {
  actionInProgress3 = false;
  digitalWrite(B3OPEN, LOW);
  digitalWrite(B3CLOSE, HIGH);
  delay(1000);
  digitalWrite(B3CLOSE, LOW);
  Serial.println("Barrier 3 Closed after Opening");
}
unsigned long currentTime4 = millis();
if (actionInProgress4 && currentTime4 - startTime4 >= stopTimeS4) {
  actionInProgress4 = false;
  digitalWrite(B4OPEN, LOW);
  digitalWrite(B4CLOSE, HIGH);
  delay(1000);
  digitalWrite(B4CLOSE, LOW);
  Serial.println("Barrier 4 Closed after Opening");
}
```

Figure 98. Gestion des barrières

Si `shouldSaveConfig` est vrai, les paramètres de configuration MQTT et les informations réseau sont sauvegardés dans un fichier JSON sur le SPIFFS

```
if (shouldSaveConfig) {
  #if defined(ARDUINOJSON_VERSION_MAJOR) && ARDUINOJSON_VERSION_MAJOR >=
  6
    DynamicJsonDocument json(1024);
  #else
    DynamicJsonBuffer jsonBuffer;
    JsonObject& json = jsonBuffer.createObject();
  #endif
  json["mqtt_server"] = mqtt_server;
  json["mqtt_port"] = mqtt_port;
  json["mqtt_username"] = mqtt_username;
  json["mqtt_password"] = mqtt_password;
  json["companyUUID"] = companyUUID;
  json["ip"] = SavedIP;
  json["gateway"] = SavedGateway;
  json["subnet"] = SavedSubnet;
  File configFile = SPIFFS.open("/park.json", "w");
  if (!configFile) {
    Serial.println("failed to open park file for writing");
  }
  #if defined(ARDUINOJSON_VERSION_MAJOR) && ARDUINOJSON_VERSION_MAJOR >=
  6
    serializeJson(json, Serial);
    serializeJson(json, configFile);
  #else
    json.printTo(Serial);
    json.printTo(configFile);
  #endif
  configFile.close();
}
```

- Gestion de l'OTA (Over-The-Air) et vérification du bouton

Cette partie gère les mises à jour OTA et vérifie l'état du bouton de réinitialisation.

```
ArduinoOTA.handle();
checkButton();
```

Figure99. Gestion de l'OTA (Over-The-Air) et vérification du bouton

- Gestion du portail web et du client MQTT

Le portail web et le client MQTT sont gérés ici. Si le client MQTT n'est pas connecté, il tente de se reconnecter.

```
wifiManager.startWebPortal();  
wifiManager.process();  
  
if (!client.connected()) {  
    reconnect();  
}  
client.loop();
```

Figure 100. Gestion du portail web et du client MQTT

- Gestion des messages d'accès

Si **accessMessage** est "open", cette section ouvrira la barrière 1, attendra un moment, puis la refermera. Ensuite, Elle réinitialisera **accessMessage**.

```
if (accessMessage == "open") {  
    Serial.println("B1OPEN OPEN");  
    digitalWrite(B1CLOSE, LOW);  
    digitalWrite(B1OPEN, HIGH);  
    delay(100);  
    digitalWrite(B1OPEN, LOW);  
    delay(5000);  
    Serial.println("B1OPEN CLOSED");  
    digitalWrite(B1CLOSE, HIGH);  
    delay(100);  
    digitalWrite(B1CLOSE, LOW);  
  
    Serial.println("Resetting access message...");  
    msg = "";  
    accessMessage = "";  
}
```

Figure 101. Gestion des messages d'accès

- Scan BLE et traitement des appareils trouvés

Cette section scanne les appareils BLE et traite les appareils trouvés. Si un appareil avec des données spécifiques est trouvé, il publie les informations sur MQTT et gère les actions des barrières en fonction des données reçues.

```
BLEScanResults foundDevices = pBLEScan->start(scanTime);
Serial.printf("\nScan done! Devices found: %d\n", foundDevices.getCount());
for (int i = 0; i < foundDevices.getCount(); i++) {
    BLEAdvertisedDevice device = foundDevices.getDevice(i);
    std::string strServiceData = device.getManufacturerData();
    uint8_t cServiceData[100];
    strServiceData.copy((char*)cServiceData, strServiceData.length(), 0);
    if (strServiceData.length() >= 3 && cServiceData[4] == 0xf8 && cServiceData[5] ==
companyUUID) {
        for (int i = 0; i < sizeof(cServiceData); ++i) {
            Serial.print(cServiceData[i]);
            Serial.print(" ");
        }

        Serial.println("//////////");
        String userID = String(cServiceData[6], HEX) + String(cServiceData[7], HEX);
        Serial.println(userID);
        cServiceData[4] = 0xf5;
        String deviceInfo = "{\"DeviceID\": \"WT32\", \"userID\": \"\" + userID + \"\"}";
        client.publish(auth_topic, deviceInfo.c_str());
        Serial.println("Published device information to MQTT: " + deviceInfo);

        switch (cServiceData[8]) {
            case 0:
                Serial.println("Barrier 1 Closed ");
                digitalWrite(B1OPEN, LOW);
                digitalWrite(B1CLOSE, HIGH);
                delay(1000);
                digitalWrite(B1CLOSE, LOW);

                break;
            case 1:
                Serial.println("Barrier 1 Opened ");

                digitalWrite(B1CLOSE, LOW);
                digitalWrite(B1OPEN, HIGH);
                delay(1000);
                digitalWrite(B1OPEN, LOW);
        }
    }
}
```

```
    actionInProgress1 = true;
    startTime1 = currentTime1;
    stopTimeS1 = cServiceData[12] * 1000;
    Serial.print(" Duration : ");
    Serial.println(stopTimeS1);
    break;
}

switch (cServiceData[9]) {
    case 0:
        Serial.println("Barrier 2 Closed ");
        digitalWrite(B2OPEN, LOW);
        digitalWrite(B2CLOSE, HIGH);
        delay(1000);
        digitalWrite(B2CLOSE, LOW);

        break;
    case 1:
        Serial.println("Barrier 2 Opened ");

        digitalWrite(B2CLOSE, LOW);
        digitalWrite(B2OPEN, HIGH);
        delay(100);
        digitalWrite(B2OPEN, LOW);

        actionInProgress2 = true;
        startTime2 = currentTime2;
        stopTimeS2 = cServiceData[13] * 1000;
        Serial.print(" Duration : ");
        Serial.println(stopTimeS2);
        break;
}

switch (cServiceData[10]) {
    case 0:
        Serial.println("Barrier 3 Closed ");
        digitalWrite(B3OPEN, LOW);
        digitalWrite(B3CLOSE, HIGH);
        delay(1000);
        digitalWrite(B3CLOSE, LOW);

        break;
    case 1:
        Serial.println("Barrier 3 Opened ");

        digitalWrite(B3CLOSE, LOW);
        digitalWrite(B3OPEN, HIGH);
        delay(100);
```

```

    digitalWrite(B3OPEN, LOW);

    actionInProgress3 = true;
    startTime3 = currentTime3;
    stopTimeS3 = cServiceData[14] * 1000;
    Serial.print(" Duration : ");
    Serial.println(stopTimeS3);
    break;
}

switch (cServiceData[11]) {
    case 0:
        Serial.println("Barrier 4 Closed ");
        digitalWrite(B4OPEN, LOW);
        digitalWrite(B4CLOSE, HIGH);
        delay(1000);
        digitalWrite(B4CLOSE, LOW);
        break;
    case 1:
        Serial.println("Barrier 4 Opened ");

        digitalWrite(B4CLOSE, LOW);
        digitalWrite(B4OPEN, HIGH);
        delay(100);
        digitalWrite(B4OPEN, LOW);

        actionInProgress4 = true;
        startTime4 = currentTime4;
        stopTimeS4 = cServiceData[15] * 1000;
        Serial.print(" Duration : ");
        Serial.println(stopTimeS4);
        break;
}
stopScan();
}
}

```

Figure 102. Scan BLE et traitement des appareils trouvés

- Sauvegarde de la configuration (callback)

Appel de la fonction **saveConfigCallback** pour sauvegarder la configuration.

```
saveConfigCallback();
```

Figure103. Sauvegarde de la configuration (callback)

6. Sprint Review

Notre sprint de contrôleur de barrière a été productif. Nous avons développé des fonctionnalités clés telles que la détection automatique des véhicules et une interface utilisateur améliorée. Malgré quelques difficultés, notre équipe a réussi à avancer efficacement. Les retours positifs des parties prenantes nous motivent à continuer sur cette lancée pour le prochain sprint.

7. Perspective

Dans le futur, nous prévoyons d'ajouter une deuxième barrière pour la sortie des véhicules. Le code actuel est déjà prêt pour cette expansion, avec quatre relais disponibles, deux pour la première barrière et deux pour la deuxième, permettant ainsi une mise en œuvre facile et rapide de cette fonctionnalité supplémentaire.

De plus, Pour améliorer notre système de contrôle des barrières de parking, nous envisageons d'intégrer des capteurs de boucle inductive enterrés dans le sol pour détecter la présence des véhicules. Actuellement, notre système utilise des temporisateurs pour ouvrir et fermer les barrières, mais cette approche peut être optimisée en utilisant des capteurs plus précis.

Les capteurs de boucle inductive, placés sous la chaussée à des emplacements stratégiques, détecteront la présence de véhicules en temps réel en mesurant les perturbations du champ magnétique terrestre causées par les masses métalliques des voitures. Une fois un véhicule détecté, le capteur enverra un signal au système de contrôle des barrières pour ouvrir ou fermer la barrière automatiquement, assurant ainsi une gestion plus fluide et sécurisée du flux de véhicules.

Cette amélioration permettra de réduire les temps d'attente et d'augmenter l'efficacité du système, tout en minimisant les erreurs liées à l'ouverture et à la fermeture des barrières. En intégrant cette technologie, nous visons à rendre notre solution plus intelligente et plus réactive aux besoins réels des utilisateurs.

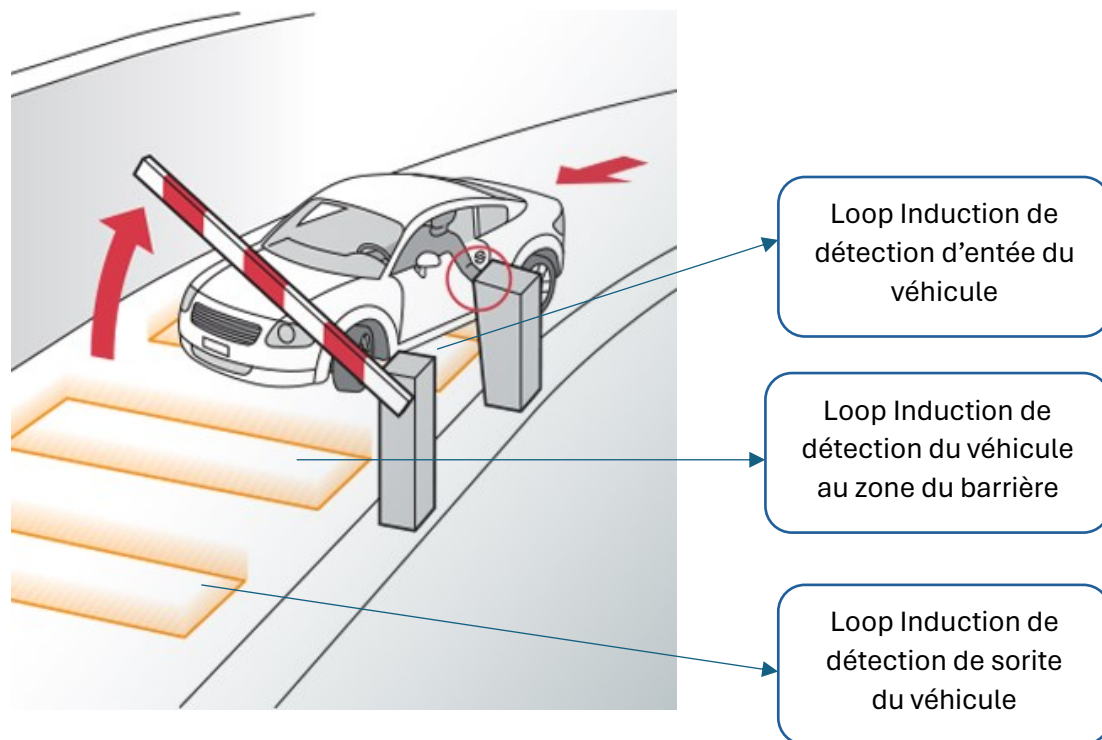


Figure104. Système de contrôle des barrières de parking

8. Conclusion

En somme, l'assemblage et la programmation du microcontrôleur WT32 ETH01 ainsi que l'intégration des différents composants matériels et logiciels démontrent l'ingéniosité et la rigueur technique de ce projet. La gestion des configurations via MQTT, la réactivité aux signaux BLE et la sécurisation des accès par TCP illustrent la polyvalence et la sophistication de cette solution de contrôle d'accès. Avec une attention portée aux détails et une optimisation du code Arduino, cette étape cruciale assure le bon fonctionnement et la fiabilité du système dans divers environnements.

CHAPITRE 6

SPRINT 4 : MACROPARK

LICENCE PLATE RECOGNIZER

1. Introduction

Ce chapitre décrit les différentes étapes de la réalisation du quatrième sprint. Nous commencerons par présenter le backlog de ce sprint, suivie par l'analyse et la conception. Enfin, nous détaillerons les différentes interfaces développées.

2. Backlog sprint 4

Selon la planification de notre méthodologie, le 4^{ème} sprint est principalement sur le cas d'utilisation « **Intégration de la caméra de reconnaissance des immatriculations** » que nous allons décortiquer par la suite.

Tableau 26. Planification de sprint 4

Identifiant	Acteur	Tâche	Estimation
Identification du matériel	Caméra	T1 : Analyse T2 : Conception T3 : Codage	T1 : 01 H T2 : 10 H
Configuration du caméra			T1 : 01 H T2 : 10 H
Serveur Caméra (Bridge)			T1 : 03 H T2 : 02 H T3 : 15 H
Serveur Principale (Core Server)			T1 : 03 H T2 : 02 H T3 : 23 H
Communication			T1 : 03 H T2 : 02 H T3 : 25 H

3. Implémentation et technologie utilisé

Nous commençons par configurer notre environnement de développement, en sélectionnant les outils nécessaires pour implémenter la caméra et les serveurs liées de manière efficace et structurée.

Voici un aperçu de chaque élément utilisé :

- **Python** : Langage de programmation utilisé pour écrire la logique côté serveur.
- **Uvicorn** : Serveur ASGI rapide et léger pour exécuter les applications Python.

MQTT : MQTT est un protocole de messagerie publish-subscribe basé sur le protocole TCP/IP.

Docker : Outil de virtualisation pour créer des conteneurs légers et portables pour les applications afin de l'exécuter dans différentes machines.

Cette architecture est équilibrée et tire parti des technologies modernes pour développer notre serveur de caméra et le serveur central, garantissant une reconnaissance des plaques d'immatriculation organisée et efficace.

4. Description détaillée du sprint 4

Dans ce point, nous avons précisé comment fonctionne notre système avec une description plus détaillée.

4.1. Description 'Identification du matériel'

- SmartLPR® Access [8] est une caméra utilisée dans la reconnaissance de plaque d'immatriculation (LPR) système spécialement conçu pour être utilisé dans les voies d'accès.
- SmartLPR® Access [8] est composé d'un seul ensemble de matériel qui doit être installé dans chaque voie qu'on veut contrôler.

4.1.1. Caractéristiques

Tableau 27. Les caractéristiques

Caractéristiques	Détail
Modèle Caméra	Quercus SmartLPR®[8]
Version du Software	4.5.x
Version du firmware	4.5.x
Type de communication	TCP / IP, Cable RJ45



Figures 105. Modèle Caméra

4.1.2. Installation

- Connexion au réseaux :

Unscrew the back side of the cable boot.	
Pass the cables through the cable glands and through the boot.	
Crimp the RJ-45 connector.	
Place the RJ-45 connector inside the boot and close it.  Please note: the boot is essential in order to assure the leak tightness of the unit.	

Figure106. câble du réseau

- Connexion vers un source de 220 V :

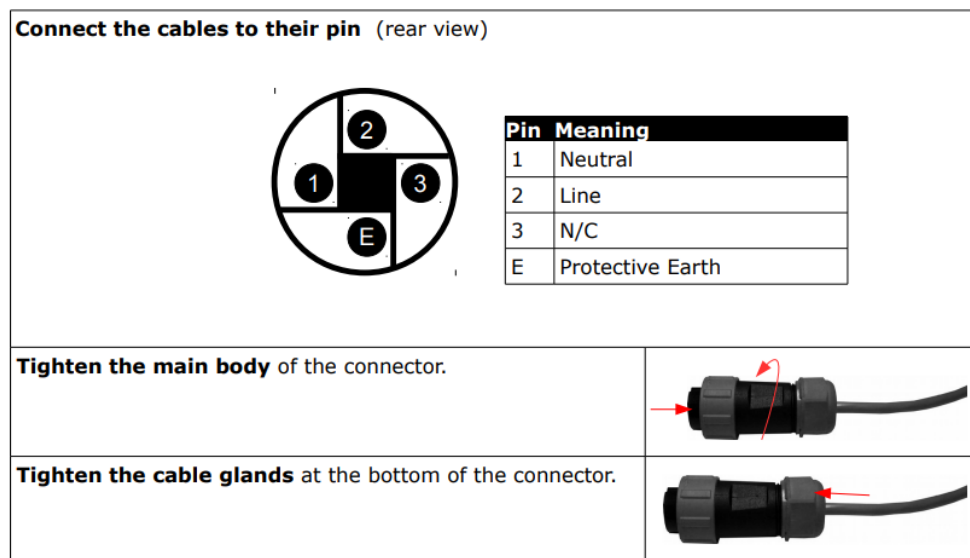


Figure107. Connexion câble d'alimentation vers une source 220 V

- Connecter les pins d'entrées & sorties :

Pin	Meaning
1	IN1
2	IN2
3	OUT1
4	+ 12V
5	GND
6	Wiegand D0
7	Wiegand D1

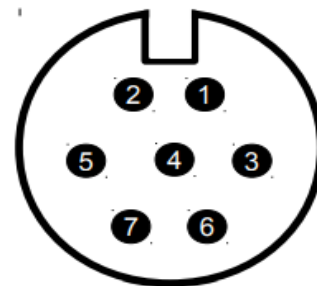


Figure 108. Pins d'entrées & sorties

- Système d'entrée / Sortie :

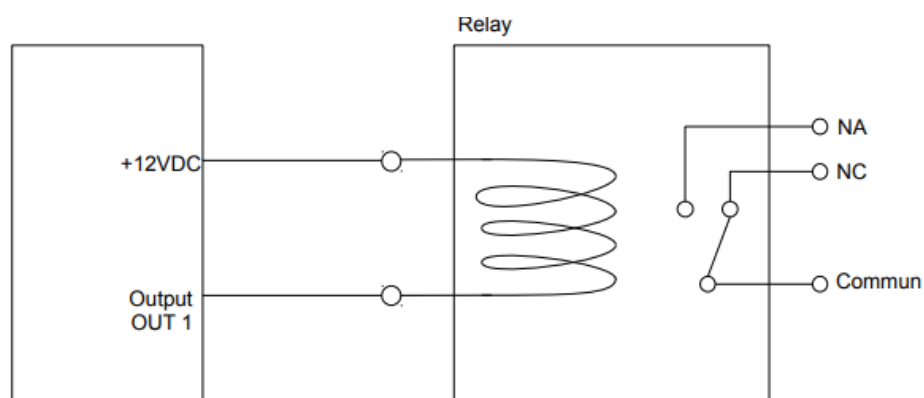
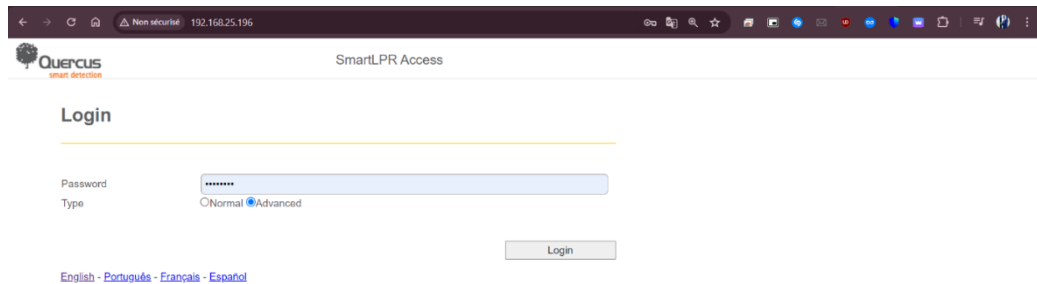


Figure 109. Système d'entrée / Sortie

4.1.3. Description 'Configuration du Caméra '

- L'accès à l'espace SmartLPR Access



SmartLPR Access

Quercus smart detection

Login

Password: [password field]

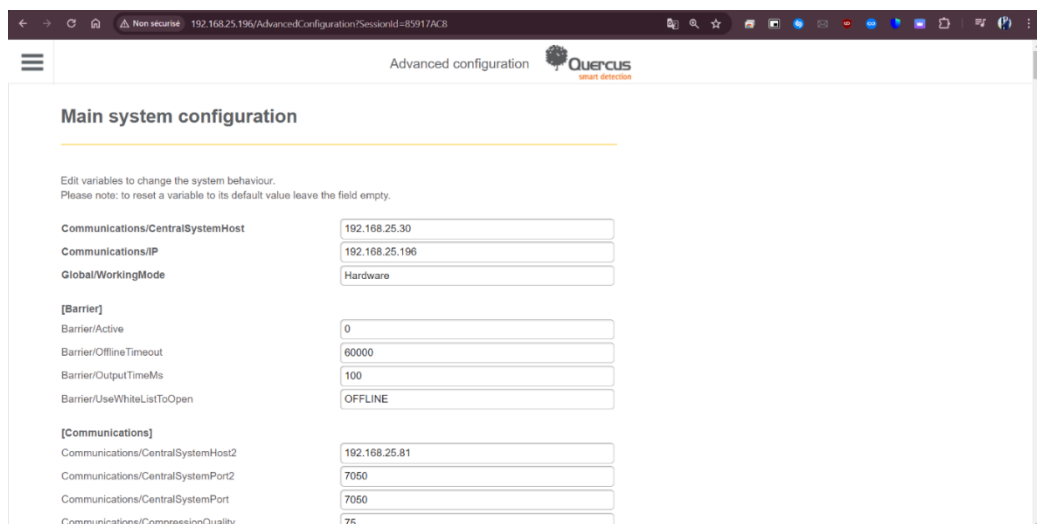
Type: ☐ Normal ☒ Advanced

Login

English - Português - Français - Español

Figure 110. Configuration camera

- Accès au page de la configuration :



Advanced configuration

Quercus smart detection

Main system configuration

Edit variables to change the system behaviour.
Please note: to reset a variable to its default value leave the field empty.

Communications/CentralSystemHost	192.168.25.30
Communications/IP	192.168.25.196
Global/WorkingMode	Hardware
[Barrier]	
Barrier/Active	0
Barrier/OfflineTimeout	60000
Barrier/OutputTimeMs	100
Barrier/UseWhiteListToOpen	OFFLINE
[Communications]	
Communications/CentralSystemHost2	192.168.25.81
Communications/CentralSystemPort2	7060
Communications/CentralSystemPort	7060
Communications/CompressionQuality	75

Figure 111. Accès au page de la configuration

Dans cette section, nous allons détailler les étapes de configuration de notre caméra pour une reconnaissance optimale des plaques d'immatriculation tunisiennes.

Voici les éléments clés à configurer :

- Configurer le central host :

Définir l'adresse IP de notre machine ou serveur qui agira comme hôte central.

Communications/CentralSystemHost 192.168.25.30

Figure 112. Configurer le central host

- Configurer l'interface de configuration :

Spécifier l'adresse IP de l'interface utilisée pour la configuration de la caméra.

Communications/IP

192.168.25.196

Figure 113. Configurer l'interface de configuration

- Reconnaissance des plaques d'immatriculation tunisiennes :

Activer et paramétrer la reconnaissance pour les plaques d'immatriculation tunisiennes.

Recognizer/Regions/Tunisia

1

Recognizer/Regions/Tunisia_Specials

2

Figure 114. Reconnaissance des plaques d'immatriculation tunisiennes

- Configurer l'adresse IP et le port de l'API REST :

Définir l'adresse IP et le port vers lesquels la caméra envoie les données capturées.

[RESTInterface]

RESTInterface/BasicAuthenticationPassword

quercus2

RESTInterface/BasicAuthenticationUser

admin

RESTInterface/CentralBaseURI

http://192.168.25.30:8180

RESTInterface/IncludeImageOnRecognition

1

RESTInterface/Port

8180

Figure 115. Configurer l'adresse IP et le port de l'API REST

En suivant ces étapes et en utilisant les captures d'écran pour référence, nous pouvons configurer efficacement notre caméra pour la reconnaissance des plaques d'immatriculation tunisiennes et assurer une transmission fluide des données capturées vers notre serveur.

4.1.4. Description 'Configuration du contrôleur de la caméra (BRIDGE) :

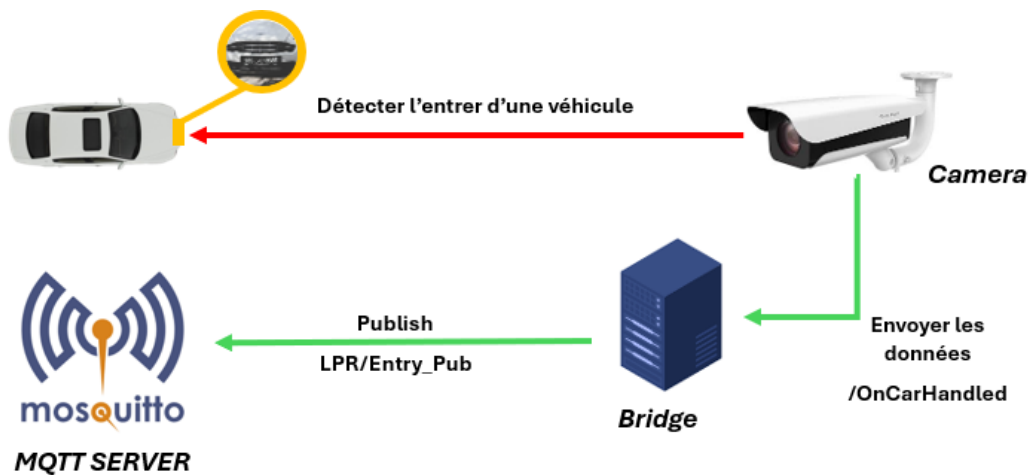


Figure 116. Description 'Configuration du contrôleur de la caméra (BRIDGE)

Le contrôleur de la caméra, connu sous le nom de BRIDGE, assume un rôle central dans notre système en surveillant en permanence la caméra pour toute action déclenchante.

Les actions fournies par la caméra, sont :

- /OnCarHandled
- /OnCarDeparture
- /OnCarArrival

INFO:	192.168.25.196:2681 - "POST /OnCarHandled HTTP/1.1" 200 OK
INFO:	192.168.25.196:2682 - "POST /OnCarDeparture HTTP/1.1" 404 Not Found
INFO:	192.168.25.196:2217 - "POST /OnCarArrival HTTP/1.1" 404 Not Found

Ces actions utilisent une interface REST API avec une méthode POST pour transmettre les données au BRIDGE.

Après l'analyse des exigences de notre projet nous avons conclu qu'on a besoin que l'action **OnCarHandled** donc on l'a choisie et préparé la structure de celui-ci.

INFO :	192.168.25.196:2681 - "POST /OnCarHandled HTTP/1.1" 200 OK
--------	--

Lorsque l'action **"/oncarhandled"** est déclenchée, signifiant que le capteur associé à la caméra détecte un mouvement devant l'espace d'entrée, le BRIDGE reçoit les données de la caméra via la méthode POST et les soumet à une vérification de qualité d'image en utilisant la variable **"minQuality"**.

Cette fonction de vérification de la qualité d'image est essentielle si la qualité minimale est en dessous de 0.5 (**minQuality<0.5**), le processus de reconnaissance est répété à l'aide de l'API (/WS/Trigger?triggerId) cette dernière utilise aussi une méthode POST avec une authentification basique basant sur un Username est mot de passe fournis par le fabricant de la caméra jusqu'à ce que la qualité de l'image dépasse 0.5.

Ceci garantit que seules les images de qualité suffisante sont utilisées pour la reconnaissance.

```
On Car Handled ----- <-- POST-->
- - - - - LPN = 138TN4516

Connected to MQTT : 192.168.25.30
Published Topic : LPR/Entry_Pub
Published message : {"license": "138TN4516", "minQuality": 0.93, "avgQuality": 0.96, "triggerId": 0, "Device_id": "Entry01", "image": "/9j//gIHU01H
PTI2RUVEQkNFNDcyRTAwN0Q3MDE3NkQ0NEFGNzE4ODg1QkU5NTAyNzhBNzVG0jVFRkex0jA0NUM1NDQ3Nk1GMzFDM0QzNUY2NUY5OTkwRDA2QzKxRkEyNTEzNkYwMjc4RjExOUAMjVCQzNEQzR
EM0ZDNEM2MEFDNzBDNjI2QkVBTk1NzK4N0UxRDQwQzZBNzBDQjKwN0E0RD1BQjczMkQxMUy0QjM3MTRFQzKzOUJCRCk0MUMwNDdBRDI0NzY0ODJGMDkzODE1MkQ0QjREQThBRENDNU
MzNDk3ODM5N0NDM0Q4OEuwODYwOTM0MjgyNDk0QzAwMEZCQUI0MDM3OUU3NEQ00EQxNTVCODM2Mjc4xMD1BNzY50EIZMDMzQjA2MERERjk3MDg40EM4Q0M4OUU3RDI4NjBDQTDQUU5NUYxM0Ex0
```

Figure 117. Résultat du bridge

Lorsque la qualité minimale dépasse 0.5, cela indique une reconnaissance réussie. À ce stade, diverses données sont transmises sur un canal spécifique topic (Publisher – Publier) "**LPR/Entry_Pub**", via un serveur Mosquitto[10] (**broker**) installé localement avec l'adresse ip de la machine et un port par défaut 1883.

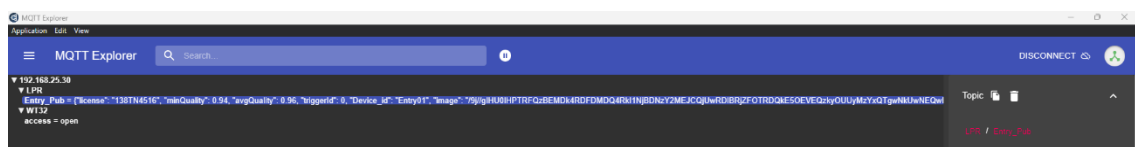


Figure 118. Test par MQTT Explorer

Ces données sont structurées au format JSON et comprennent :

- **"license"** : l'immatriculation identifiée dans la demande.
- **"minQuality"** : la qualité minimale de l'image.
- **"avgQuality"** : la qualité moyenne de l'image.
- **"triggerId"** : l'identifiant du déclencheur de l'événement.
- **"Device_id"** : l'ID de la caméra associée.
- **"image"** : les données de l'image, encodées en base64 pour l'efficacité du transfert et la taille réduite.

Cette organisation des données permet d'une transmission claire et ordonnée des informations, simplifiant ainsi la gestion et le traitement des événements de reconnaissance des plaques d'immatriculation (LPR).

4.1.5. Description 'Configuration du serveur principale (CORE SERVER) :

Le rôle principal du CoreServer est d'établir une communication fiable entre le Bridge (le serveur lié à la caméra de reconnaissance des immatriculations) et le Backend (le serveur qui contient les données des utilisateurs) pour la vérification. De plus, nous pouvons augmenter le nombre de caméras dans le parking pour assurer une communication efficace et éviter les conflits de données.

```
{'userID': '6638c73bd235e4d3dd6e2098', 'name': 'Yassine Manai', 'appID': 0}
Connected to MQTT : 192.168.25.30
Published Topic : WT32/access
Published message : open
{'userID': '6638c73bd235e4d3dd6e2098', 'user_name': 'Yassine Manai', 'user_app': 0, 'Method': 'Camera Action', 'License': '138TN4516', 'deviceId': 'Entry01', 'action': 'enter', 'date': '2024-05-28', 'time': '09:23:13', 'imageData': '/9j/gIHU0LHPTIzQTKxRDNGNky1RUiWnTHGNI4NTNGMjdBmUYxNzdGNORGO
DlGOTREOEY50dkwM0YzMKJDMTRC0DLDMjcxMTVERDg0RTNFMjBE0DVBmJE3QkJGMEZEM0RBRDcz0zdFMkEzRTE0Njh0j1FRjYzNkEyNTLCMUU1NkMxMjFCRTAy0jddMENGNDk2QzZDN0FDNDg0
NTH0QkFCMEU1Mjcw0Q0z0TY0NEUyM0U2NzRC0Dg2MEJF0kU2MTNGNUIz0BRFQjKz00VEH2ZBRE0508UwQkY3RDHBMUJz0UM2Qk1x0jMyMUJEX0jgz0z0z0MjK2NU040zE3NDg1MjEyNDQzNDhG0Tg
```

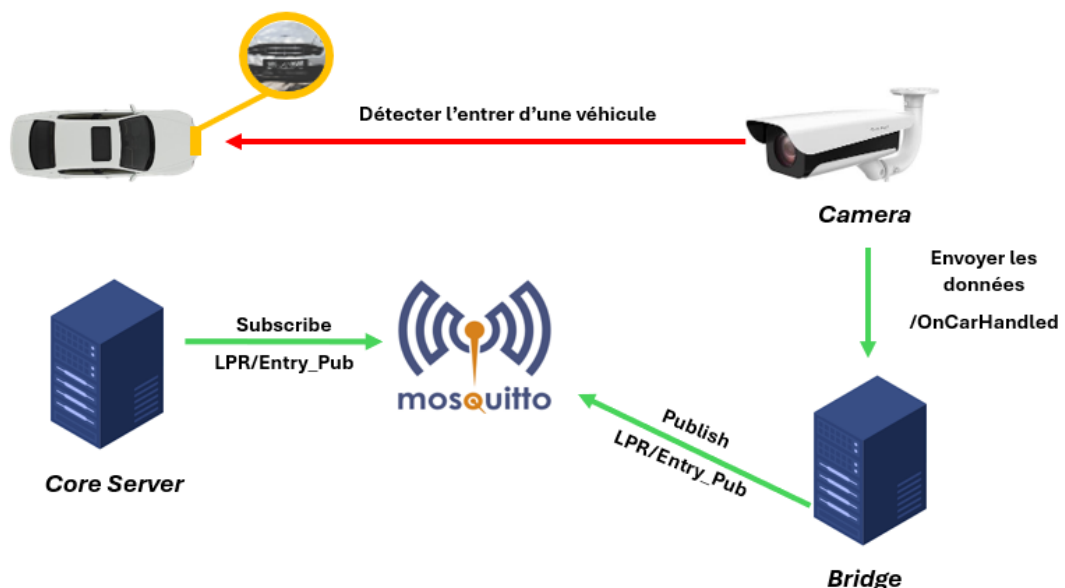


Figure119. CoreServer

Le serveur principal, appelé **CORE SERVER**, est un contrôleur central qui écoute en permanence un topic Mosquitto (Subscribe - abonné) nommé **"LPR/Entry_Pub"**.

Lorsque des données sont publiées sur ce topic depuis le contrôleur du caméra (BRIDGE), le serveur reçoit le **payload** (message), contenant la plaque d'immatriculation (Licence Plate Number) et on utilise une fonction de vérification pour déterminer l'existence d'immatriculation dans notre base de données.

Si l'immatriculation est trouvée, le CORE SERVER renvoie une réponse 200 (indique une réponse réussite) incluant l'identifiant et le nom d'utilisateur associés. Ensuite, un message **"open"** est publié sur un autre topic nommé **"WT32/access"** pour ouvrir la barrière.

Simultanément, une fonction appelée **"create_event()"** est exécutée pour enregistrer l'historique d'entrée au parking de cet utilisateur.

Les données enregistrées comprennent :

- **"id_user"** : l'identifiant de l'utilisateur.
- **"user_name"** : le nom de l'utilisateur.
- **"user_email"** : l'email de l'utilisateur.
- **"MQTT Server"** : le serveur de courtage.
- **"MQTT Topic"** : le topic de publication.
- **"Methode"** : "Camera Action".
- **"license"** : la plaque d'immatriculation.
- **"minQuality"** : la qualité minimale de l'image (par défaut 0.5).
- **"avgQuality"** : la qualité moyenne de l'image (par défaut 1).
- **"triggerId"** : l'identifiant du déclencheur de l'événement.
- **"deviceId"** : l'identifiant de l'appareil.
- **"date"** : la date actuelle.
- **"time"** : l'heure actuelle.
- **"imageData"** : les données de l'image capturé (encodée en base64).

Ce processus permet l'ouverture automatique de la barrière à l'aide de la caméra.

En cas d'ouverture de la barrière via un téléphone, un message est publié par le contrôleur de la barrière sur un topic **"WT32/auth"**. Le CORE SERVER, est abonné à ce topic, reçoit les données au format JSON contenant l'identifiant de l'appareil (deviceId) et l'identifiant de l'utilisateur. À la réception de ces données, une fonction appelée **"create_event()"** est déclenchée pour enregistrer les informations dans la base de données.

Les données enregistrées dans ce cas incluent :

- **"id_user"** : l'identifiant de l'utilisateur.
- **"user_name"** : le nom de l'utilisateur.
- **"user_email"** : l'email de l'utilisateur.
- **"MQTT Server"** : le serveur de courtage.
- **"MQTT Topic"** : le topic de publication.
- **"Methode"** : "Phone Action".
- **"license"** : "None".
- **"minQuality"** : "0".
- **"deviceId"** : l'identifiant de l'appareil.
- **"date"** : la date actuelle.
- **"time"** : l'heure actuelle.

Ce processus garantit que chaque ouverture de barrière, qu'elle soit effectuée via la caméra ou le téléphone, est soigneusement enregistrée dans la base de données, permettant un suivi complet et détaillé des accès au parking. Le CORE SERVER joue ainsi un rôle vital en assurant le bon fonctionnement et la sécurité du notre système de gestion du parking.

4.2. Description 'Communication'

4.2.1. Communication entre la caméra et Bridge

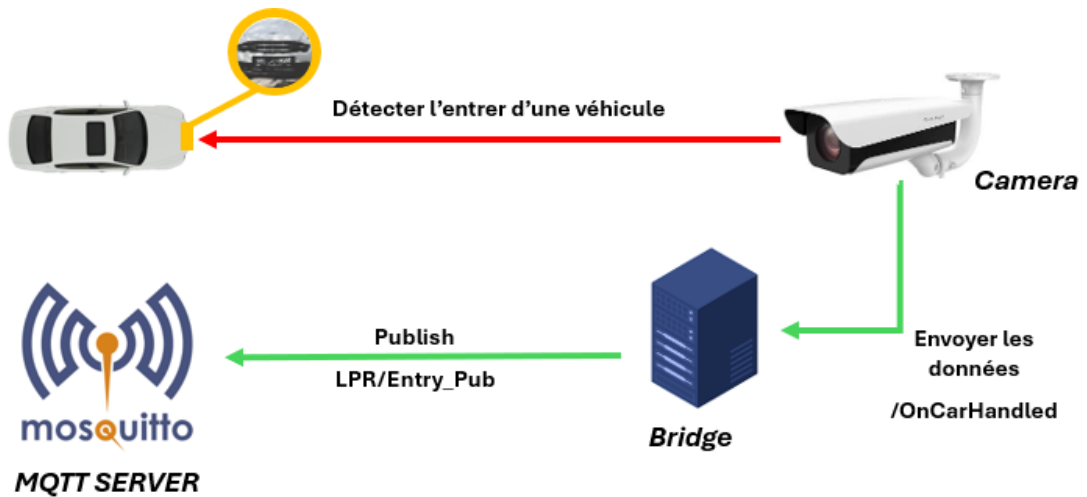


Figure 120. Entre la caméra et Bridge

4.2.2 Communication entre le CoreServer et le Bridge

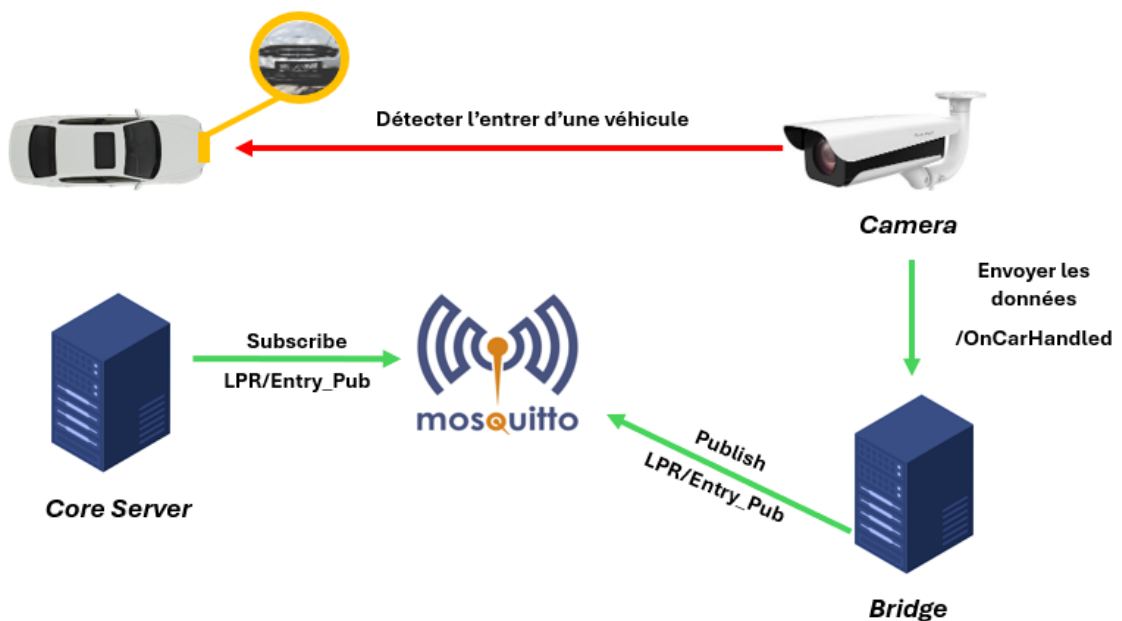


Figure 121. Entre CoreServer et le Bridge

4.2.3 Communication entre le CoreServer et le Contrôleur (Backend)

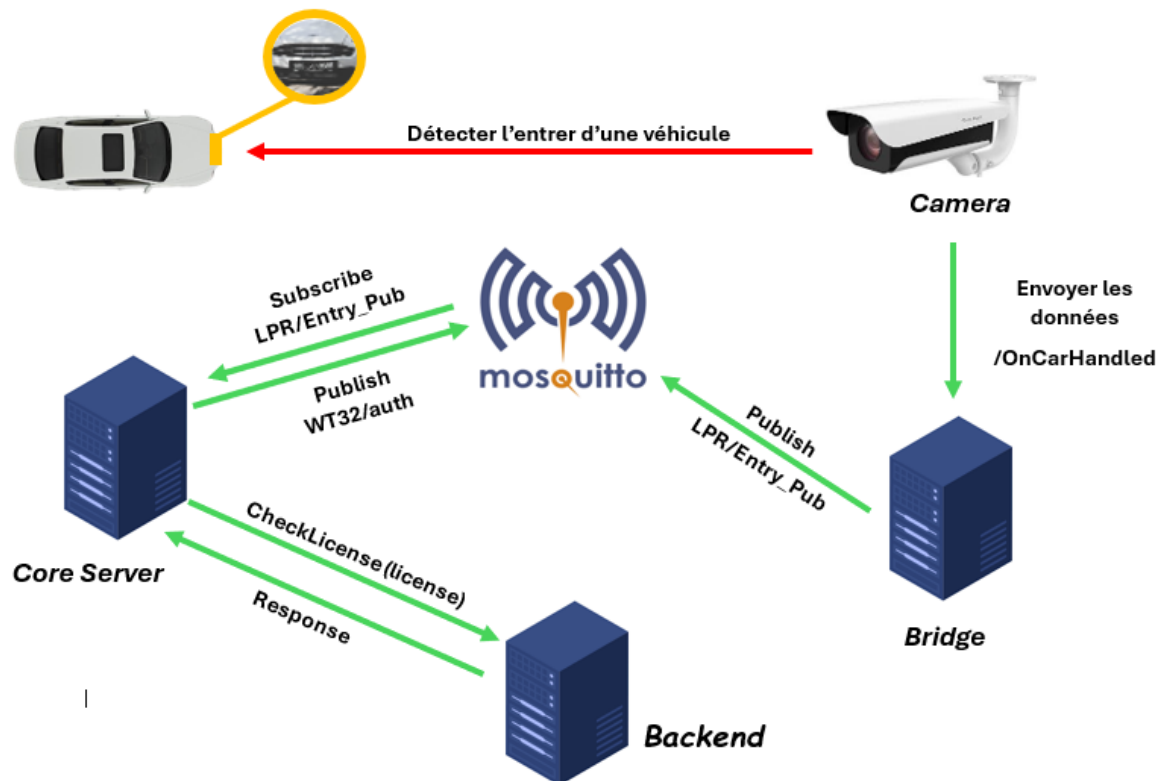


Figure 122. Entre CoreServer et le contrôleur

4.2.4 communication entre le CoreServer et le contrôleur de la barrière

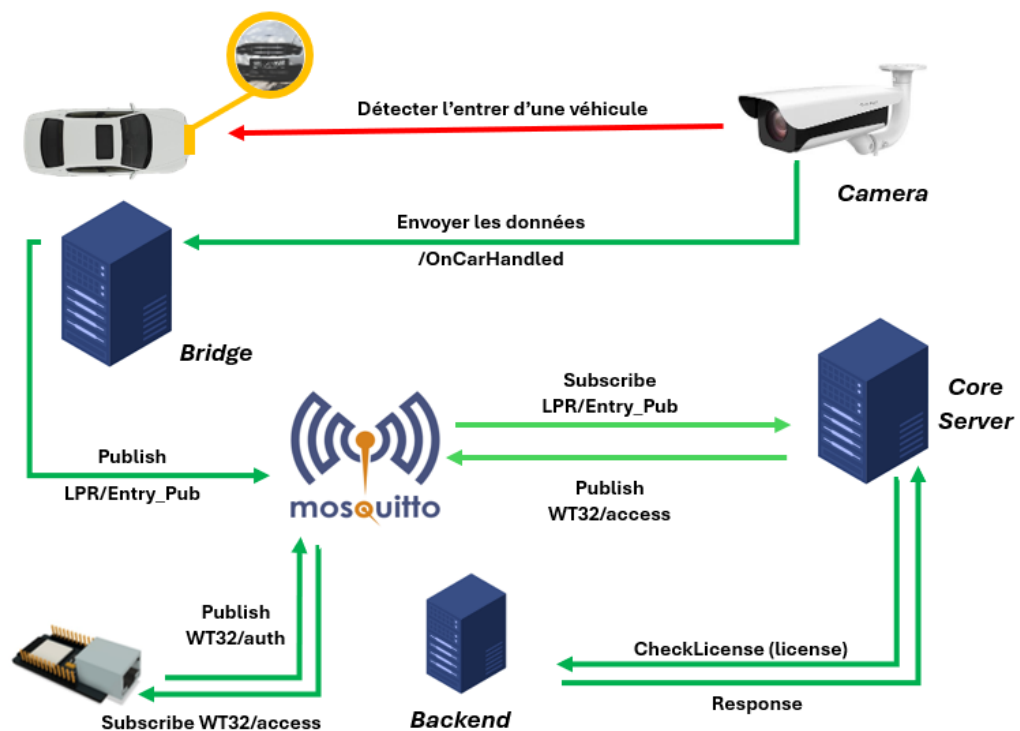


Figure 123. Entre CoreServer et le contrôleur de la barrière

5 Sprint Review

Dans ce sprint, nous avons intégré avec succès la caméra, le bridge et le Core Server. La caméra est configurée pour la reconnaissance des plaques d'immatriculation, avec des fonctionnalités backend et une interface frontend développées. Le bridge facilite la communication entre la caméra et le Core Server, avec une conception backend et une interface utilisateur. Le Core Server a été amélioré pour le traitement et le stockage des données, avec des optimisations pour la reconnaissance, la sécurité et les performances.

6 Perspectives

❖ **Optimisation de la Reconnaissance** : Nous envisageons d'améliorer encore la précision de la reconnaissance des plaques d'immatriculation en optimisant les algorithmes et en intégrant de nouvelles techniques de traitement d'image.

❖ **Évolutivité du Système** : Nous planifions d'assurer une évolutivité maximale du système en prévoyant des stratégies d'extension pour gérer efficacement une augmentation du nombre de caméras et de connexions au serveur.

❖ **Sécurité Renforcée** : Nous nous engageons à renforcer davantage la sécurité du système en mettant en œuvre des mesures supplémentaires pour protéger les données des utilisateurs et garantir l'intégrité des transactions.

❖ **Amélioration de l'Interface Utilisateur** : Nous avons l'intention d'améliorer l'interface utilisateur pour une expérience plus fluide et intuitive, en tenant compte des retours des utilisateurs et des nouvelles tendances en matière de conception.

7 Conclusion

Dans ce Sprint, nous avons réussi à configurer notre caméra pour la reconnaissance des immatriculations. Nous avons débuté en élaborant le Backlog de ce sprint, suivi par l'analyse et la présentation des technologies utilisées. Enfin, nous avons décrit les différentes phases du sprint.

CHAPITRE 7

SPRINT 5: MACROPARK

TEST & VALIDATION

1. Introduction :

Ce chapitre se concentre sur les tests, essentiels pour garantir la qualité et la fiabilité du produit.

Nous détaillons les stratégies de test mises en œuvre : tests unitaires, d'intégration et de système. Nous décrivons également les outils et processus de validation utilisés. Enfin, nous présentons les résultats des tests et les corrections apportées, affirmant notre engagement à fournir une solution robuste et sans défauts.

2. Backlog sprint 5

Selon la planification de notre méthodologie, le cinquième sprint est principalement sur le thème « **Test & Validation** » que nous allons décortiquer par la suite.

Tableau 28. Planification de sprint5

Identifiant	Acteur	Tâche	Estimation
Interface Web Admin	Testeur / Développeur	T1 : Test T2 : Correction des erreurs	T1 : 08 H T2 : 15 H
Application Mobile			T1 : 05H T2 :10 H
Intégration de la caméra			T1 : 05H T2 : 18 H
Contrôleur de la barrière			T1 : 04 H T2 : 25 H

3. Description des outils utilisés

- Utilisation de **Swagger UI** [6] pour le test des APIs :

Pour assurer la fiabilité et l'efficacité de nos API, nous avons utilisé Swagger UI, un outil interactif et convivial pour tester et documenter les APIs RESTful. Swagger UI permet aux développeurs et aux testeurs de visualiser, d'explorer et de tester facilement les endpoints d'API directement depuis un navigateur.

Voici une description détaillée de l'utilisation de SwaggerUI :

- **Accès à Swagger UI :**

Swagger UI est généralement hébergé avec l'application backend. Pour y accéder, il suffit de naviguer vers l'URL spécifique de Swagger (par exemple, <http://127.0.0.1:8200/docs>).

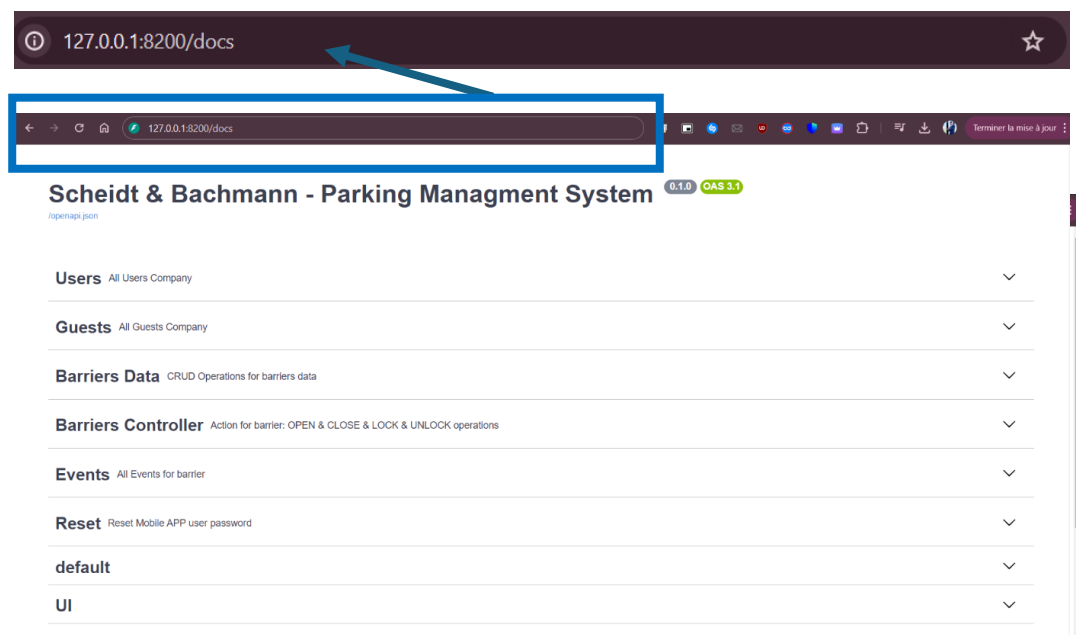


Figure 124 : Swagger UI

Swagger UI affiche tous les apis disponibles, organisés par catégorie. Chaque endpoint est accompagné de sa méthode HTTP (GET, POST, PUT, DELETE) et d'une courte description.

- **Détails des Endpoints :**

En cliquant sur une api spécifique, Swagger UI affiche des détails supplémentaires tels que les paramètres requis, les schémas de requêtes et de réponses, ainsi que des exemples de payloads.

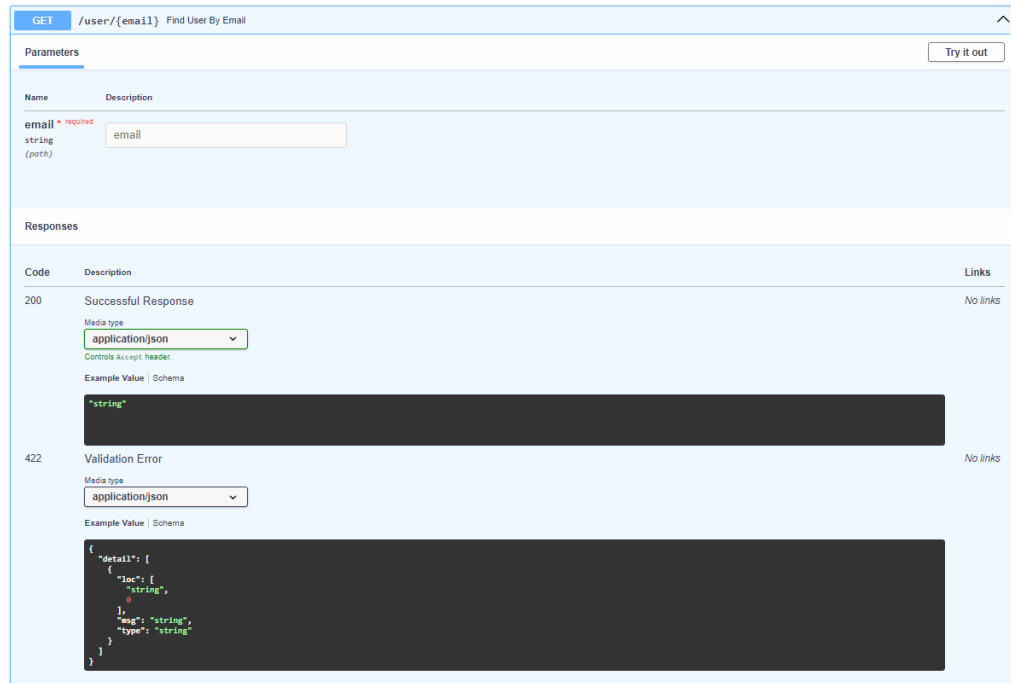


Figure 125. Détail de Swagger UI

- **Exécution des Requêtes**

Pour tester un endpoint, les utilisateurs peuvent entrer les valeurs nécessaires dans les champs fournis et cliquer sur le bouton "Try it out".

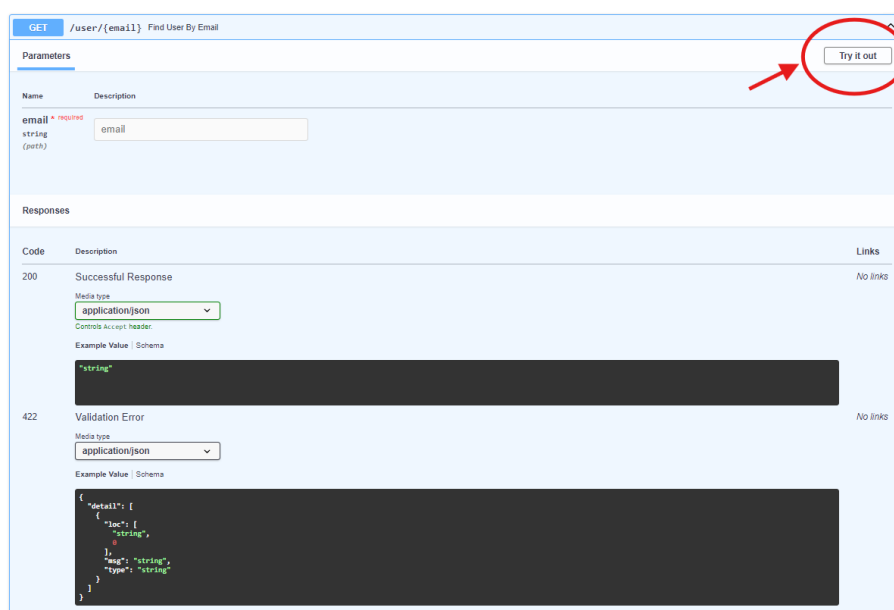


Figure 126. Exécution des APIs

Swagger UI envoie alors une requête HTTP à l'API et affiche la réponse en temps réel, y compris le code de statut, les en-têtes et le corps de la réponse.

○ **Analyse des Résultats :**

Les résultats des tests sont affichés directement sous le formulaire de requête, ce qui permet aux testeurs de vérifier immédiatement si l'API fonctionne comme prévu. Les informations d'erreur détaillées aident à diagnostiquer et à corriger les problèmes.

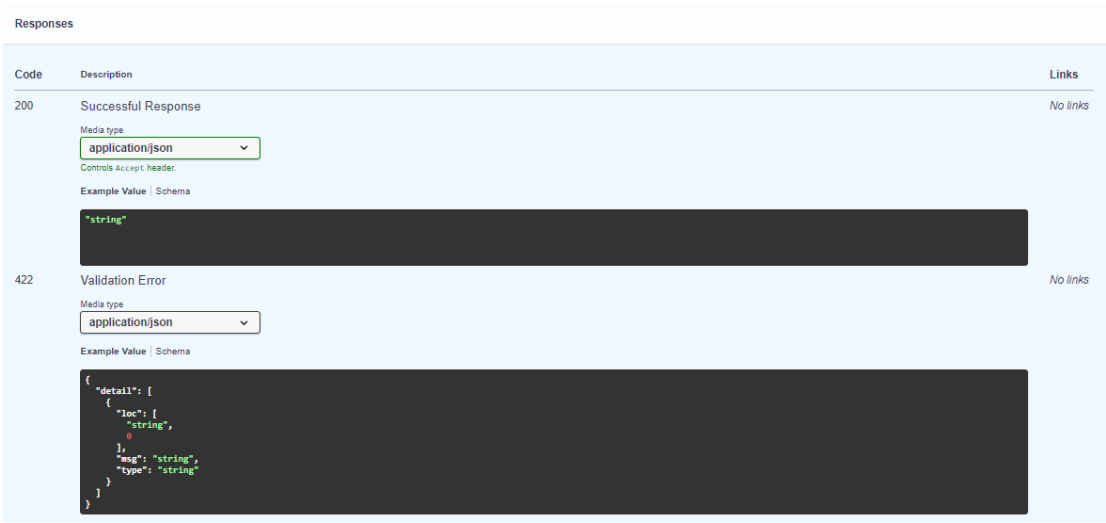


Figure 127. Modèle Response de Swagger

○ **Documentation Interactive :**

En plus du test des APIs, Swagger UI sert de documentation interactive pour les développeurs. La clarté et la structure des informations facilitent la compréhension de chaque endpoint et de son utilisation.

Cela réduit également les erreurs de communication et garantit que toutes les parties prenantes disposent d'une source de vérité commune.

4. Description détaillée du sprint 5

Dans cette section, nous avons examiné en détail les tests effectués, en fournissant une analyse approfondie des essais menés pour évaluer ses performances.

4.1 Répartition du notre APIs

- Users :

Users All Users Company ^		
GET	/user/{email}	Find User By Email
GET	/userid/{id}	Find User By Id
GET	/allusers	Get All Users
POST	/signin	Sign In
PUT	/modifyUser/{email}	Update User
PUT	/modifyWeb/{id}	Update User Id
DELETE	/deleteUser/{user_id}	Delete User

Figure 128. Répartition du notre APIs : Users

- Guests :

Guests All Guests Company ^		
POST	/addGuest	Addguest
GET	/allguests	Get All Guests
DELETE	/delete_guest/{email}	Delete Guest
GET	/guest/{email}	Find Guest By Email
GET	/guestid/{id}	Find Guest By Id
GET	/approve_guest/{email}	Approve User In Progress

Figure 129. Répartition du notre APIs :Guests

- Barriers Data :

Barriers Data CRUD Operations for barriers data ^		
POST	/add	Add Barrier
GET	/Barrier/{id}	Get Barrier By Id
GET	/Barrier	Get All Barriers
PUT	/modify/{barrier_id}	Modify Barrier
DELETE	/delete/{barrier_id}	Delete Barrier

Figure 130. Répartition du notre APIs : Barriers Data

- **Barriers Controller :**

Barriers Controller Action for barrier: OPEN & CLOSE & LOCK & UNLOCK operations			^
POST	/open/{id}	Open Barrier By Id	▼
POST	/close/{id}	Close Barrier By Id	▼
POST	/lock/{id}	Lock Barrier By Id	▼
POST	/unlock/{id}	Unlock Barrier By Id	▼
GET	/status/{id}	Get Status	▼

Figure 131. Répartition du notre APIs : Barriers

- **Events :**

Events All Events for barrier			^
GET	/Event	Get All Events	▼
GET	/Event/{id}	Get All Events	▼
DELETE	/deleteEvent/{barrier_id}	Delete Event	▼
DELETE	/deleteEvent	Delete All Events	▼
GET	/alluserevents	Get All Userevents	▼
GET	/usersLogs/{date}	Find Users By Date	▼
GET	/usersoid/{id}	Find Users Log By Id	▼
GET	/userappid/{id}	Find Users By Date	▼

Figure 132. Répartition du notre APIs : Events

4.2 Tester les différents API :

Dans cette partie nous avons choisis quelque API de chaque catégorie pour les tester :

- **Ajouter un utilisateur :**

POST	/adduser	Adduser	⌵
Parameters			
No parameters			
Request body required			application/json ▼
<pre>{ "email": "yassinemana@gmail.com", "ipn1": "1247N524", "ipn2": "557N4414", "ipn3": "1627M1201", "ipn4": "", "name": "Yassine Ranaï", "password": "1bb65d92ac03a4c39af048087d9f9b", "phoneNumber": "930144027", "_id": "12d" }</pre>			
Execute			

Figure133. Tester les différents API : Ajouter un utilisateur

- Se connecter à l'application mobile :

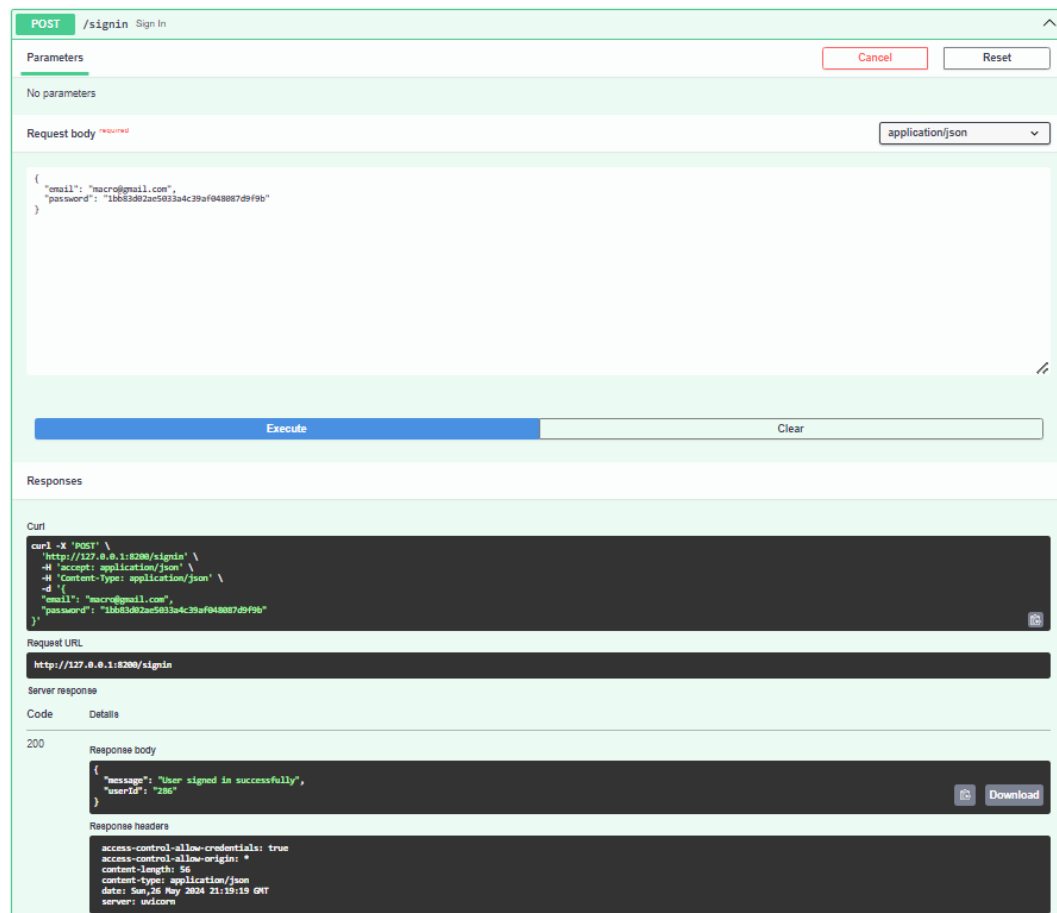


Figure134. Tester les différents API : Se connecter à l'application mobile

- Ajouter un invité :

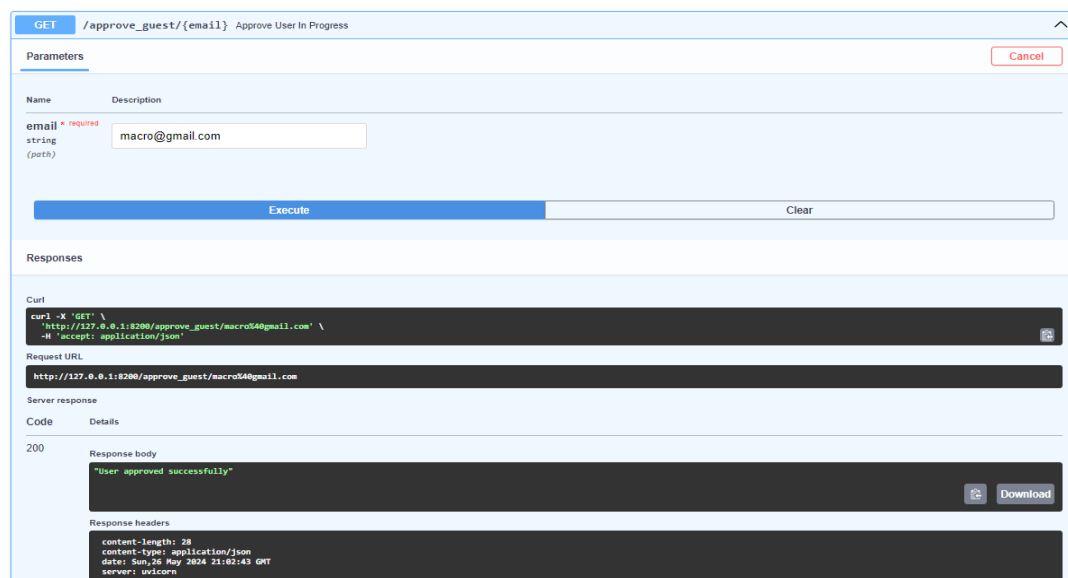


Figure135. Tester les différents API : Ajouter un invité

- Lister invité(s) :

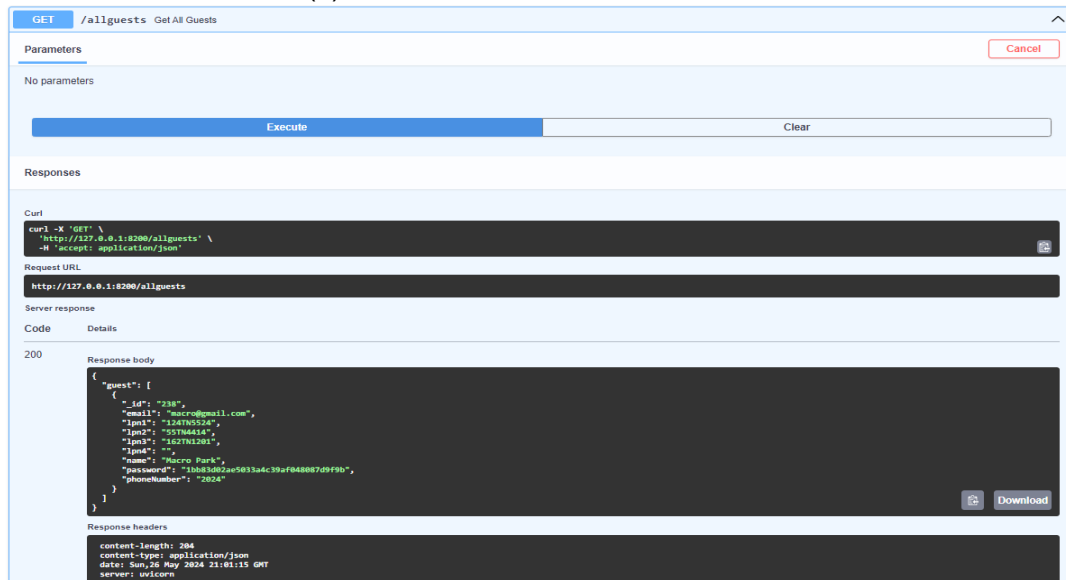


Figure136. Tester les différents API : Lister invité(s)

- Accepter invité :

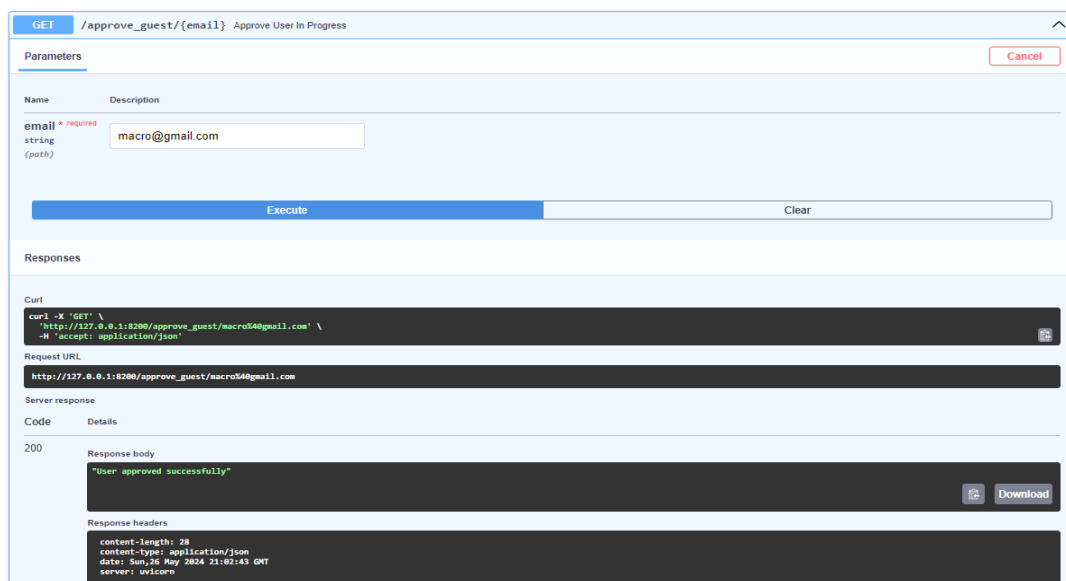


Figure137. Tester les différents API : Accepter invité

- Ajouter une barrière :

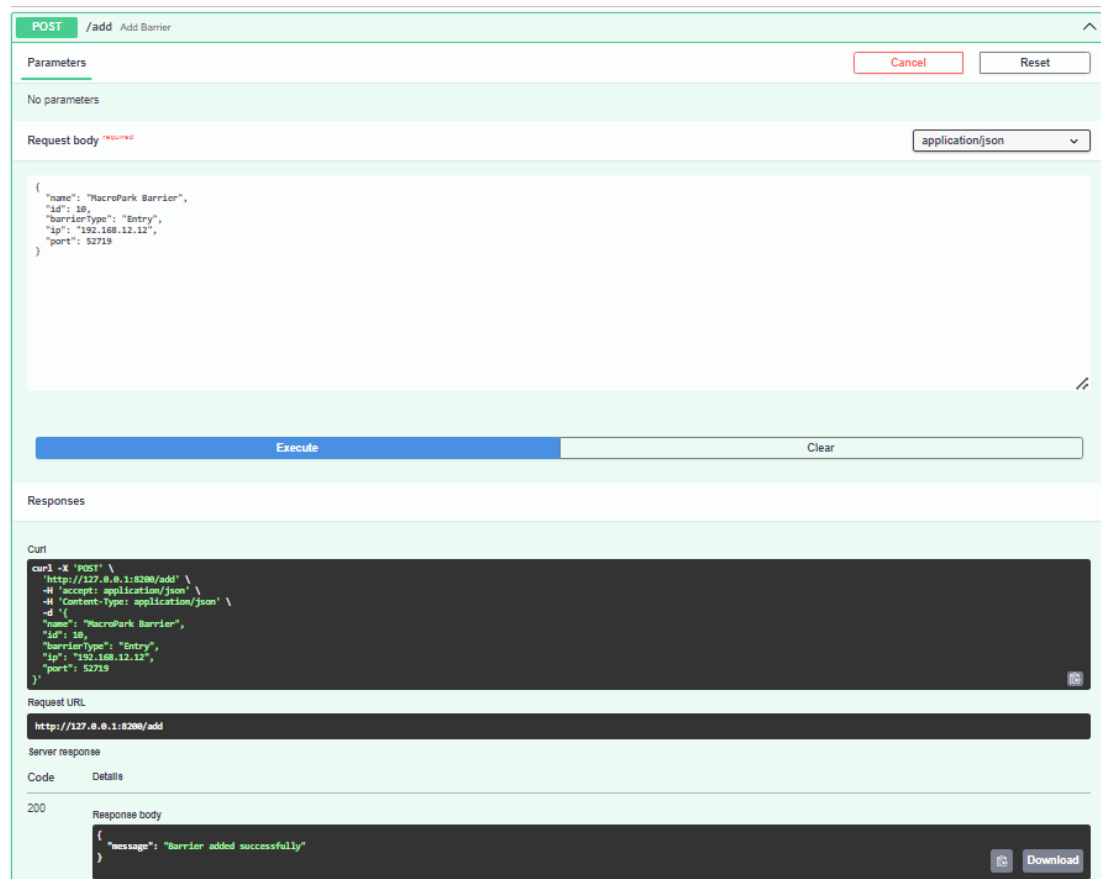


Figure138. Tester les différents API : Ajouter une barrière

- Lister toutes les barrières :

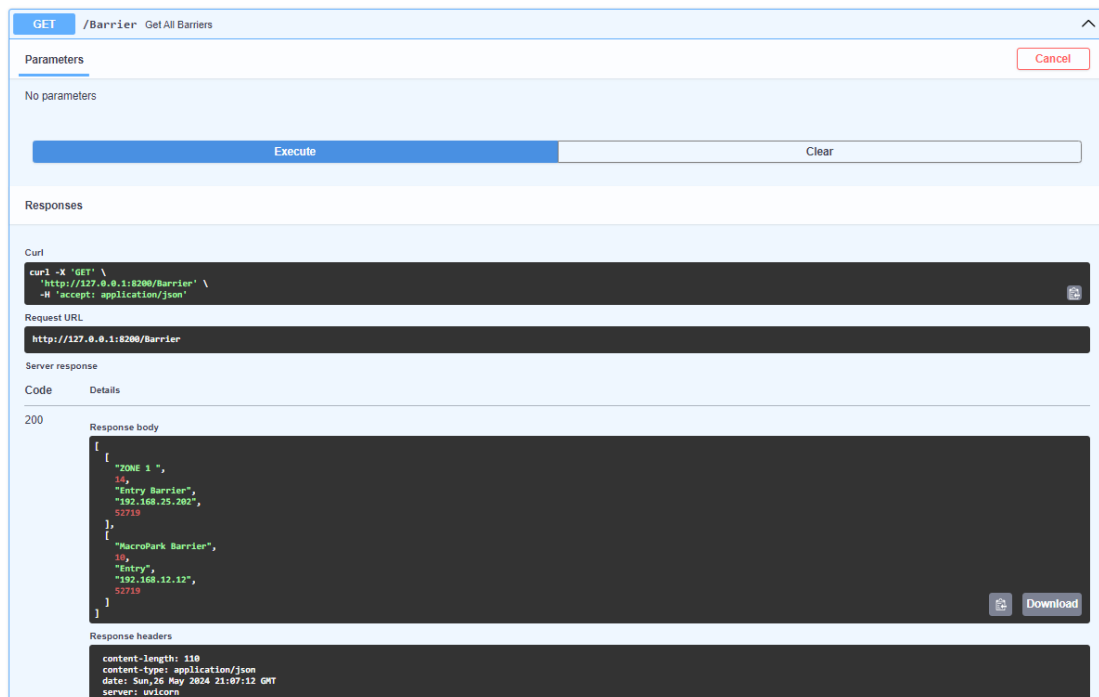


Figure139. Tester les différents API : Lister toutes les barrières

- Lister l'historique des actions pour les barrières :

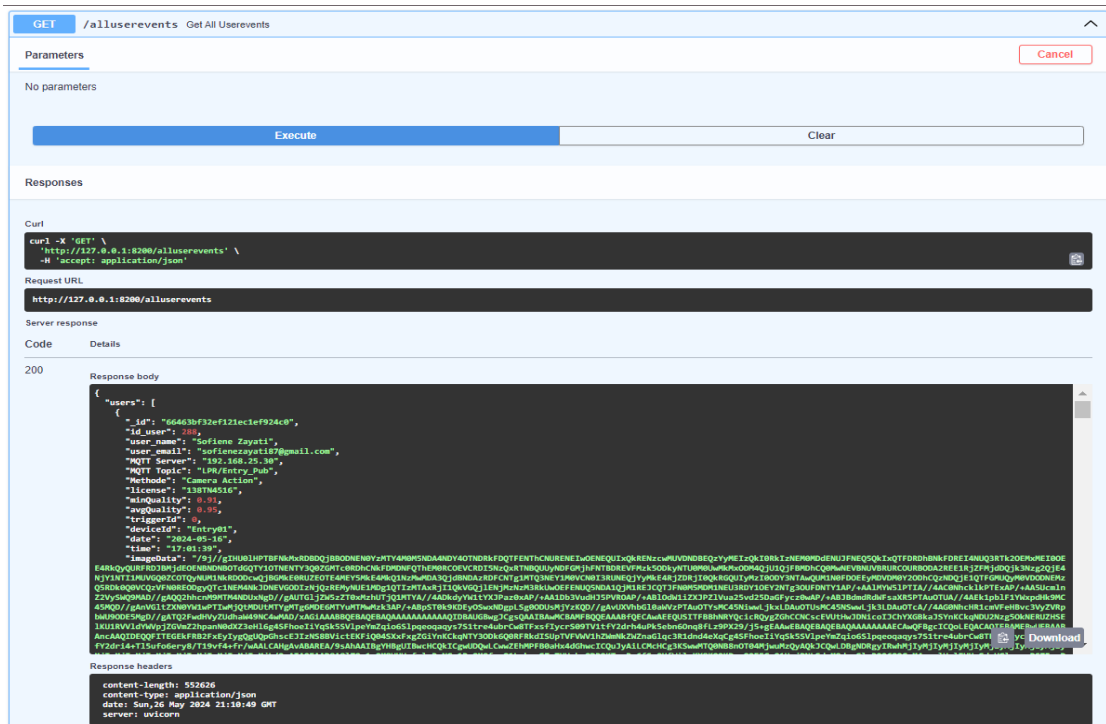


Figure140. Tester les différents API : Lister l'historique des actions pour les barrières

- Lister l'historique des entrées au parking :

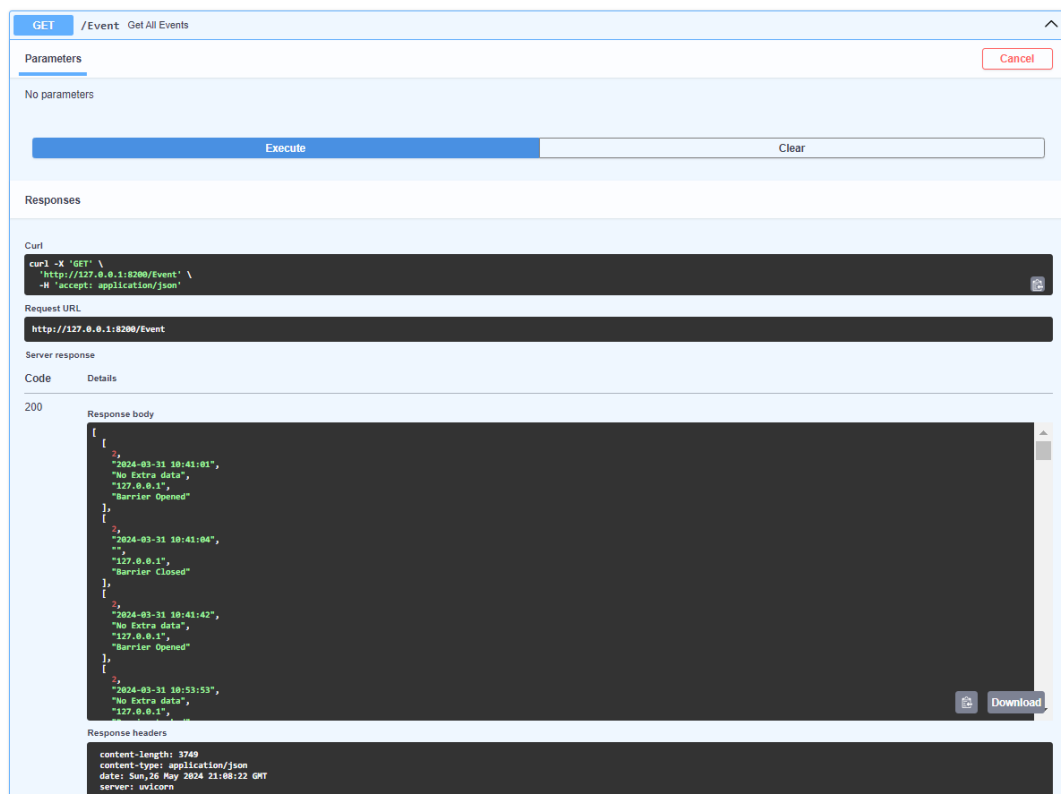


Figure141. Tester les différents API : Lister l'historique des entrées au parking

- Lister l'historique par date (filtrer) :

GET /usersLogs/{date} Find Users By Date

Parameters

Cancel

Name	Description
date * required	
string	2024-05-16
(path)	

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8200/usersLogs/2024-05-16' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8200/usersLogs/2024-05-16

Server response

Code Details

200

Response body

```
{
  "users": [
    {
      "id": "66463f72ef121ec1ef924c1",
      "id_user": 280,
      "user_name": "Sofiane Zayati",
      "user_email": "sofinmezayati87@gmail.com",
      "MQTT Server": "192.168.25.38",
      "MQTT Topic": "LRR/Entry_Pub",
      "Methode": "Phone Action",
      "License": "No Licence",
      "minQuality": 0,
      "avgQuality": 0,
      "triggerId": 0,
      "deviceId": "Entry01",
      "date": "2024-05-16",
      "time": "17:18:47"
    },
    {
      "id": "66463c122ef121ec1ef924c1",
      "id_user": 280,
      "user_name": "Sofiane Zayati",
      "user_email": "sofinmezayati87@gmail.com",
      "MQTT Server": "192.168.25.38",
      "MQTT Topic": "LRR/Entry_Pub",
      "Methode": "Camera Action",
      "License": "1387M4516",
      "minQuality": 0,
      "avgQuality": 0,
      "triggerId": 0,
      "deviceId": "Entry01",
      "date": "2024-05-16",
      "time": "17:18:47"
    }
  ]
}
```

Response headers

```
content-length: 53532
content-type: application/json
date: Sun, 26 May 2024 21:13:04 GMT
server: unicorn
```

GET /usersLogs/{date} Find Users By Date

Parameters

Cancel

Name	Description
date * required	
string	2024-05-28
(path)	

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8200/usersLogs/2024-05-28' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8200/usersLogs/2024-05-28

Server response

Code Details

404

Undocumented

Error: Not Found

Response body

```
{
  "detail": "Logs not found"
}
```

Response headers

```
content-length: 27
content-type: application/json
date: Sun, 26 May 2024 21:14:38 GMT
server: unicorn
```

Figure142. Tester les différents API : Lister l'historique par date (filtrer)

4.2. Test de l'application mobile

Pour la partie mobile, nous avons rencontré quelques problèmes tels que l'envoi des signaux BLE. Voici une capture d'écran de l'application (AndroidManifest.xml) localisé dans ce chemin (**/projet/android/app/src/main/androidManifest.xml**).\$

```
android > app > src > main > AndroidManifest.xml
1  <manifest xmlns:android="http://schemas.android.com/apk/res/android">
2
3      <uses-feature android:name="android.hardware.bluetooth_le" android:required="true"/>
4
5      <uses-permission android:name="android.permission.INTERNET" />
6      <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
7
8      <uses-permission android:name="android.permission.BLUETOOTH"/>
9      <uses-permission android:name="android.permission.BLUETOOTH_ADMIN"/>
10     <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
11     <uses-permission android:name="android.permission.BLUETOOTH_ADVERTISE"/>
12     <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
13
14     <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
15     <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
16
```

Figure143. Capture d'écran du code *androidManifest.xml*

La bibliothèque `beacon_broadcast` nécessite l'autorisation et l'utilisation de la recherche des appareils locaux, et dans notre code, nous l'avons définie. Cela montre un problème où il faut l'activer manuellement. Pour résoudre cela, nous avons écrit un petit code qui vérifie au démarrage de l'application que toutes les permissions nécessaires sont validées. Sinon, une fenêtre s'affiche pour les activer.

Pour résoudre les problèmes liés à l'envoi des signaux BLE et s'assurer que toutes les permissions nécessaires sont accordées, nous devons vérifier et demander les permissions appropriées lors du démarrage de l'application. Nous utiliserons la bibliothèque `permission_handler` pour gérer les permissions.

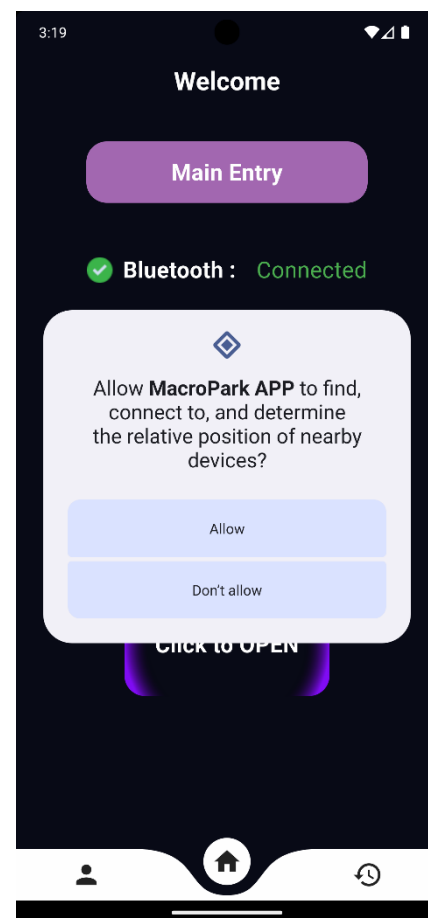


Figure 144. Dialogue de permission (Application mobile)

```
void requestPermissions() async {  
    bool permGranted = false;  
  
    // Request permissions  
    var statuses = await [  
        Permission.location,  
        Permission.bluetoothAdvertise,  
    ].request();  
  
    // Check if all permissions are granted  
    if (statuses[Permission.location]!.isGranted &&  
        statuses[Permission.bluetoothAdvertise]!.isGranted ) {  
        permGranted = true;  
    }  
    else  
    {  
        print('Access Denied');  
        showAlertDialog(context);  
    }  
    print('Permissions granted: $permGranted');  
}
```

4.3. Exécution du projet

Pour exécuter notre projet, nous avons utilisé Docker (conteneurisation du projet) afin de pouvoir l'utiliser sur différentes machines et endroits sans rencontrer de problèmes ou avoir besoin de remplacer les dépendances liées au projet.

Au cours du développement de MacroPark, nous avons dû exécuter trois serveurs : Backend Server, CoreServer et Bridge Server, et les avons également chargés dans Docker Hub (un espace permettant de partager les images pour faciliter la communication entre les développeurs). Ceci une capture d'écran de la page Docker Hub.

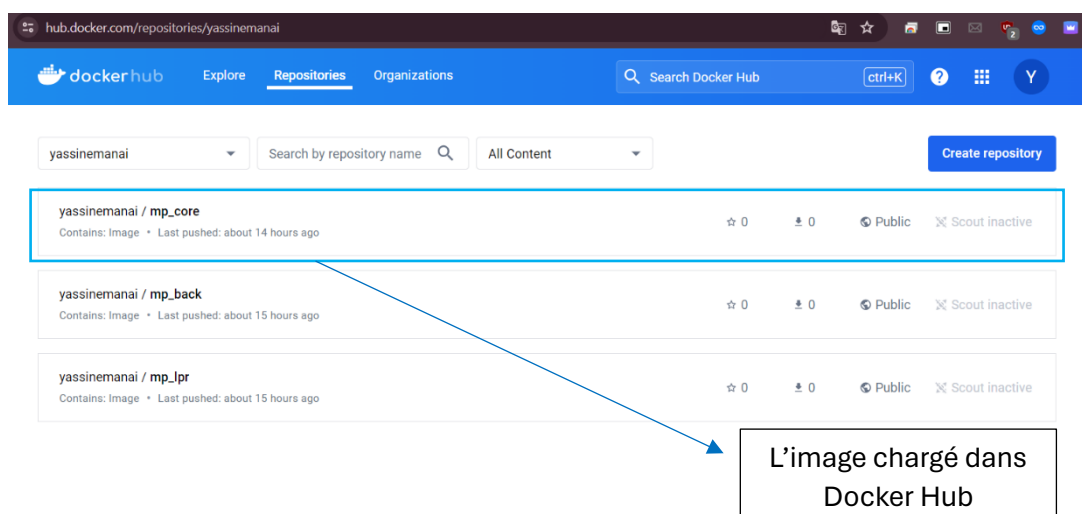


Figure 145. Docker Hub

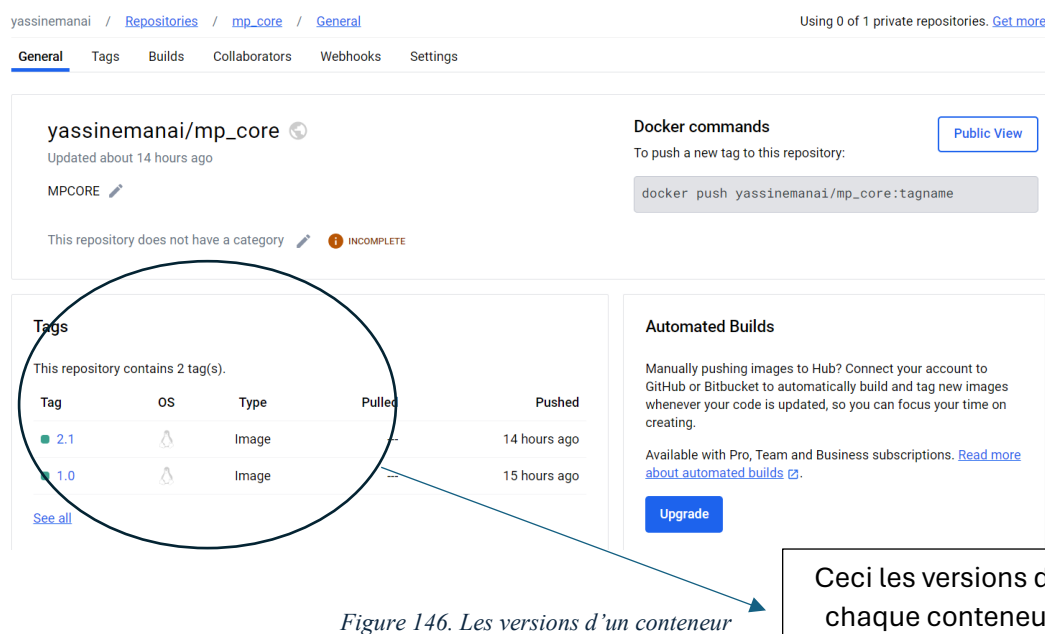


Figure 146. Les versions d'un conteneur

Cependant, nous avons rencontré un défi majeur lors de l'exécution du projet MacroPark : pour garantir son bon fonctionnement, il était impératif d'activer simultanément les trois serveurs principaux (Core, Bridge, Backend) ainsi que le serveur MQTT. Cette exigence complexifiait considérablement le processus de déploiement, nécessitant plusieurs commandes distinctes pour chaque serveur.

Pour remédier à ce problème, nous avons trouvé une solution : l'utilisation de Docker Compose.

Docker Compose est un outil qui permet de définir et de gérer des applications multi-conteneurs Docker[9]. En regroupant tous les composants du projet dans un fichier de configuration unique, nous avons pu exécuter l'ensemble du système en une seule commande, simplifiant ainsi considérablement le processus de déploiement.

Le fichier Docker Compose associé au projet MacroPark définit les images des serveurs ainsi que les ports attribués à chaque conteneur. De cette manière, chaque service est déployé de manière cohérente et prévisible, garantissant une exécution sans faille du projet dans divers environnements. Cette approche centralisée nous a permis de gagner un temps précieux lors du déploiement et de minimiser les risques d'erreurs liées à la configuration manuelle des serveurs.

```
version: '3'
services:
  CORE_SERVER:
    image: yassinemanai/mp_core:2.3
    ports:
      - "8280:8280"
    environment:
      - APP_IP = 127.0.0.1
      - APP_PORT = 8280
      - MQTT_SERVER = 192.168.1.13
      - MQTT_PORT = 1883
      - MQTT_TOPIC_PUB= LPR/Entry_Pub
      - MQTT_TOPIC_SUB= LPR/Entry_Sub
      - MQTT_TOPIC_OPEN= WT32/access
      - MQTT_TOPIC_UID= WT32/auth
      - VERIF_URL = /lpns/Parking/verifier_lpn
      - VERIF_URL_ESP = /users/Parking/ESPuser
      - Log_Url = /reports/Parking/UploadAction

  LPR_SERVER:
    image: yassinemanai/mp_lpr:2.0
    ports:
      - "8180:8180"
    environment:
      - APP_IP=192.168.25.59
      - APP_PORT=8180
      - Cam_username=admin
      - Cam_password=quercus2
      - Cam_id=Entry01
      - MQTT_SERVER = 192.168.1.13
      - MQTT_PORT=1883
      - MQTT_TOPIC_PUB=LPR/Entry_Pub

  BACKEND_SERVER:
    image: yassinemanai/mp_back:3.0
    ports:
      - "8200:8200"

  mqtt5:
    image: eclipse-mosquitto
    container_name: mqtt5
    restart: always
    ports:
      - "1883:1883"
      - "9001:9001"
    volumes:
      - ./mosquitto.conf:/mosquitto/config/mosquitto.conf:rw
      - ./data/mosquitto/data:/mosquitto/data:rw
      - ./data/mosquitto/log:/mosquitto/log:rw
```

Figure 147. code Docker-compose.yaml

L'exécution du projet se fait avec la commande suivante :

```
C:\test_api\Serveurs>docker-compose up
time="2024-06-09T20:02:10+01:00" level=warning msg="C:\\test_api\\Serveurs\\docker-compose.yml: 'version' is obsolete"
[+] Running 4/4
✔ Container serveurs-LPR_SERVER-1      Created           0.0s
✔ Container serveurs-CORE_SERVER-1     Created           0.0s
✔ Container serveurs-BACKEND_SERVER-1  Created           0.0s
✔ Container mqtt5                      Recreated         0.6s
```

Figure 148. Exécution du Docker compose file

docker-compose up

Figure 149. Création des Conteneurs dans Docker

```
PORTS  POSTMAN CONSOLE  TERMINAL  COMMENTS  DEBUG CONSOLE  PROBLEMS
✓ TERMINAL

CORE_SERVER-1 | TimeoutError: timed out
CORE_SERVER-1 exited with code 1

Gracefully stopping... (press Ctrl+C again to force)
[+] Stopping 1/0
- Container mqtt5 Stopping
[+] Stopping 1/4veurs-LPR_SERVER-1 Stopping
- Container mqtt5 Stopping
[+] Stopping 1/4veurs-LPR_SERVER-1 Stopping
- Container mqtt5 Stopping
[+] Stopping 1/4veurs-LPR_SERVER-1 Stopping
- Container mqtt5 Stopping
[+] Stopping 1/4veurs-LPR_SERVER-1 Stopping
- Container mqtt5 Stopping
[+] Stopping 2/4veurs-LPR_SERVER-1 Stopping
✓ Container mqtt5 Stopped
[+] Stopping 2/4veurs-LPR_SERVER-1 Stopping
✓ Container mqtt5 Stopped
[+] Stopping 2/4veurs-LPR_SERVER-1 Stopping
✓ Container mqtt5 Stopped
[+] Stopping 2/4veurs-LPR_SERVER-1 Stopping
✓ Container mqtt5 Stopped
[+] Stopping 2/4veurs-LPR_SERVER-1 Stopping
✓ Container mqtt5 Stopped
[+] Stopping 3/4veurs-LPR_SERVER-1 Stopping
✓ Container mqtt5 Stopped
[+] Stopping 4/4veurs-LPR_SERVER-1 Stopped
✓ Container mqtt5 Stopped
✓ Container serveurs-LPR_SERVER-1 Stopped
✓ Container serveurs-CORE_SERVER-1 Stopped
✓ Container serveurs-BACKEND_SERVER-1 Stopped
```

Figure 150. Arrêt du Docker

5. Test Hardware

Dans cette section, nous allons présenter les différents tests effectués pour vérifier le bon fonctionnement de notre système, illustrés par des captures d'écran du moniteur série. Ces tests démontrent le processus de configuration et d'exploitation de notre dispositif.

5.1. Configuration WiFi Initiale

Lorsque notre microcontrôleur WT32 ETH01 ne trouve aucun réseau WiFi enregistré dans sa mémoire, il ouvre un point d'accès (AP) pour permettre la connexion à un réseau WiFi. Voici une capture d'écran du moniteur série illustrant ce processus :

- **Recherche de Réseau WiFi** : Le dispositif recherche les réseaux WiFi enregistrés.
- **Ouverture du Point d'Accès** : En l'absence de réseaux enregistrés, il ouvre un point d'accès nommé "Legacy".
- **Connexion et Configuration** : À travers ce point d'accès, nous pouvons nous connecter et configurer les paramètres réseau tels que l'IP statique, la passerelle, le masque de sous-réseau, ainsi que les configurations du serveur MQTT (adresse du serveur, port, utilisateur, mot de passe).

Cette approche assure que le dispositif est toujours accessible pour la configuration initiale ou les ajustements futurs, même si les paramètres réseau changent ou sont perdus.

```
20:48:39.545 -> *wm:AutoConnect
20:48:39.545 -> *wm:STA IP set: 192.168.25.200
20:48:39.545 -> *wm:No wifi saved, skipping
20:48:39.545 -> *wm:AutoConnect: FAILED for 9 ms
20:48:39.545 -> *wm:StartAP with SSID: Legacy
20:48:40.094 -> *wm:AP IP address: 192.168.4.1
20:48:40.094 -> *wm:Starting Web Portal
```

Figure151. Access Point Serial Monitor

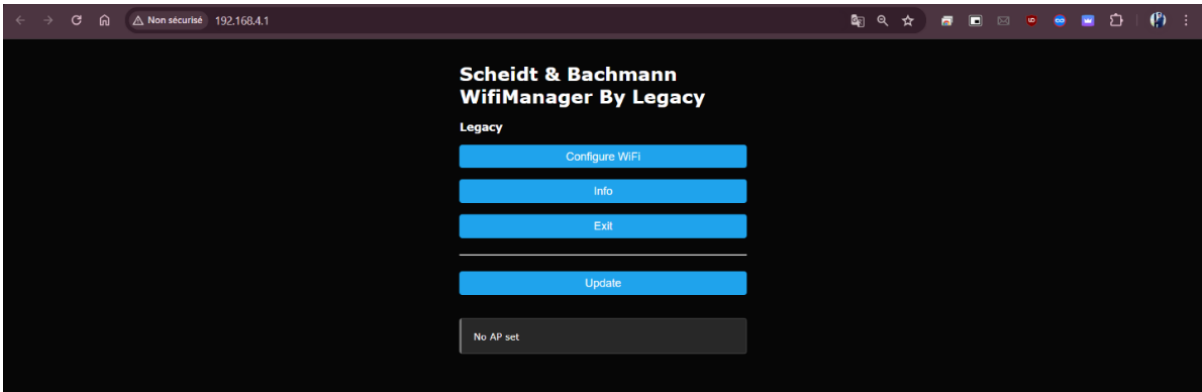


Figure 152. Interface graphique - Accueil

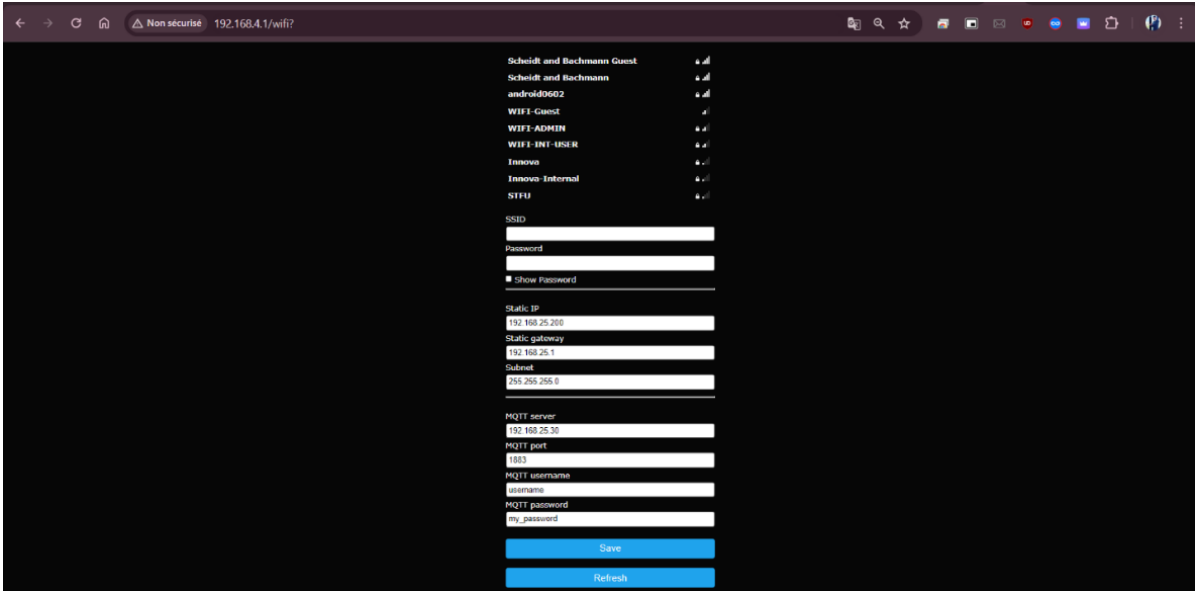


Figure 153. Interface graphique - Config

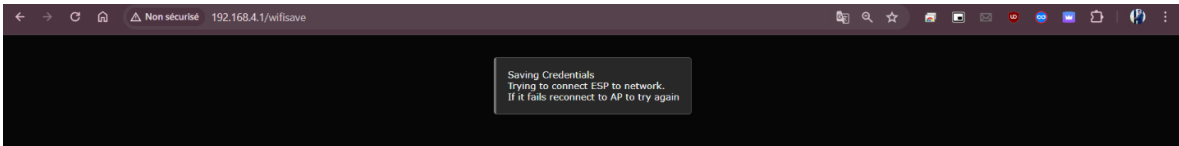


Figure 154. Interface graphique - Submit

5.2. Fonctionnement Normal après Configuration

Après avoir entré les détails de configuration via le point d'accès, le dispositif commence son fonctionnement normal. Il recherche activement les périphériques BLE spécifiques ou attend les messages MQTT. Voici une capture d'écran du moniteur série illustrant ce processus :

- **Connexion Réussie** : Le dispositif se connecte au réseau WiFi avec les paramètres fournis.
- **Configurations Appliquées** : L'adresse IP, la passerelle et le masque de sous-réseau sont affichés pour vérification.
- **Serveur MQTT** : Connexion réussie au serveur MQTT avec les détails configurés (adresse du serveur, port, utilisateur, mot de passe).
- **État de Veille BLE/MQTT** : Le dispositif entre en mode de surveillance active, où il recherche les périphériques BLE spécifiques ou attend les messages du serveur MQTT.

Cette capture d'écran démontre la capacité du système à basculer automatiquement du mode de configuration à son état opérationnel, garantissant ainsi une transition fluide et une opération fiable.

5.3. Réaction aux Messages MQTT après Approbation de la Plaque d'Immatriculation

Lorsque la caméra approuve une plaque d'immatriculation, un message MQTT est envoyé au dispositif pour action. Voici une capture d'écran du moniteur série montrant la réaction du dispositif après réception d'un message MQTT :

- **Réception du Message** : Le dispositif reçoit le message MQTT contenant les informations d'approbation depuis le topic WT32/access.
- **Validation de la Plaque** : Le message est analysé pour vérifier la validité de la plaque d'immatriculation approuvée.
- **Action Appropriée** : En fonction du message reçu, le dispositif effectue l'action correspondante, telle que l'ouverture de la barrière.
- **Confirmation et Journalisation** : Une fois l'action réalisée, une confirmation est envoyée et l'événement est enregistré dans les logs pour référence future.

Cette capture d'écran démontre l'efficacité et la réactivité du système à traiter les messages MQTT en temps réel, assurant une gestion fluide et sécurisée des accès.

```
20:46:15.083 -> Received order to open
20:46:15.126 -> Received access message: open
20:46:15.126 -> B1OPEN OPEN
20:46:20.220 -> B1OPEN CLOSED
20:46:20.313 -> Resetting access message...
```

Figure 155. Relay Opened With MQTT

5.4. Réaction à l'Ouverture avec BLE et Envoi de Message MQTT

Lorsque le dispositif détecte une présence via BLE (Bluetooth Low Energy), il effectue les actions suivantes :

- **Détection via BLE** : Le dispositif identifie un utilisateur autorisé à proximité en scannant les périphériques BLE.
- **Ouverture de la Barrière** : Après validation de l'utilisateur, la barrière s'ouvre automatiquement.
- **Envoi de Messages MQTT** : Un message MQTT est envoyé au serveur central sur le topic WT32/auth contenant le DeviceID, l'userID.

Voici une capture d'écran du moniteur série illustrant cette séquence :

```
18:29:57.505 -> Scan done! Devices found: 6
18:29:57.505 -> 24 1 190 172 248 38 36 64 1 34 51 68 5 9 156 57 143 193 153 210 0 1 0 100 197 0
18:29:57.550 -> 2440
18:29:57.550 -> Published device information to MQTT: {"DeviceID": "WT32", "userID": "2440"}
18:29:57.550 -> Barrier 1 Opened
18:29:58.560 -> Duration : 5000
18:30:01.655 -> Barrier 1 Closed after Opening
```

Figure 156. Relay Opened With BLE

- **DeviceID et UserID** : Le message contient les identifiants uniques du dispositif et de l'utilisateur pour un suivi précis.
- **Durée de l'Ouverture** : La durée pendant laquelle la barrière reste ouverte est également incluse dans le message.

Cette capture d'écran démontre la capacité du système à intégrer les technologies BLE et MQTT pour une gestion d'accès efficace et sécurisée, tout en maintenant une traçabilité complète des événements.

5.5. Réception de l'ID d'Entreprise via MQTT et Modification du Filtre de Scan BLE

Le dispositif WT32 est configuré pour recevoir des paramètres de configuration, notamment l'ID d'entreprise, via le sujet MQTT WT32/config. Ce processus permet de modifier dynamiquement les filtres de scan BLE selon les besoins spécifiques de l'entreprise.

- **Réception de l'ID d'Entreprise** : Lorsque le WT32 reçoit un message sur le sujet WT32/config, il extrait l'ID d'entreprise inclus dans ce message.
- **Modification du Filtre de Scan BLE** : Le filtre de scan BLE est mis à jour pour inclure uniquement les périphériques qui correspondent à cet ID d'entreprise, garantissant que seuls les dispositifs autorisés peuvent interagir avec le système.
- **Enregistrement dans SPIFFS** : L'ID d'entreprise reçu est également enregistré dans le système de fichiers SPIFFS du WT32, assurant ainsi que la configuration est conservée même après un redémarrage.
- **Application des Changements** : Les changements de configuration sont appliqués immédiatement, permettant une adaptation rapide et flexible des paramètres du système sans nécessiter de redémarrage manuel ou d'intervention physique.

- Voici une capture d'écran montrant ce processus en action

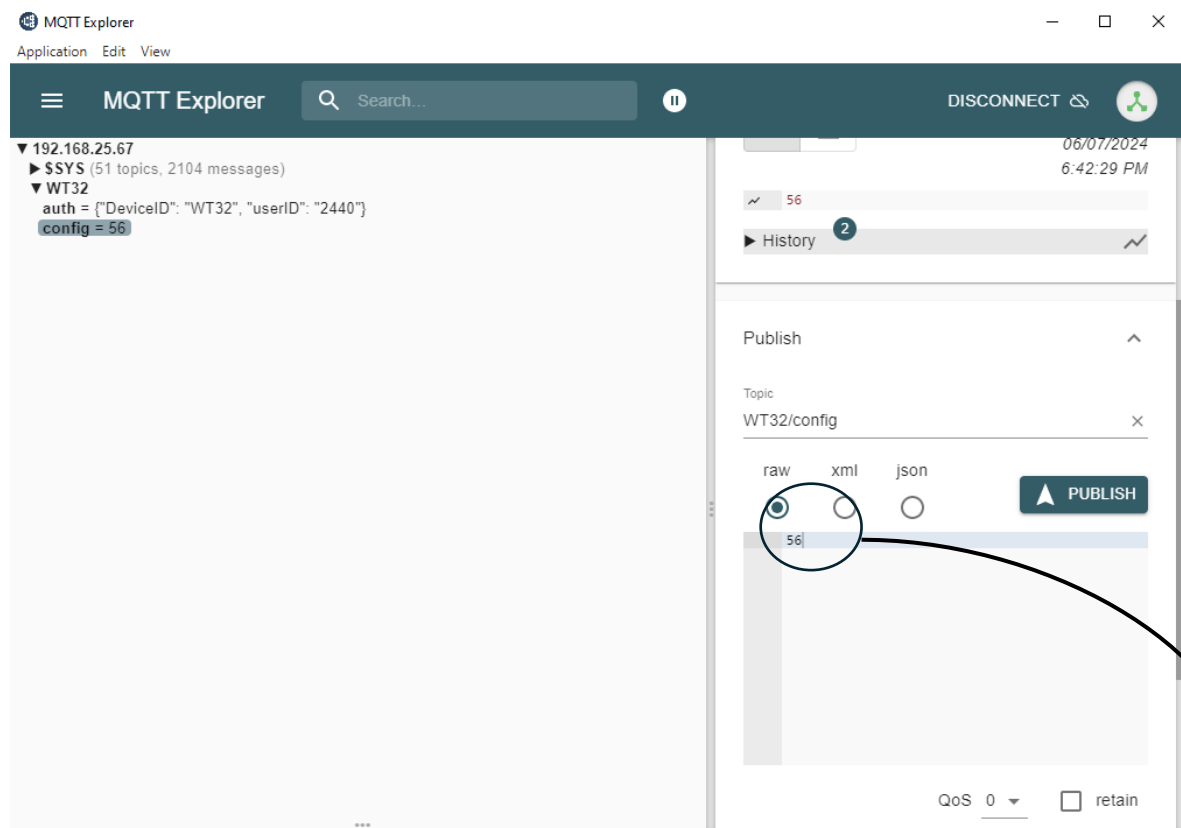


Figure 157. Mqtt Explorer

:

```
18:42:29.713 -> Received Company ID
18:42:29.713 -> Company ID: 56
```

Figure 158. Company ID Scan

- **ID d'Entreprise** : L'ID reçu est affiché, montrant que le dispositif a correctement reçu et traité la configuration.
- **Mise à Jour du Filtre BLE** : Le log montre la confirmation de la mise à jour du filtre de scan BLE, assurant une reconnaissance précise des périphériques autorisés.

Ce mécanisme permet une gestion flexible et centralisée des configurations, améliorant l'efficacité et la sécurité du système de contrôle d'accès. En enregistrant l'ID d'entreprise dans le SPIFFS, le système garantit la persistance des configurations importantes, même en cas de coupure de courant ou de redémarrage.

6. Sprint Review

Notre sprint de test et validation pour le contrôleur de barrière a été crucial pour garantir la qualité et la fiabilité du système. Nous avons effectué des tests approfondis sur la détection automatique des véhicules et l'interface utilisateur, identifiant et corrigeant plusieurs bugs critiques.

7. Perspective

Les retours des parties prenantes ont confirmé que nous étions sur la bonne voie, mais ont également souligné des domaines nécessitant des améliorations. En perspective, nous nous concentrerons sur l'optimisation des performances et l'amélioration de l'expérience utilisateur.

8. Conclusion

Ce sprint de test et validation a été crucial pour garantir la qualité du projet MacroPark. En utilisant différents outils, nous avons testé et corrigé les erreurs efficacement, assurant la fiabilité de l'interface administrateur, de l'application mobile, de la caméra et du contrôleur de barrière. Cela a renforcé la robustesse du système en vue de son déploiement final.

CONCLUSION GENERALE

Le projet MacroPark, intégrant la reconnaissance des plaques d'immatriculation (LPR), une application mobile, une interface web d'administration et un contrôleur de barrière, a atteint ses objectifs en transformant la gestion des parkings.

Grâce à la technologie LPR, l'entrée des véhicules s'est automatisée, réduisant les temps d'attente et augmentant la sécurité. Le contrôleur de barrière a joué un rôle crucial en garantissant un accès fluide et sécurisé aux installations.

L'application mobile a offert aux utilisateurs une commodité sans précédent, leur permettant d'ouvrir la barrière en toute sécurité.

L'interface web d'administration a facilité la gestion centralisée et efficace des opérations, offrant des outils pour la gestion et le contrôle du parking.

En conclusion, MacroPark a réussi à moderniser la gestion des parkings, améliorant à la fois l'expérience utilisateur et l'efficacité opérationnelle, et se positionne comme une solution innovante et fiable dans le domaine de la gestion de stationnement.

De plus, ce projet possède un potentiel significatif pour l'avenir, car il peut être intégré dans des initiatives de ville intelligente, offrant des solutions avancées pour la gestion urbaine et contribuant à la création de villes plus connectées et efficaces.

BIBLIOGRAPHIE et NETOGRAPHIE

- [1]. https://files.seeedstudio.com/products/102991455/WT32-ETH01_datasheet_V1.1-%20en.pdf (06/02/2024)
- [2]. <https://github.com/egnor/wt32-eth01> (14/03/2024)
- [3]. <https://github.com/tzapu/WiFiManager> (21/03/2024)
- [4]. <https://en.wireless-tag.com/product-item-2.html> (24/03/2024)
- [5]. Original instructions, D-ID: V1_0 –04.20 ELKA BARRIERS (10/03/2024)
- [6]. <https://docs.python.org/3/library/venv.html> (06/02/2024)
- [7]. <https://swagger.io/tools/swagger-ui/> (20/02/2024)
- [8]. Access User Manual Quercus SmartLPR (29/04/2024)
- [9]. <https://mosquitto.org/> (01/05/2024)
- [10]. <https://www.docker.com/> (20/05/2024)
- [11]. Forum Arduino
- [12]. <https://github.com/ldijkman/WT32-ETH01-LAN-8720-RJ45->

ملخص

تعتبر مواقف السيارات الذكية ضرورية لتطوير المدن المستدامة والسلامة الحضرية. فهي تعمل على تحسين المساحة وتقليل الازدحام، وبالتالي تحسين نوعية الحياة وكفاءة البنية التحتية. يؤدي تكامل التقنيات مثل إنترنت الأشياء والتعرف على لوحات تعريف السيارات إلى تعزيز الأمان وإمكانية التتبع، مما يوفر حماية وإدارة أفضل للبيانات. تجعل هذه التطورات التكنولوجية أنظمة مواقف السيارات أكثر كفاءة وانتشارًا، مما يساهم في السلامة والإدارة الفعالة لأماكن وقوف السيارات.

يتكون مشروعنا من تطوير نظام لإدارة مواقف السيارات يعتمد على تقنيات إنترنت الأشياء والتعرف على لوحات السيارات بواسطة كاميرات المراقبة. ولتسهيل استخدام النظام بشكل أكبر، تم أيضًا تطوير تطبيق للهاتف المحمول كجزء من هذا المشروع.

Abstract

Smart parking is essential for the development of sustainable cities and urban safety. It optimizes space and reduces congestion, thereby improving quality of life and infrastructure efficiency. The integration of technologies like IoT and license plate recognition strengthens security and traceability, providing better data protection and management. These technological advances promise to make parking systems even more efficient and widespread, contributing to the safety and efficient management of parking spaces.

Our project consists of developing a parking management system based on Internet of Things technologies and the recognition of automobile registrations by surveillance cameras. To further facilitate the use of the system, a mobile application is also developed as part of this project.

Résumé

Le parking intelligent est essentiel pour le développement de villes durables et la sécurité urbaine. Il optimise l'espace et réduit la congestion, améliorant ainsi la qualité de vie et l'efficacité des infrastructures. L'intégration de technologies comme l'IoT et la reconnaissance de plaques d'immatriculation renforce la sécurité et la traçabilité, offrant une meilleure protection et gestion des données. Ces avancées technologiques promettent de rendre les systèmes de parking encore plus performants et répandus, contribuant à la sécurité et à la gestion efficace des espaces de stationnement.

Notre projet consiste à développer un système de gestion de parking qui s'appuie sur les technologies de l'Internet des Objets et la reconnaissance des immatricules des automobiles par caméras de surveillances. Pour faciliter plus encore l'utilisation du système, une application mobile est aussi développée dans le cadre de ce projet.